Kurdistan Region-Iraq
Kurdistan Regional Government
Ministry of High Education and Scientific Research
Sulaimani Polytechnic University



SMS Spam Detection System Using Hybrid Gradient Boosting with Wrapper Algorithms

A Thesis
Submitted to Council of Technical College of Informatics
Sulaimani Polytechnic University in Partial Fulfillment of the
Requirements for the Degree of Master of Science in
Information Technology

By

Diyari Jalal Mussa
B. Sc. Statistics and Computer – Sulaimani University (2015)

Supervised by

Dr. **Noor Ghazi M. Jameel**
Assistant Professor

February 2020 **Reşemî 2720**

Supervisor Certification

I certify that this thesis entitled "**SMS Spam Detection System Using Hybrid Gradient Boosting with Wrapper Algorithms**" was prepared by **Diyari Jalal Mussa**, under my supervision at Sulaimani Polytechnic University/ Technical College of Informatics, in partial fulfillment of the requirements for the degree of Master of Science in Information Technology.

Signature:

Name: Assist. Prof. **Dr. Noor Ghazi M. Jameel**

(Supervisor)

Date: / /2020

In view of the available recommendation, I forward this thesis for debate by the examining committee.

Signature:

Name: Assist. Prof. **Dr. Alaa Khalil Jumma**

(Head of Department)

Date: / / 2020

Certificate of Proofreading and Editing

This is to certify that this thesis entitled **"SMS Spam Detection System Using Hybrid Gradient Boosting with Wrapper Algorithms"** prepared by the student **(Diyari Jalal Mussa)** has been proofread and edited for appropriate writing style, grammar, punctuation and spelling. Therefore, after making all the required corrections and considering the comments by the student for further improvement, I confirm that this last copy of the thesis is ready for submission.



Signature:

Name: Dlnya Raouf Muhealddin

Qualification: MA in English Studies: Professional Applications and Intercultural Communication

Date: 20.04.2020

Examining Committee Certification

We certify, as an examining committee, that we have read this thesis entitled "**SMS Spam Detection System Using Hybrid Gradient Boosting with Wrapper Algorithms**", examined the student "**Diyari Jalal Mussa**" in its contents and found it meets the standard of a thesis for the degree of Master of Science in Information Technology.

Signature:

Name: **Dr. Soran A. Mohammed**

Title: **Professor**

Date: 15 / 7 / 2020

(Chairman)

Signature:

Name: **Dr. Omar Y. Abdulhammeed**

Title: **Assistant Professor**

Date: 15 / 7 / 2020

(Member)

Signature:

Name: **Dr. Miran Taha Abdullah**

Title: **Lecturer**

Date: 15 / 7 / 2020

(Member)

Signature:

Name: **Dr. Noor Ghazi M. Jameel**

Title: **Assistant Professor**

Date: 15 / 7 / 2020

(Supervisor - Member)

Approved by the Dean of the Technical College of Informatics Signature:

Name: **Dr. Ronak Mohammed Abdullah**

Title: **Lecturer**

Date: 15 / 7 / 2020

(Dean)

Dedication

This thesis is dedicated to:

My late Brother

My beloved family

My supervisor

And

Anyone who supported me

Acknowledgements

Firstly, endless thanks to Allah who helped me and gave me the capability to accomplish this study. I would like to express my gratitude to my thesis supervisor Assistant Professor Dr. Noor Ghazi for the beneficial notes, comments, and appointments through the learning process of this master thesis. Also, I would like to thank her for introducing me to the topic as well for the support on the way. Special Thanks to my wife, my mother, my father, and brother for their support and encouragement. I would like to thank anyone, who has helped me during the process of this thesis. I will be grateful forever for your support.

Abstract

In the modern world, the mobile phone has become a necessary part of every human life by providing a variety of services to its users, which makes it more likable. Short Message Service (SMS) is one of the vital communication services provided for mobile phone users. The good usage of this service leads to different benefits to the user. However, the widespread usage of such a service attracted attackers to perform mobile phone spam, which is a sort of spam directed via SMS. SMS spam is often considered annoying and even hazardous to recipients because it lures recipients to open web addresses for illicit activities like spreading malware apps and stealing personal information. In this study, the main intention is to detect spam and ham messages as accurately as possible, proposing gradient boosting machine learning algorithms with wrapper feature selection methods. A number of structural features were collected from previous studies besides suggesting new ones, aiming to find the feature set that best identifies the problem because one of the most important factors for improving the performance of machine learning is relevant features. The features are categorized into three groups; 18 binary, 8 numeric, and a group of both types of features. Additionally, each extracted feature is based on the structure and content of spam and ham messages such as promotions, award-winning, capital letters, and etc. For optimal feature selection, two sorts of wrapper selection methods are compared and tuned with different sorts of boosting algorithms. Sequential Backward Selection (SBS) method leads to better accuracy comparing to Sequential Forward Selection (SFS). The three algorithms, Gradient Boosting Machine (GBM), XGBoost and LightGBM are compared according to accuracy and time of classification using 10-fold cross-validation. LightGBM algorithm scored the highest accuracy of 99.139% with 9 features selected by SBS comparing to XGBoost and GBM when their best features are selected. It takes less time for classification comparing to XGBoost and GBM. The proposed approach is assessed with a dataset containing 747 SMS spam messages and 4827 normal SMS messages. According to the results, suggested features in this study such as, currency symbol, service numbers and currency as a word are relevant features when accuracy is 99.121% with 10 features using LightGBM classifier, and 99.103% using XGBoost classifier. However, only currency symbol and service numbers considered relevant features when LightGBM scored its best accuracy 99.139% with 9 features.

List of Contents

Acknowledgements i

Abstract ii

List of Contents iii

List of Figures v

List of Tables vii

List of Abbreviations ix

List of Symbols x

Chapter One: General Introduction

1.1 Introduction 1

1.2 Problem Statement 2

1.3 Literature Review 2

1.4 Aim of the Thesis 6

1.5 Thesis Layout 6

Chapter Two: Theoretical Background

2.1 Introduction 8

2.2 Short Message Service (SMS) 8

2.3 SMS Problems 10

2.4 SMS Spam Concept 11

2.5 Effects of SMS Spam 13

2.6 Feature Extraction 14

2.7 Feature Selection 15

2.7.1 Feature Selection for Classification 17
2.7.2 Main Approaches for Feature Selection 18

2.8 Sequential Selection Algorithms 21

2.9 K-Fold Cross-Validation 23

2.10 Boosting Algorithms 25

2.10.1 Gradient Boosting Machine (GBM) 26
2.10.2 Extreme Gradient Boosting (XGBoost) 27
2.10.3 Light Gradient Boosting Machine (LightGBM) 29

2.11 Model Evaluation 30

2.11.1 Confusion Matrix 31
2.11.2 Classification Metrics 32

Chapter Three: The Proposed System

3.1 Introduction 33

3.2 Design and Implementation 34

3.3 Preprocessing 35

3.3.1 Importing Libraries 35
3.3.2 Preparation of the Dataset 36
3.3.3 Handling Missing Data in the Dataset 37
3.3.4 Categorical Data Encoding 38
3.3.5 Feature Extraction 39

3.4 Feature Selection Using Sequential Wrapper Feature Selector 55

3.5 Classification of Spam and Ham Messages 57

3.5.1 GBM Classifier 57
3.5.2 XGBoost Classifier 58
3.5.3 LightGBM Classifier 59

Chapter Four: Experiments and Results

4.1 Introduction 62

4.2 Involved Parameters of Sequential Feature Selection Methods 63

4.3 Involved Parameters of Gradient Boosting Algorithms 64

4.4 Training and Testing Using 10-Fold Cross-Validation 65

4.4.1 Confusion Matrix for Spam Filtering 65

4.5 Test Results for Features Category Selection 66

4.6 Test Results for Determining Accurate Feature Selection Method 67

4.6.1 SFS Using Each Gradient Boosting Algorithm 68
4.6.2 SBS Using Each Gradient Boosting Algorithm 71
4.6.3 Comparison between SFS and SBS Results According to Each Algorithm 75

4.7 Test Results of Optimal Feature Set 77

4.7.1 GBM with Optimal Feature Set 78
4.7.2 XGBoost with Optimal Feature Set 79
4.7.3 LightGBM with Optimal Feature Set 80
4.7.4 Comparison between Gradient Boosting Algorithms Results 82

4.8 Determining Relevant Features 84

4.9 Proposed System’s Comparative Study 85

Chapter Five: Conclusions and Suggestions for Future Work

5.1 Conclusions 86

5.2 Suggestions for Future Work 87

References 88

List of Figures

Figure (2.1)	Conventional Network Architecture for SMS	9
Figure (2.2)	User Interface of SMS Gateway	10
Figure (2.3)	Examples of SMS Spam Messages	12
Figure (2.4)	Phases of Phishing Attack	14
Figure (2.5)	Feature Selection Flow in General	17
Figure (2.6)	Wrapper Algorithms of Feature Selection for Classification General Framework	19
Figure (2.7)	5-Fold Cross-validation Process	24
Figure (2.8)	General Way of Boosting Algorithm Process	26
Figure (2.9)	Creating Ensemble Learning of XGBoost	28
Figure (2.10)	Working of LighGBM Algorithm in General	29
Figure (3.1)	Block Diagram of the Proposed System	34
Figure (3.2)	SMS Spam Corpus Statistics	36
Figure (3.3)	The Flow Chart of SMS Spam Corpus Preparation	37
Figure (3.4)	Flowchart of Essential Preprocessing Phases	38
Figure (3.5)	Essential Preprocessing Phases	39
Figure (3.6)	Extraction of Binary Features using Proposed System Software	49
Figure (3.7)	Extraction of Numeric Features using Proposed System Software	51
Figure (3.8)	General Feature Extraction Flow Chart	54
Figure (3.9)	Extraction of All Features using Proposed System Software	55
Figure (3.10)	Flowchart of Sequential Feature Selection Method	56
Figure (3.11)	Proposed System GUI for Sequential Feature Selection Methods	57
Figure (3.12)	Gradient Boosting Classifiers Evaluation using Selected Feature Set	61
Figure (4.1)	Evaluation of Three Gradient Boosting Classifiers using Different Categories of Features	67
Figure (4.2)	Accuracy Score of Different Feature Sets using GBM Regarding SFS	69
Figure (4.3)	Accuracy Score of Different Feature Sets using XGBoost Regarding SFS	70
Figure (4.4)	Accuracy Score of Different Feature Sets using LightGBM Regarding SFS	71
Figure (4.5)	Numbers of Features Accuracy Score using GBM Regarding SBS	72
Figure (4.6)	Numbers of Features Accuracy Score using XGBoost Regarding SBS	73
Figure (4.7)	Numbers of Features Accuracy Score using LightGBM Regarding SBS	75
Figure (4.8)	GBM's Most Accurate Feature Set of SBS and SFS Comparison	76
Figure (4.9)	XGBoost's Most Accurate Feature Set of SBS and SFS Comparison	76
Figure (4.10)	LightGBM's Most Accurate Feature Set of SBS and SFS Comparison	77

Figure (4.11) Evaluation Metric Scores Regarding Best Feature Sets using GBM Classifier 78
Figure (4.12) GBM Classification Time Scores of Best Feature Subsets 79
Figure (4.13) Evaluation Metric Scores Regarding Best Feature Sets using XGBoost Classifier 80
Figure (4.14) XGBoost Classification Time Scores of Best Feature Subsets 80
Figure (4.15) Evaluation Metric Scores Regarding Best Feature Sets using LightGBM Classifier 81
Figure (4.16) LightGBM Classification Time Scores of Best Feature Subsets 82
Figure (4.17) Comparison between Gradient Boosting Classifiers Performance 83
Figure (4.18) Comparison between Gradient Boosting Classifiers Regarding Classification Time 83

List of Tables

Table (2.1)	Confusion matrix in Classification Problem with Two Classes	31
Table (3.1)	Libraries used in The Proposed System	35
Table (3.2)	Instance of Messages and Categories in The Dataset used in This Study	37
Table (3.3)	Description of Extracted Feature	53
Table (4.1)	Hardware and Software Specifications of The Computer used for The Experiments in This Study	63
Table (4.2)	Parameter Names and Their Default Values Regarding Gradient Boosting Classifiers	64
Table (4.3)	Confusion Matrix	66
Table (4.4)	GBM Accuracy Result of Different Categories of Features	66
Table (4.5)	XGBoost Accuracy Result of Different Categories of Features	66
Table (4.6)	LightGBM Accuracy Result of Different Categories of Features	67
Table (4.7)	Accuracy Score of Selected Features in Each Iteration using SFS algorithm and GBM classifier	68
Table (4.8)	Accuracy Score of Selected Features in Each Iteration According to SFS algorithm and XGBoost classifier	69
Table (4.9)	Accuracy Score of Selected Features in Each Iteration According to SFS algorithm and LightGBM classifier	70
Table (4.10)	Accuracy Score of Selected Features in Each Iteration According to SBS depending on GBM Classifier	72
Table (4.11)	Accuracy Score of Selected Features in Each Iteration According to SBS depending on XGBoost Classifier	73
Table (4.12)	Accuracy Score of Selected Features in Each Iteration According to SBS depending on LightGBM Classifier	74
Table (4.13)	Comparison between SBS and SFS Accuracy using GBM Classifier	75
Table (4.14)	Comparison between SBS and SFS Accuracy using XGBoost Classifier	76
Table (4.15)	Comparison between SBS and SFS Accuracy using LightGBM Classifier	77
Table (4.16)	Comparing GBM Performance via Four Best Feature Subsets	78
Table (4.17)	Comparing GBM Classifier According to Classification Time and Feature Type	78
Table (4.18)	Comparing XGBoost Performance via Four Best Feature Subsets	79
Table (4.19)	Comparing XGBoost Classifier According to Classification Time and Feature Type	79
Table (4.20)	Comparing LightGBM Performance via Four Best Feature Subsets	81
Table (4.21)	Comparing LightGBM Classifier According to Classification Time and Feature Type	81
Table (4.22)	Comparison among Algorithms using Accuracy and Evaluation metrics	82

Table (4.23)	Comparison among Algorithms using Classification Time	83
Table (4.24)	Optimal Feature Set of the Proposed Model	84
Table (4.25)	Optimal Feature Set of XGBoost and GBM classifiers	84
Table (4.26)	Comparison with the Most Relevant Study	85
Table (4.27)	Comparative Study with the Most Recent Studies	85

List of Abbreviations

Abbreviation	Description
AUC	Area Under Curve
BoW	Bag of Words
CAP	Cumulative Accuracy Profile
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DoS	Denial-of-Service
EMS	Enhanced Messaging Service
ESME	External Short Message Entities
FN	False Negative
FP	False Positive
GBM	Gradient Boosting Machine
GUI	Graphical User Interface
HLR	Home Location Register
IDE	Integrated Development Environment
k-NN	K-Nearest Neighbors
LightGBM	Light Gradient Boosting Machine
Mlxtend	Machine Learning Extensions
MMS	Multimedia Messaging Service
MNB	Multinomial Naïve Bayes
MO	Mobile Originating
MSC	Mobile Switching Center
MT	Mobile Terminating
PCA	Principal Component Analysis
regex	Regular Expression
ROC	Receiver Operating Characteristics
SBFS	Sequential Backward Floating Section
SBS	Sequential Backward Selection
SF	Structural Features
SFFS	Sequential Forward Floating Selection
SFS	Sequential Forward Selection
SMS	Short Message Service
SMSC	Short Message Service Center
SS7	Signaling System 7
SVM	Support Vector Machine
TF-IDF	Term Frequency - Inverse Document Frequency
TN	True Negative
TP	True Positive
URL	Uniform Resource Locator
VLR	Visitor Location Register
XGBoost	Extreme Gradient Boosting

List of Symbols

Symbol	Description
€	Euro
¥	Yen
$	Dollar
£	Pound Sterling
₣	French Franc
₹	Indian Rupee
₩	Korean Won
₦	Nigerian Naira

Chapter One
General Introduction

1.1 Introduction

In the age of wireless communication, one of the simplest and inexpensive ways of communications is Short Message Service (SMS). SMS has become one of the most popular telecommunication services around the world because of its security, affordability, and high responsivity. However, there are certain problems that mobile phone users come up with, such as spam messages. Now, to define spam texts, it is safe to say spamming is the process of sending bulk messages to mobile devices either randomly or targeting specific groups of phone numbers. Spam messages can have different forms. For instance, users may be asked to reply to an email or a phone number that does not have any expected identity or lead to anything that relates it to the sender. In some scenarios, spam messages may contain free services, advertisements, prize winnings, and so on. The reason behind these scenarios are for manipulating the users to hand over personal information, tricking economic purposes or for political advancements [1][2].

Recipients begun to notice the increase of unwanted messages sent to their mobile phones in early 2000s. By the end of 2003, according to [3], the number of SMS spam messages that users received was greater than the number of email spam in Korea. Moreover, according to the Chinese National Spam Report Center, in 2014 mobile phone users in China have received an average of 12 spam messages per week, and the number of spam messages have reached 120 million in total. In many countries especially like India, Pakistan, and China, SMS spam messages have caused a disaster where SMS spam missing effective supervision [1]. The increase in spam problems led the researchers to develop different sorts of countermeasures in order to tackle SMS spam messaging problems. For instance, researchers used black and white list techniques, but they were not enough to efficiently control spam messages. In addition, there are some apps in both IOS and Android operating systems that filter messages. However, not too many people are aware of them [4][5]. Other than mobile applications, there are some filtering methods which concentrate mostly on email spam because it is a problem that is older than SMS

spam [6]. Therefore, the popularity of mobile phone devices and using SMS technology widely all over the world made researchers more interested about the topic in recent years, with respect to the last decade.

1.2 Problem Statement

SMS services motivated spammers for many reasons. One of the reasons is the limited availability of mobile phone applications for SMS spam detection and filtering, lack of laws and regulations to control the purchase of phone numbers in some countries, and the availability of low-cost solutions that send bulk SMS messages effortlessly. SMS spam has caused mobile users and mobile network operators several problems. Certain types of SMS spam try to bill mobile users by tricking them into calling premium-rate numbers to subscribe to expensive services, or tricking the users into calling specific numbers to collect their personal and confidential information. Moreover, the victims of SMS spam can be exposed to some risks by accessing harmful websites or downloading malware.

Furthermore, mobile network providers have their share of this issue as well. For example, they are prone to lose their subscribers because the signaling performance of the network can be degraded by the load that SMS spam generates. This issue may also have some severe impacts on the providers' profit because they cannot receive a termination fee as some types of SMS spam are sent from fraudulent addresses and spend more on spam prevention. Additionally, improving the accuracy of filtering these sorts of messages is one of the most challenging tasks in this field of study, as there are many research worker has proposed different techniques and methods to tackle this problem. In contrast, not many of them have taken time as a practical measure with accuracy. As a result, the necessary calls to make some useful techniques to improve the accuracy of SMS spam detection is needed besides considering time.

1.3 Literature Review

Nowadays, various approaches for SMS spam detection have been proposed. For instance, there are various techniques and methods out there which lead to the

development of Android applications to separate spam messages from normal ones by using different sorts of classification algorithms and other techniques. In this section, reviewing of the most relevant studies will be discussed according to their dataset, feature extraction, feature selection and machine learning algorithms used.

Almeida et al., 2011 [7] offered a new message dataset containing both spam and non-spam messages. The dataset was real, non-encoded and made it publically available. They presented statistics concerning the proposed dataset as tokens frequencies while an approach regarding duplicate messages was proposed as well. The data corpus created with subgroups of text messages was obtained from the same sources. Moreover, they compared some learning algorithms in their study such as (Support Vector Machine (SVM), k-nearest neighbors (k-NN), Naive Bayes, C4.5, etc.). Accordingly, the outcome they achieved was acquired by the combination of classifier models and tokens. Then, they confirmed that SVM is the best model for classification, scoring 97.64% of accuracy for filtering spam and normal messages among the other used algorithms in their study.

Yadav et al., 2011 [8] created mobile phone based system called SMS Assassin that can detect SMS spam texts based on Bayesian learning algorithm and blacklisting techniques, which concerned with the sender of the message. The dataset used in their study is called India Centric Real World SMS Spam Dataset. Their dataset was acquired over two and a half months, and it has more than 4000 messages. In addition, the set of attributes used in their study included 20 fine structured features. The dataset was split into 2000 text messages for training purposes and the remaining includes 2318 messages for testing. The test set contained 1123 spam and 1185 normal text messages while the training set included 1000 spam and 1000 ham messages. The machine learning algorithms that they used for classifying were SVM and Bayesian learning algorithm. According to their proposed method, the recipients that received a text message in their mobile phones, SMS Assassin automatically grabs the text message without letting the user to be disturbed, the system therefore decides which texts to show and which to hide, all is based on their server. The detected spam message automatically will be in spam folder whenever it is reported as spam. The highest accuracy scored in their study reached 97% for ham and 72.5% for spam when Bayesian learning algorithm applied.

Nuruzzaman et al., 2012 [9] evaluated cell phones performance regarding SMS spam detection by using text classification techniques. According to their proposed study on android mobile devices, SMS spam can be filtered with having satisfactory accuracy, least storage utilization, and moderate processing time without receiving support from a computer nor do they rely on massive servers for training the system in order to identify SMS spam. In their conclusion, they proposed that detection of spam messages can be done successfully on individual mobile devices. They also decrease the number of word variables by nearly 50% without reducing accuracy accordingly, using usability-based technique.

Uysal et al., 2013 [10] observed the effects of various sorts of feature extraction and selection methods on spam text messages for English and Turkish languages. The English dataset contains 425 spam and 450 normal text messages while the Turkish SMS spam dataset was collected by volunteers and contains 420 spam and 430 normal messages. Moreover, six structural features (SF) were extracted besides features originated form the Bag of Words (BoW) model. For classifying text messages, various combinations of BoW and Structural Features were used, and then they created a list of feature sets. The results show that the combination of both BoW and SF provides better outcome for classification rather than only BoW feature.

Ahmed et al., 2014 [11] merged Naïve Bayes classifier and Apriori algorithm in order to filter spam and normal messages by presenting a hybrid approach for text message classification. The dataset used in their study was collected from UCI Machine learning repository "SMS Spam collection Data Set" which contains 5574 text messages of spam and normal messages. As for the extracted features, they created vector of features from word attributes. The presented system has obtained levels of accuracy that can beat Naïve Bayes classifier algorithm. The improvement attained by their proposed system reached 98.7% of accuracy while Naïve Bayes classifier accuracy alone reached 97.4% only.

Akbari et al., 2015 [12] used boosting technique for spam detection problem and discovered that boosting models are fine options between other types of classifiers when the classification dataset is unbalanced. The dataset, used in their study, is collected from UCI Machine learning repository namely "SMS Spam collection Data Set" which contains 4,825 normal and 747 spam text messages. The features that were used were word variables and elimination of unused features that result in fewer

attributes without any effect on efficiency and accuracy. Thus, 32 optimal word attributes were extracted. When it comes to classification, in most scenarios when classification is binary with having an unbalanced dataset, Gentleboost is preferable Boosting Classifiers because it performs better than other algorithms compared in their study.

Zheng et al., 2016 [13] proposed a system using Adaboost algorithm as a learning classifier for filtering spam text messages in Chinese language. They used a dataset containing 800,000 SMS messages, including 720,000 ham and 80,000 spam text messages. Improving the performance of final classification decision is obtained via introducing three content-based weak filters. In their proposed approach, they did not used accuracy for performance evaluation, the attained results were from Receiver Operating Characteristics (ROC). The approach had some advantages, for instance, higher efficiency and less parameter over those established SMS spam filtering methods. Moreover, the proposed approach can handle SMS spam classification with straightforward data processing and training parameters that are small in number.

Choudhary et al., 2017 [14] presented new technique that can detect and determine SMS Spam messages using machine learning. They studied features of spam messages broadly and found 10 optimal structured features according to their analysis. They used dataset which consists of 2608 text messages, 2408 collected from publically available SMS spam corpus and 200 messages collected manually containing 25 spam and 175 ham messages. Moreover, different machine learning algorithms such as (Naïve Bayes, Logistic regression, J48, etc.) scored various outcome, while Random Forest classification algorithm scored best, results as the following 96.5% true positive rate and 1.02% false positive rate.

Wang et al., 2018 [15] contributed HIT SMS Spam Corpus which is a new categorized SMS spam dataset. It contains total of 13,078 messages divided into 4,204 spam and 8874 normal text messages. According to their approach, reducing the feature sparse problem in the process of SMS spam classification can be obtained throughout their system. In addition to that, non-linguistic variables were reserved and for improving the quality of tokens some replacement rules were used in their presented approach. They also compare their presented approach with three classifiers using the created spam messages dataset. In the results of the experiments,

they concluded that their study is presenting an effective method for spam detection. However, the dataset they proposed is not available publically.

Popovac et al, 2018 [16] used Convolutional Neural Network (CNN) leaning method for filtering spam and ham SMS messages. In their study, Tiago's dataset was used which is imbalanced dataset because nearly %87 of the dataset is ham messages. Moreover, Extraction of features applied using Term Frequency - Inverse Document Frequency (TF-IDF) matrix, which is created as a result of converting text to matrix. In conclusion, they presented CNN as SMS spam classifier by scoring 98.4% of accuracy and 0.95% as Area Under Curve (AUC).

Sjarif et al, 2019 [17] proposed methods of TF-IDF and Random Forest learning algorithm for filtering spam and ham messages. The dataset used in their study known as Tiago's dataset including 5574 text messages. Feature extraction of TF-IDF used with different learning algorithms such as Multinomial Naïve Bayes (MNB), KNN, SVM, Decision Tree and Random Forest. As a result, Random Forest scores 97.50% of accuracy which outperforms other algorithms used in their study.

1.4 Aim of the Thesis

The main purpose of this work is to design and implement a new, fast and intelligent detecting system that can detect SMS spam using an appropriate machine learning approach based on a set of optimal SMS features. The most significant feature is that this system is focused on SMS structure rather than textual feature. This system is using an appropriate feature selection algorithm to select optimal set of features for accurate and fast detection of SMS spam. Then the system is compared with other approaches in the field of study.

1.5 Thesis Layout

In this section, a short description of the remaining parts of this thesis is explained and organized as follow:

- **Chapter Two:** It contains the background theory, and relevant concepts of SMS messaging, problem of spam and its effects. It also includes theoretical

approaches of the methods that are used in detection and classification of SMS spam filtration.

- **Chapter Three:** This chapter includes the implementation of the proposed system in detail containing extraction of features, selection using wrapper approach, and classification of spam and non-spam messages using different boosting algorithms.
- **Chapter Four:** For assessing the proposed system performance a number of tests have been conducted. After that, the outcome of the experimental tests are illustrated and quantified in tables for comparison purposes.
- **Chapter Five:** It includes the derived conclusions and suggestion for future work.

Chapter Two
Theoretical Background

2.1 Introduction

This chapter covers the background theory related to the main subjects of the thesis. The core topics of this thesis initiate from mobile communication through messaging service which is primarily a digital network provision that permits digital phone users to send or receive short texts on their cell phones. Then introducing spam concept and the problems that mobile users might face because of the different sorts of spam messaging techniques. The basics of feature selection and its types will be discussed and various types of boosting algorithms, which will be used as a model classifier for filtering spam and ham messages.

2.2 Short Message Service (SMS)

SMS essentially deals with sending and receiving short text messages from a digital device to another over the mobile networks [18]. SMS is usually limited to be no longer than 160 alphanumeric characters and should not include photos or any sort of graphics. Short Message Service Center (SMSC) stores the messages that are sent from sender mobile then forwards it to the recipient mobile device. This demonstrates that when the message is sent to the unavailable recipient, it will be delivered again at another time when the recipient is available.

Signaling System 7 (SS7) is a common channel that allows transmission of SMS messages. Primarily, it is a global standard which determines the protocols and procedures in order to exchange data between wireless telephone carriers and network element of wire line. The SS7 standard is used by network elements to exchange and control information for call setup, mobility management, routing, etc. Figure (2.1) demonstrates the structure of exemplary SMS network architecture [19]. Theoretically, the architecture of network is divided into two main parts that are core to SMS model. The first part, Mobile Originating (MO) primarily contains the sender's mobile handset, a base station which can prepare wireless communications radio infrastructure, and Mobile Switching Center (MSC) that is responsible for

switching and routing whole traffic into and out of the cellular system on behalf of the sender. The other part, Mobile Terminating (MT) contains receivers terminating MSC and a base station, in addition to store and forward server which is centralized and known as SMSC. It deals with storing and accepting text messages, forwarding messages to the available recipients, and retrieving status of the account. SMSC is supported by two databases explicitly: Home Location Register (HLR) and Visitor Location Register (VLR). These databases only register or store telephony information of the data and subscriber. Primarily, HLR stores the information permanently, while VLR is storing the data of the user who has moved from state to another temporarily [19]. Nevertheless, the popularity of SMS goes back to the high rate of exchanging text messages by cellphone users among each other, and also because of its low cost, it is used more and more by businesses in order to spread different sorts of content like ring tones, quizzes, stock price, casting of votes and news. The content providers are known as External Short Message Entities (ESMEs); they send and receive SMS across gateways that cross over SMS interface to the internet.

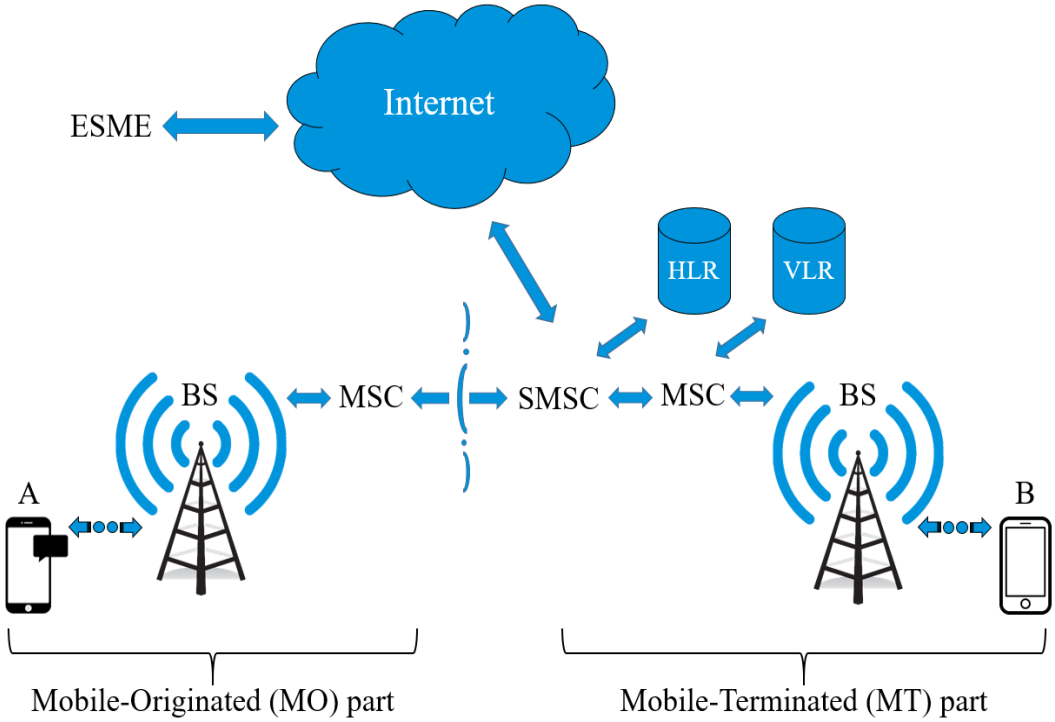


Figure (2.1) Conventional Network Architecture for SMS [19]

SMS gateway is primarily a device or account contributing SMS transit; converting text messages to cellphone network traffic from alternative media, or vice versa,

granting transmission or certificate of the text messages with or without using mobile device. The gateway uses internet in order to directly connect to SMSC. In addition, through SMS gateway, the most accelerated and reliable way is provided to send mass and bulk SMS. It is concerned with cellphone service provider that transmits short messages with the sender identity due to authentication and textual sender ID [19].

Figure (2.2) shows an example of SMS gateway called Now SMS which provides multiple services like sending SMS, Enhanced Messaging Service (EMS), Binary, Multimedia Messaging Service (MMS) and other types of messages [20].

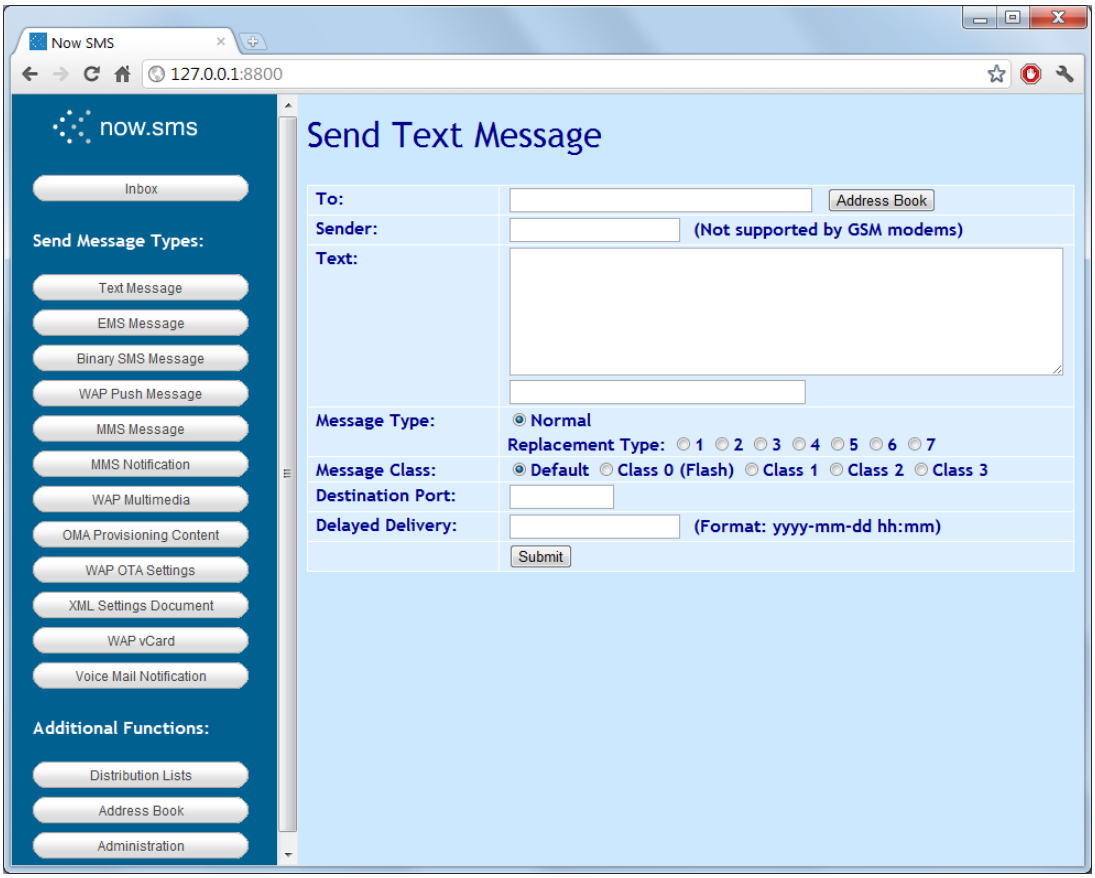


Figure (2.2) User Interface of SMS Gateway [20]

2.3 SMS Problems

SMS nowadays is one of the very widely used communication mechanism. Besides its advantages and benefits, there are some risks of using this technology that should be taken seriously. For instance, with the increasing uses of SMS mechanism for exchanging information, threats will also grow in this regard. There are some threats

that can harm the users through using SMS, for instance flooding or Denial-of-service (DoS) attacks. It can be implemented by transmitting repeated messages to a cellphone and make it unavailable. Studies show that SMS protocol can be exploited on cellular phone networks through its weaknesses. Furthermore, according to an experiment in Manhattan, it was discovered that transmitting 165 messages in a second can lead to disruption [21]. Another threat of short message is SMS phone crash, which can take an advantage of some vulnerable phones and lead them to system crash when particular sort of malformed text message is received [22].

Furthermore, SMS gateways can also be a medium for various types of malicious behavior. According to another study that was conducted on SMS gateways, the members gathered and defined the volume, lifetime and content of the traffic seen in SMS gateways. As a result, they discovered numerous malicious behaviors, containing bulk spam and phishing. Most untowardly, their data demonstrates that these systems are used to back phone-verified account frauds and other malicious acts [23].

Accordingly, SMS spam can be identified as a real problem because e-Marketers are using it for legitimate marketing convey while most of the people have annoyance of receiving those types of texts. The availability of bulk SMS propagation utilities makes it easy for nearly everyone to send out mass text messages.

2.4 SMS Spam Concept

Spam messages mainly refer to undesirable, unsolicited, and particularly commercial communications. It can also be defined as any junk text messages delivered to a mobile phone via SMS. Yet, sometimes both legitimate and spam messages can be extremely similar when legitimate message deals with advertising or bulk communication regarding the different purposes such as, event organization, political campaigns, subscriber lists, crowdfunding campaigns, religious communities, etc. Most of the people have experienced these types of advertisement and communications constantly when surfing internet either on social networking sites or in their email inboxes. Nevertheless, most of the times these messages are delivered based on services they used before or assigned into, and sometimes it can be irrelevant or even irritating. Though, spam should be constrained and charged

without costing the recipient. Furthermore, mostly the services that spread spam are deceptive to the recipient, and they sometimes contain scams and financial fraud.

Spammers nowadays are more organized than before in terms of the content of the campaigns and sending bulk messages. There are many legitimate affiliate networks that propagate legitimate products in permissible ways. Nevertheless, the type of content used for campaigns is the most important part for understanding and observing spam [24]. The most common sort of affiliate promotion observed was dating/adult space using unofficial domains, and using URL shortening services. Another sort of spam named affiliate-driven includes rogue pharmacy and 'work from home' types of campaigns that both of them also use URL shortening services. Figure (2.3) shows some real instances of how spammers express or create their spam messages.

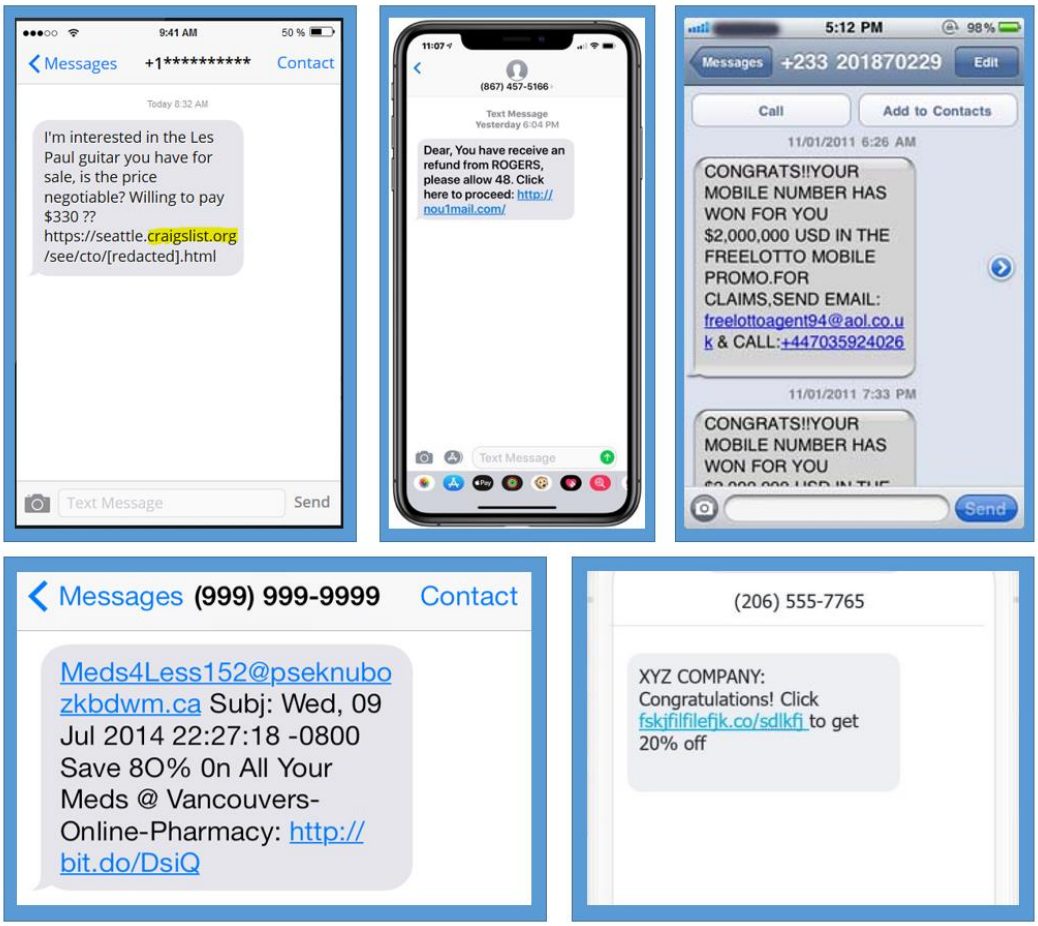


Figure (2.3) Examples of SMS Spam Messages

2.5 Effects of SMS Spam

SMS spamming has turned into a source of annoyance to cellphone users because of its pervasive nature. It requires extraordinary cost in terms of network bandwidth usage, lost productivity, management, and takeover personal confidentiality[25]. Hence, the profits of service providers will be less through SMS spamming [23][26]. Cellphone SMS spams annoy mobile subscribers similar to email spams. They lead to new social frictions to cellphone handset devices [27]. Email spams are usually delivered via internet, while mobile spams usually broadcasted via using mobile network. SMS spam is identified as an expanding threat according to the following aspects: First, available low-cost bulk SMS system. Second, low chance of receiving replies from certain unsuspecting receivers. Third, reliability (when the text message delivers to the users mobile). Fourth, personalization of the message can be done.

Moreover, SMS spam can lead to a financial loss for mobile users, besides the risks of malware downloads or phishing attacks, such as, leading to subscription towards premium rate services. Above all, phishing attacks can be categorized as a social engineering attack, most of the times, the attacker uses this technique in order to steal user's data including user login credentials and credit card numbers. The attacker masquerades as a person who is trusted, deceives the target to open instant message, email, or SMS message. The target may be deceived in order to click on the provided malicious link, which potentially leads to fake webpages or installation of malicious software that gain access to the target devices. Figure (2.4) shows the general approach of phishing attack, starting from providing communication media such as email, text message and etc. to gain access and collect victims' information by the attacker [28].

Similar to Internet phishing attack through email, with Smishing technique, attackers are pursuing to deceive cellphone subscribers with tricky text messages. Smishing makes subscribers consider the tricky messages and potentially connect to the URL or website sent to them via text message and sometimes they might download the malware into their personal mobile devices. Therefore, this will directly lead the attacker to gain access to the victim's mobile devices and use obtained information for criminal activities such as fraud or identity theft [29]. Additionally, SMS phishing or Smishing is a type of phishing attack where the attacker transmits social media messages or malicious text in order to attain confidential and valuable

information. This form is considered more hazardous and creates more security risks than the regular phishing attack in general because people are usually attached to their phones and tend to open text messages more than checking their emails.

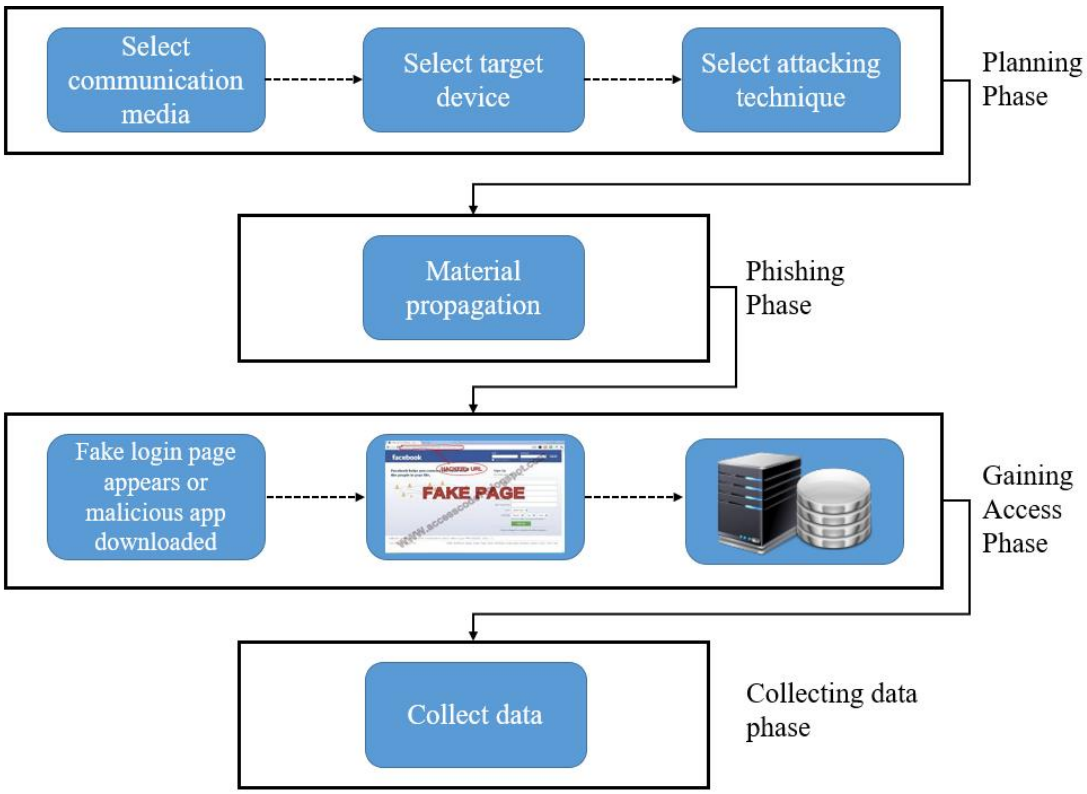


Figure (2.4) Phases of Phishing Attack [28]

2.6 Feature Extraction

Feature extraction means dealing with turning raw data into input variable that specific learning algorithm needs. Usually, for solving any machine learning problem, features must describe the information of the data in a way that fully fit the algorithm needs [30].

The problem of spam detection can be determined as an ordinary task of text classification. Thus, the structure of SMS spam messages can have significant differences with formal texts. SMS text messages' size is limited to 160 characters [31], which makes an essential feature of the text messages. In contrast, with the appearance of instant messaging and new messaging systems these days, the message length feature can be considered less valuable comparing to the past. Structure of

SMS spam messages may include the following: number of terms, uppercase letters, URL and presence of special characters.

Researchers do feature extraction in various ways. For instance, some of them are using BoW model in order to extract term frequencies [32]. In BoW model, each unique term in the text is considered as individual features. Those terms obtained certain weight which refers to their significance in the current text [33]. TF-IDF is the most common weighting system which is responsible for scaling down the quantity of each term's occurrences in the text through considering the amount of texts in the collection holding that term [34].

2.7 Feature Selection

A feature is a specific measurable characteristic of the process being discovered. Each machine learning algorithm can implement classification with using a set of features. In recent years, the domain of attributes has increased from tens to hundreds of attributes or variables regarding the applications of machine learning. Various approaches are developed to focus on the problem of reducing redundant and irrelevant variables which are difficult and challenging. Feature selection helps in reducing computation needs, understanding data, reducing dimensionality effects and increasing the predictor performance. The objective of feature selection is to choose subset of variables from the input data which leads to an efficient identification of the input during the decreasing noise or irrelevant attributes and still granting good results [35].

Extra information about the classes will not be provided by dependent variables and hence it acts as a noise for the predictor. This means that the content of the total information can be attained from less number of unique features which includes maximum bias information about the classes. Therefore, by removing the dependent variables, the amount of information can be reduced which can make improvement in the performance of the classification. In certain applications, where classes have no correlation with variables and serve as noise, it will turn to bias in the predicator besides decreasing classification performance. This may happen when there is not enough information about the process to be studied. By implementing feature selection methods some insights could be obtained to the process and the

computation requirement and prediction accuracy can be improved. In order to eliminate an irrelevant feature, a test of feature selection must be performed, which can measure the relevancy of every feature with the output label. From machine learning's point of view, if irrelevant variables used by the system, poor conclusion will occur. Moreover, when the number of features grow, straightforwardly evaluating all the feature subsets for a specific data becomes difficult [36].

Eliminating irrelevant features should not be compared to other methods of dimensionality reduction like Principal Component Analysis (PCA) [37], whereas good features could be independent from the remaining data [38]. Using the input variables itself for feature reduction does not create any new variables. Before the feature selection criterion is chosen, the process that leads in finding the subset of relevant features must be improved and developed. Therefore, a suboptimal process must be implemented that can eliminate redundant data with manageable computations.

Moreover, with the availability of numerous features, a machine learning model is likely to over-fit and lead to performance problems. Extremely high dimension data leads to serious challenges to the available learning methods such as the curse of dimensionality [39][40]. Feature selection is a broadly employed method for reducing dimensionality. It intends to select a small subset of relevant features with regard to certain equation criterion of relevancy, which mostly leads in better learning performance (e.g., higher accuracy in classification), better model interpretability, and reduced computation cost. Figure (2.5) shows the flow of feature selection in general [41].

Essentially, there are four fundamental steps a feature selection method consists of: subset generation, subset evaluation, stopping criterion, and result validation. In subset generation, a nominee feature subset will be selected based on a given search strategy, which is sent in subset generation in order to evaluate regarding certain evaluation criterion. The subset with best fitting evaluation criterion will be selected among all the candidates that have been evaluated after the stopping criterion are met. In the last step, the selected subset will be validated using the validation set or domain knowledge [42].

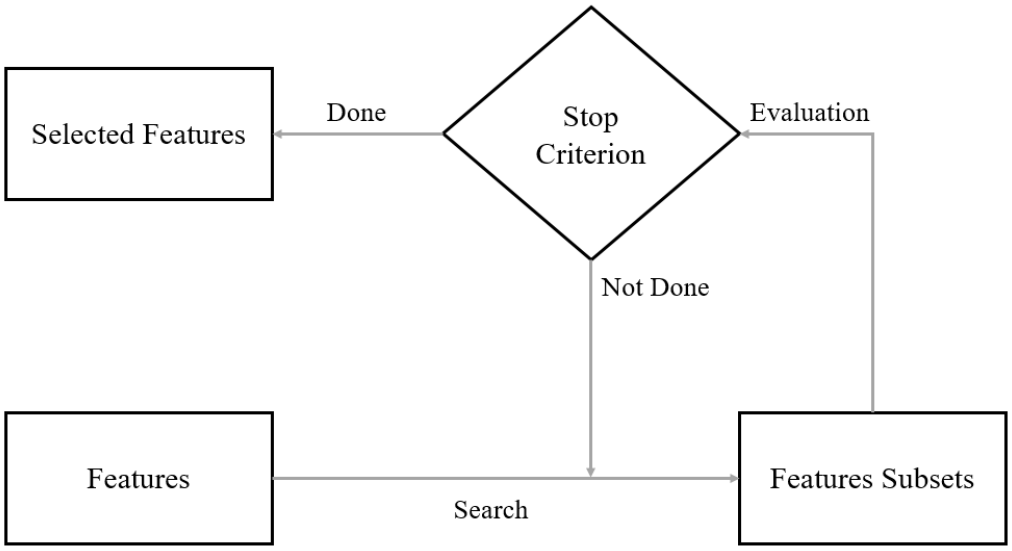


Figure (2.5) Feature Selection Flow in General [41]

Feature selection algorithms can be categorized into three categories such as supervised, unsupervised and semi-supervised. Typically, the categorization is based on the training set whether it is labeled or not. Supervised feature selection approaches can be categorized into filter models, wrapper models and embedded models [43].

2.7.1 Feature Selection for Classification

Most of the classification problems demand supervised learning where class-conditional and underlying class probabilities are unknown, besides, every instance is related to the label of the class [44]. In real world cases, usually a little knowledge about relevant features are known. Accordingly, in order to represent the domain in a better way, numbers of candidate features are brought in coming from the existence of redundant/irrelevant variables to the target concept. A relevant feature is neither redundant nor irrelevant to target concept; an irrelevant feature is not associated with the objective concept directly; however, it influences the learning process. The existence of redundant features does not add or change anything new to the target concept [44]. In some classification scenarios, it is hard to teach good classifiers before eliminating these unwanted or undesirable features due to the extremely large size of the data. Eliminating the number of redundant and irrelevant features can extremely decrease machine learning running time.

In addition, feature selection impacts the training phase of classification. After generating variables, instead of directly processing the data with all variables to the

learning classifier, feature selection for classification starts the selection to choose a subset of variables, and then processes the data with chosen features to the learning classifier. The feature selection stage of the learning model can be independent like filter algorithms, or it can iteratively utilize the accuracy of the learning methods to assess the quality of the selected features like wrapper models. After finalizing the feature selection, a classifier algorithm is prepared for the prediction stage [43].

2.7.2 Main Approaches for Feature Selection

Feature selection can be divided into three main types of methods, where features are considered as independent. The main three categorized algorithms include filter model, wrapper model, and embedded models.

2.7.2.1 Filter approach

Depending on the components of the data, filter methods assess the features without any types of classification algorithms utilization [39]. A regular filter algorithm involves the following two steps. First, it deals with ranking features based on specific criteria. Feature assessment can be either multivariate or univariate. In the univariate system, each variable is ranked separately from the feature space, while the multivariate system assesses the variables in a batch way. Accordingly, the multivariate system is mainly able to have redundant variables. Second, it deals with features with the highest ranks that are chosen to induce classification methods [43].

2.7.2.2 Embedded approach

Embedded approach is related to machine learning algorithm which charges lower computational cost rather than the wrapper approach. Also, this approach captures variable dependencies. It searches locally for variables that lead to better local discrimination along with considering interaction among one input feature and an output one. It implements the independent criteria in order to choose the optimal subsets for known cardinality. After that, the learning method is used in order to pick out the final optimal subset between the optimal subsets amongst various cardinalities [43].

2.7.2.3 Wrapper approach

Filter methods select features without depending on any particular classifier, still the main disadvantage of the filter method is that it completely neglects the effects of the selected feature subset on the induction algorithm’s performance [45][46]. Wrapper

methods are based on an assumption that the optimal feature subset should depend on the particular biases and heuristics of the induction method. Depending on this assumption, wrapper methods make use of specific classifier to evaluate the quality of selected features [45][47]. With a predefined classifier, a wrapper model will perform the following steps:

- Searching the subset of features.
- Using the performance of the classifier in order to evaluate the selected subset.
- Repeating first and second steps until the desired quality is obtained.

The Wrapper Algorithms of feature selection is demonstrated in figure (2.6) [45]. It includes three components: features selection search, feature evaluation and induction algorithm.

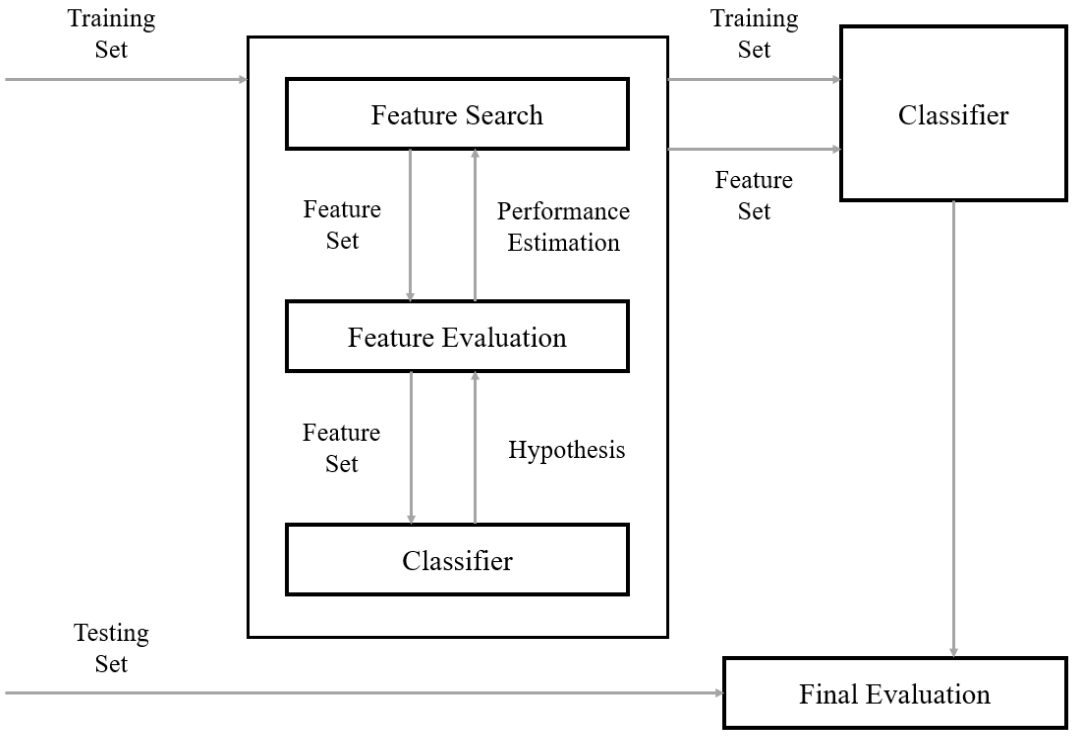


Figure (2.6) Wrapper Algorithms of Feature Selection for Classification General Framework [45]

In wrapper methods, the conducted classifier works as a black box. The part that is responsible for feature search will generate a feature set, and the feature evaluation part will use the classifier to assess the performance, which will go back to the feature search part for later iteration of feature subset selection. The feature set that

obtains the best estimated value will be selected as the concluding set to learn the classifier. After that, the classifier is assessed using an independent testing data that is not used during training phase [45]. The amount of search space for m features is: $(O(2^m))$, hence an intensive search is impossible except that m is small, this problem is defined as NP-hard. Numbers of search approaches can be implemented containing hill-climbing, best-first, branch-and-bound, and genetic algorithms [35]. The Hill-climbing approach increases the current set and steps to the subset with the topmost accuracy, and finishes when there is no subset that leads to improvement to the current set. The best approach is to choose the set that has not previously increased, and it is a more robust approach than hill-climbing [45]. Another approach is Greedy search that seems to be specifically computationally advantageous and robust against overfitting. It has two types which are: forward selection and backward selection or backward elimination. Forward selection deals with a search that initiates at the empty set of features and the features that are increasingly organized into larger and larger subsets. Whereas backward selection starts with a set full of features and progressively removes the least promising features. The search part is willing to observe a subset of features with the highest assessment, using a heuristic method to show the path to it. Whenever the actual accuracy of the classifier is unknown, accuracy estimation will be used as both the valuation method and heuristic method in the feature evaluation step. Usually performance evaluation will be done by using validation set or via cross-validation [43]. Algorithm Figure (2.1) demonstrates the general wrapper method for feature selection [48].

Studies show that wrapper methods attain a better predictive accuracy than filter methods [45][49]. Yet, wrapper methods are computationally more complex compared to the filter methods. It causes a better performance for the predetermined classifier while our focus is to select features that increase the quality. Consequently, the selected feature subset is necessarily biased to the predetermined classifier.

In general, wrapper methods are classified into two types of algorithm, sequential selection algorithm and heuristic search algorithm. Firstly, in order to obtain maximum objective function, the sequential selection approach initiates with an empty set and adds features, or starts with full set and eliminates features. To accelerate the selection, a criteria is selected which will gradually increase the objective function until it reaches its maximum with the lowest number of features.

Various subsets can be created either by searching in a search space or via creating solutions to the optimization problem. This thesis, concentrate on sequential selection algorithms which are used in the proposed model.

Algorithm (2.1) Steps of General Wrapper Method [48]

Input:

$D = \{X, L\}$ // *a training data set with n number of feature where* // $X = \{f_1, f_2, f_3, \dots, f_n\}$ *and L labels*

X' // *predefined initial feature subset* ($X' \subset X or X' = \{\phi\}$)

θ // *a stoppage criterion*

OUTPUT: X'_{opt} // *an optimal subset*

Begin:

Initialize:

$X'_{opt} = X';$

$\varphi_{opt} = E(X', A);$ // *evaluate* X' *by using mining algorithm A*

do begin

$X_g = generate(X);$ // *Subset generation for evaluation*

$\varphi = E(X_g, A);$ // X_g *current subset evaluation by A*

If $(\varphi > \varphi_{opt})$

$\varphi_{opt} = \varphi;$

$X'_{opt} = X_g;$

repeat (*until* θ *is not reached*);

end

return X'_{opt};

end;

2.8 Sequential Selection Algorithms

These methods are named sequential because of their iterative nature. In this study, two types of this sequential wrapper feature selection method will be depended on, sequential forward selection and sequential backward selection. The Sequential Forward Selection (SFS) method begins with a set that is empty and adds one feature to the first iteration which provides the topmost value to the objective function [50][51]. Starting from the second iteration, the remaining features will be added one

by one to the ongoing subsets and the subset will be assessed each at a time. In this type of algorithm when an individual feature gives the maximum classification accuracy, it will be included in the subset permanently. The process is going into a loop until the specified number of features is included. Algorithm (2.2) demonstrates steps of SFS implementation [52].

Algorithm (2.2) Sequential Forward Selection [52]

Require: Set of features $X = \{x_0, x_1, \dots, x_n\}$ size of feature set n, size of target feature subset d

Ensure: Suboptimum feature subset Y_{subopt} of size d

1: $Y_{subopt} \leftarrow \emptyset$

2: **for** $j = 1 \rightarrow d$ **do**

3: $x \leftarrow \max\left(J\left(Y_{subopt} \cup \{x_i\}\right)\right) \mid x_i \in X \text{ and } x_i \notin Y_{subopt}$

4: $Y_{subopt} \leftarrow Y_{subopt} \cup \{x\}$

5: **end for**

A Sequential Backward Selection (SBS) method can be created similar to SFS but the method begins from the entire set of features and eliminates one variable at a time which its removal represents the lowest decrease in performance of the predictor. The steps of implementing SBS are demonstrated in Algorithm (2.3) [52].

Algorithm (2.3) Sequential Backward Selection [52]

Require: Set of features $X = \{x_0, x_1, \dots, x_n\}$ size of feature set n, size of target feature subset d

Ensure: Suboptimum feature subset Y_{subopt} of size d

1: $Y_{subopt} \leftarrow X$

2: **for** $j = 1 \rightarrow n - d$ **do**

3: $x \leftarrow \min\left(J\left(Y_{subopt} - \{x_i\}\right)\right) \mid x_i \in X \text{ and } x_i \in Y_{subopt}$

4: $Y_{subopt} \leftarrow Y_{subopt} - \{x\}$

5: **end for**

In practical point of view, there are some parameters of both SFS and SBS that should be tuned before implementing these methods. For example, specifying

machine learning algorithm because the accuracy obtained during all iteration will be determined by having at least a learning model. Another parameter that should not be neglected is the number of folds of cross-validation because training and testing will be attained through it.

2.9 K-Fold Cross-Validation

K-fold cross-validation is the most common and widely used technique for model selection and model evaluation in machine learning. This term is sometimes identified by researchers and specialists in an inaccurate way, where they denote the technique as a holdout method of training or testing. Thus, considering cross validation by means of train cross over and phases of validation in consecutive sequences makes it more logical. At this point, the core concept of cross-validation regarding specified dataset is that every individual sample from the dataset gets a chance to be tested. Cross-validation has a common case called k-fold cross-validation, it deals with iterating in k times with regard to the specified dataset. In each and every iteration, the dataset will be divided into k pieces where one of them determined for validation while the remaining parts of k – 1 are combined and represent a subset of training in order to evaluate the model. For demonstration, figure (2.7) illustrates 5-fold cross-validation process [53].

Generally, in implementing k-fold cross-validation, a machine learning algorithm will be used besides using fixed settings of hyper parameter in order for the models to be fit regarding the training folds of all iteration individually. For instance, in cross validation of 5-fold, the following practice leads to fitting five various models. The five models will be fit in order to separate partially corresponding training sets, and they will be evaluated according to the validation sets that are entirely distinct. Finally, the performance of cross-validation will be calculated by finding the mean divided by performance of k which evaluates in the validation set.

Usually, in k-fold cross validation, the entire data will be used for training and testing. The concept of using all data from the dataset is for lessening the bias via increasing the training data, opposite to the testing data, which is putting a reasonably great part to the dataset.

The overlapping will not be done by test folds from k-fold cross validation. Moreover, for validation, samples will be used one by one in k-fold cross validation. While using k-fold cross validation is being demonstrated for model evaluation, it usually can be applied for machine learning algorithm selection and model selection which is more common [53].

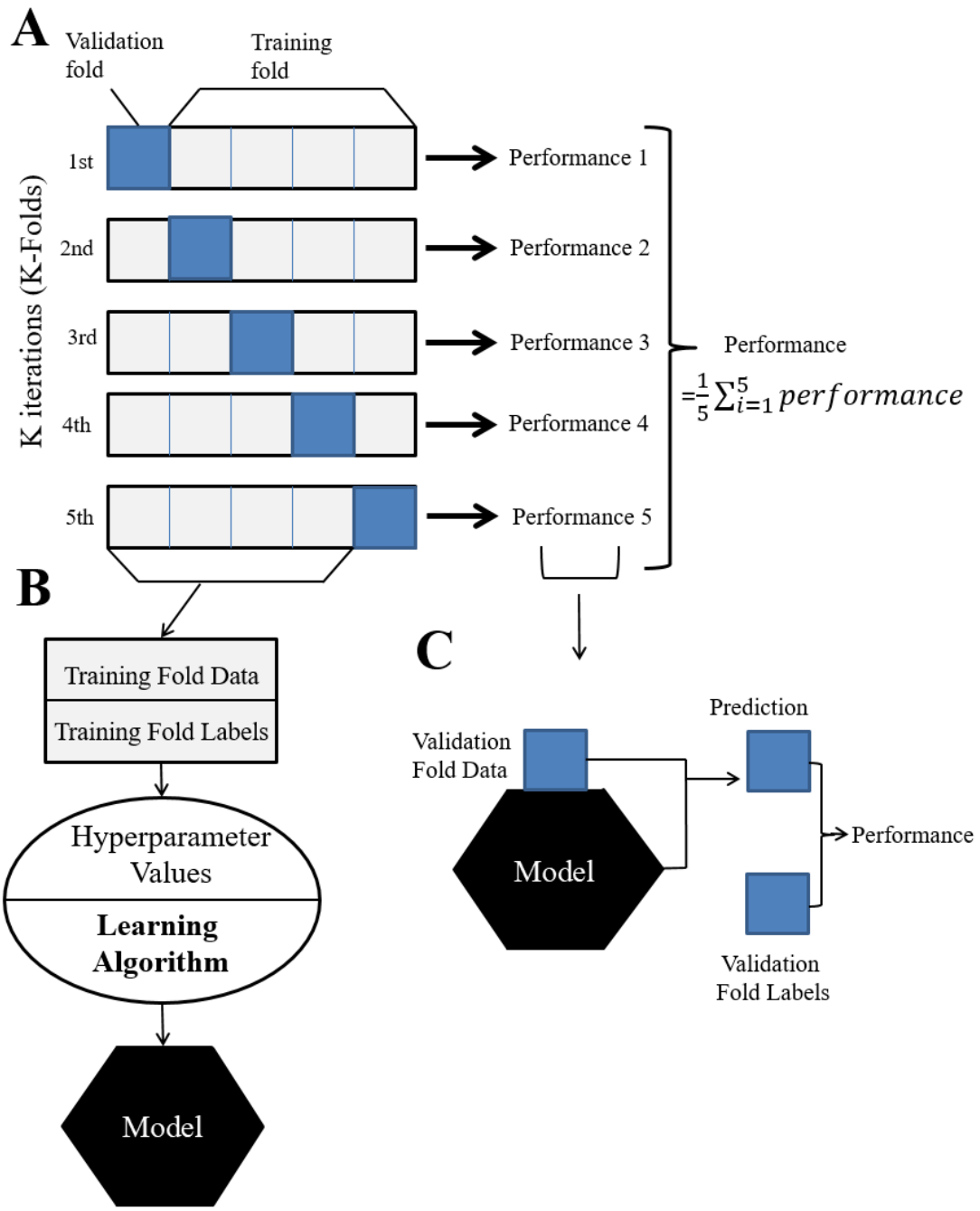


Figure (2.7) 5-Fold Cross-validation Process [53]

2.10 Boosting Algorithms

Supervised learning of boosting can be defined as an ensemble meta-algorithm that decreases both bias and variance [54], and it can be categorized as a learning algorithm which is responsible for turning weak learners to strong learners in order to increase the accuracy of the model [55]. The main aim of the boosting algorithm is to make weak learners' performance improve while considering it like a "black box" and constantly could be called like a subroutine, though its contents are invisible and cannot be modified.

In Boosting or sequential ensemble, throughout training stage the weak learners are presented in sequential form. The improvement in performance could be obtained via assigning greater weight to previous samples that are incorrectly classified. Moreover, the essential concept behind boosting learners is generating numbers of weak learners and combination of their prediction in order to create one strong rule. Base machine learning algorithms can be applied to various distributions of the dataset which results in generating weak rules. The algorithms generate rules in each iteration. Furthermore, the prediction of greater accuracy regarding the outcome can be obtained when the weak learners combined in order to create a strong learner after multiple iterations [56]. The following steps demonstrated in figure (2.8) explain how boosting algorithm works [57]:

- The process starts with reading data by the base learner and assigning identical weight to every observation of the sample.
- The base algorithm's false predictions are determined. After that in the next iteration, the incorrect predictions are assigned with higher weight to the afterward base learner.
- Previous steps must be repeated until it reaches the point where the algorithm is able to make output's classification correctly.

Lastly, Strong learner, who helps in improvement of the models prediction power, will be created from the combination of weak learners. The boosting's main objective is to concentrate more on miss-classified predictions or examples that have higher errors via preceding weak rules [56].

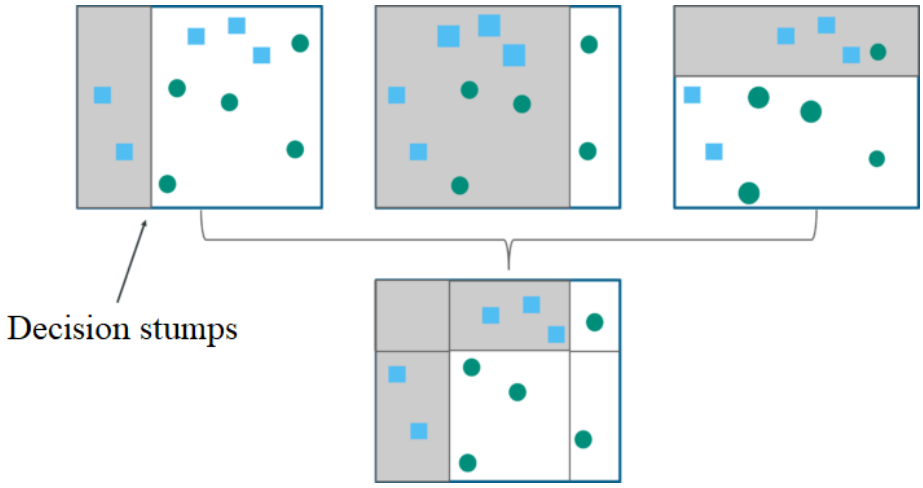


Figure (2.8) General Way of Boosting Algorithm Process [56]

There are different methods of boosting such as Adaptive boost and gradient boosting. In this study, the concentration is on different gradient boosting algorithms that each one of them will be covered in detail:

- Gradient Boosting Machine (GBM)
- Extreme Gradient Boosting (XGBoost)
- Light Gradient Boosting Machine (lightGBM)

2.10.1 Gradient Boosting Machine (GBM)

In a wide range of practical applications, powerful machine learning method such as gradient bosting machine has obtained a significant outcome. It can detect nonlinear function dependencies in an effective way. According to the certain application needs, GBM is favorably customizable, for instance, it could learn according to various loss functions. Mainly in GBMs, the procedure of learning consecutively suits models that are new in order to prepare an estimate of the response variable that is more accurate. The main idea of GBM algorithm is creating brand-new base-learners to be correlated best with the loss function's negative gradient, related with the each and every ensemble. The loss functions that will be practiced are managed to be arbitrary, the procedure of learning could lead to a consecutive error-fitting while the error function is the classic squared-error loss [58].

The flexibility of GBM is high which makes it highly customizable regarding any specific data-driven activity. It proposes much freedom towards the model design, makes decision on most convenient loss function. Generally boosting algorithms can

be implemented easily, which lets individuals practice it with various model designs. Additionally, the GBM learning algorithm has proved a great success in both practical applications and several challenges in data mining and machine learning [59][60].

GBM offers several tuning parameters which is challenging because they need more time in order to discover the optimal combination of parameters. Some common parameters found in many GBM implementations are discussed below:

- Tree Number: deals with fitting the total number of trees. Usually GBM needs many trees; nevertheless, it can over-fit. For that, the main point is to discover the optimal number of trees that leads in minimizing loss function.
- Trees depth: boosted ensemble's complexity can be controlled via the number of d of splits in each tree. Most of the time $d = 1$ gives good result, more generally d is greater than one, but it is not considerable when d greater than ten is.
- Learning Rate: controls the speed of the algorithm to reach down to the gradient decent. Smaller values of learning rate leads to decreasing the chance of overfitting, though, to find the optimal fit, it increases the time.
- Subsampling: controls the usage of portion of the available training observation if it will be used or not.

2.10.2 Extreme Gradient Boosting (XGBoost)

XGBoost is an implementation of gradient boosting framework which is adaptive and expandable, and it is based on decision-tree ensemble learning algorithm. The XGBoost algorithm is available publically in an open source package [61]. It was developed at the University of Washington as a research project. It presented at SIGKDD Conference in 2016 by Tianqi Chen and Carlos Guestrin, and their paper caught fire in the machine learning world [62]. This algorithm has won numbers of Kaggle machine learning competitions after its presentation. It took the challenges of Kaggle competitions, and it is the best website for providing machine learning contests that is used by every winning team.

It has also become the driving force under the hood for some modern applications. This algorithm has specifications which make it special, for instance, it can be

offered for classification, regression and user-defined problems of predictions. XGBoost is proved to give the best of computing power regarding boosting tree learning algorithms because it was created and enhanced for performance of the model and computational acceleration. Particularly, it was developed to take advantage of every bit of hardware resources and memory for the sake of algorithms' of tree boosting [62].

In XGBoost algorithm, creation of decision trees is done in sequential form. Weights are considered to have an important part in XGBoost algorithm. These weights are given to every independent variables, and later fed into the decision tree that predicts outcomes. The Wight of variables will increase when the prediction is wrong via the tree that results in feeding these variables to the second decision tree. Finally, these separate predictors/classifiers make an ensemble in order to offer robust and more accurate model.

The practice of XGBoost presents various advanced attributes to the tuning of the model, algorithm improvement and computing environments. Figure (2.9) demonstrates XGBoost in creation of final classifier [63].

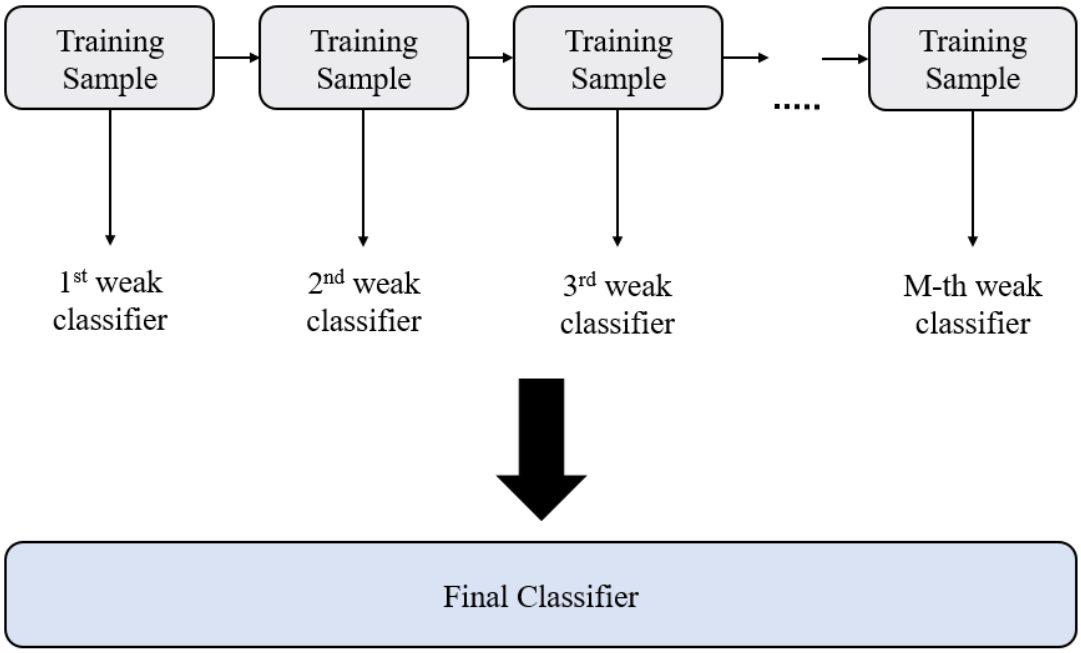


Figure (2.9) Creating Ensemble Learning of XGBoost [63]

Before implementing XGBoost, the following types of parameters should be noted: general parameters, task parameters and booster parameters [64].

- General parameters: determine the booster that will be used to do the boosting (tree and linear are most common).
- Booster parameters: deal with booster that have been picked.
- Parameter of learning task: the decision will be taken on the learning scenario. For instance, ranking tasks may use various parameters with regression tasks.
- Parameters of Command line: deal with behavior of CLI version of the algorithm.

2.10.3 Light Gradient Boosting Machine (LightGBM)

It is a relatively new algorithm, except its documentation of implementation which does not have many reading resources. LightGBM is execution of gradient boosting framework which uses learning algorithm based on trees. It grows trees vertically while other tree based algorithms grow trees horizontally. Thus, it means that this algorithm grows its trees leaf-wise, while other boosting algorithms grow their trees in level-wise form. For growing the leaf, it will select the leaf with max delta. Considering growing of the same leaf, leaf wise algorithm leads to reducing more loss compared to the level-wise algorithms [65]. Figure (2.10) demonstrates how leaf wise algorithm works [66].

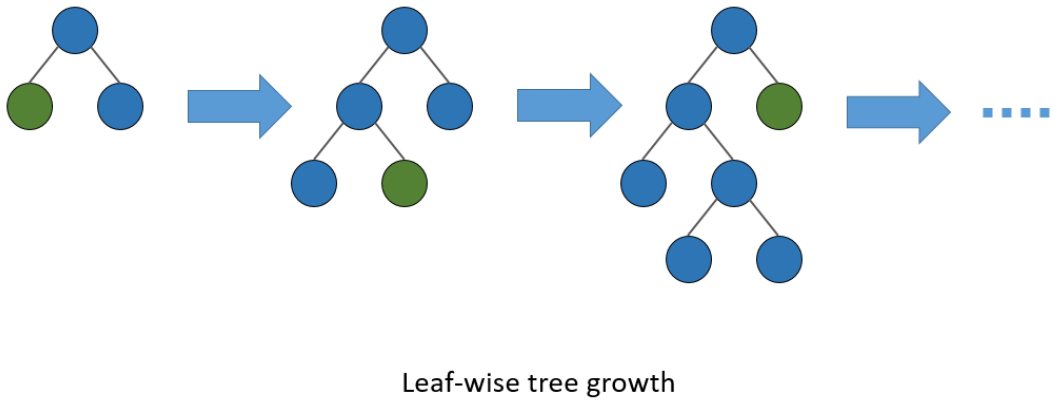


Figure (2.10) Working of LighGBM Algorithm in General [66]

While the size of the data increases on a daily base, the traditional data science algorithms find it harder to give their results in a short time. The name, LightGBM, came from prefixed 'Light' because of speed of light and GBM which is mentioned before. Large size of data can be handled by LightGBM and lower memory in order to run. Besides, this algorithm is popular because it concentrates on outcomes accuracy.

For implementing LightGBM algorithm in any problem of machine learning, there are some sorts of parameters that should be dealt with. For instance, core parameters include [67]:

- Boosting: deals with the type of boosting algorithm that needs to be run.
    - gdbt: traditional gradient boosting decision tree.
    - Dart: dropouts meets multiple adaptive regression trees.
    - Goss: gradient-based one-side sampling.
    - rf: random forest.
- num_boost_round: Deals with boosting iteration number.
- learning_rate: This detects the effect of each tree on the last result.
- num_leaves: This deals with full trees number of leaves.
- Device: It can pass GPU and CPU.

Moreover, the control parameters of LGBM contain:

- max_depth: This parameter is important because it is used to handle overfitting, and it determines the maximum depth of the tree.
- min_data_in_leaf: It deals with the minimum number of the records a leaf may have.
- feature_fraction: This means when the boosting is random forest, the algorithm for building trees will choose 80% of hyper-parameters randomly in each tree.
- early_stoping_round: It helps in reducing excessive iteration.
- Lambda: Determines regularization.
- Min_gain_to_split: Deals with defining the minimum gain to make a split.
- max_cat_group: When the number of category is large, finding the split point on it is easily over-fitting. Hence, the LightGBM merges them into ‘max_cat_group’ groups, and it finds the split points on the group boundaries.

2.11 Model Evaluation

There are different sorts of classification models in machine learning, which should be evaluated to discover the optimal solution among them. Depending on the observing method, there are various measures for assessing the performance of the

model. The most direct criterion that can be measured quantitatively in the evaluation of classification models is classification accuracy [68].

The task of assessing classifiers is to estimate the degree to which the recommended model resembles the actual classification of the problem. Depending on the observing approach, there are several measures for the model's performance evaluation. The most appropriate measures shall be made depending on the problems components and their implementation ways [68].

In this study, because it is supervised learning, three classification models such as GBM, XGBoost and LightGBM will be evaluated using different sorts of metrics and all of them based on the confusion matrix.

2.11.1 Confusion Matrix

The confusion matrix is responsible for providing insights into predictions. It indicates correct and incorrect predictions for each class. In this case of binary classification, it covers a 2x2 matrix. The two classes of this classification problem are demonstrated in the table (2.1), and it shows four different outcome predictions. Correct positive and correct negative outcomes are correctly classified, while incorrect positive and incorrect negative results are measured as two possible sorts of errors.

Incorrect positive is an instance of the negative class that mistakenly classified as positive and wrong negative is an instance of the positive class that mistakenly classifier as negative. According to [69] confusion matrix has the following meanings:

- *a* is the number of true predictions that examples are negative,
- *b* is the number of false predictions that examples are positive,
- *c* is the number of false predictions that examples are negative,
- *d* is the number of true predictions that examples are positive.

Table (2.1) Confusion matrix in Classification Problem with Two Classes [69]

Actual \ Predicted	**Negative**	**Positive**
Negative	a	b
Positive	c	d

2.11.2 Classification Metrics

From the confusion matrix with the two classes, a few standard terms can be defined: accuracy, precision, sensitivity, specificity and f1-score. Each of these metrics can be taken into consideration according to a specific problem [69].

- Accuracy: Deals with the number of correct predictions in the classification problems.

$$Accuracy = \frac{a+b}{a+b+c+d} \quad (2.1)$$

- Precision (positive predictive value): Determines how effective the model is when the prediction is positive.

$$Precision = \frac{d}{b+d} \quad (2.2)$$

- Sensitivity or Recall: Focuses on the model's ability to correctly predicting positive classes.

$$Sensitivity = \frac{d}{c+d} \quad (2.3)$$

- Specificity: It deals with the proportion of negative class that is correctly predicted as negative.

$$Specificity = \frac{a}{a+b} \quad (2.4)$$

- F1 Score: It can be identified as a mean among recall and precision. F1 score is necessary for observing the true balance among recall and precision.

$$F1\ Score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (2.5)$$

Throughout this study, these evaluation metrics will be considered in order to evaluate the model's performance. In some scenarios, recall and f1-score may take the lead for deciding between the classifiers that archive similar or almost the same score of accuracy.

Chapter Three
The Proposed System

3.1 Introduction

In the age of technology, using smart devices has become a necessity. Thus, the usage of smart devices created a problem for its users. Spamming, a method to reach out to as many smart device users as possible, with relatively low cost. Beside the good use of this service, spammers exploit it for many different malicious purpose which results in harming the mobile phone users. Spam messages are usually deceiving and contain URL that may lead to install malware to end user devices or steal identity, bank account and etc.

Moreover, most of the people who are normal users of internet and mobile phones are not aware of these down sides of technology and can be victimized easily by clicking on URL provided in spam texts. Spams are constructed in a way to either give subliminal messages to the recipients, or to rip them off their personal and sensitive information by tricking them to fall into a trap. The goal of the thesis is to detect the spam messages as accurate as possible, by proposing a system which is fast and leads to accurate outcome that can be used for mobile applications.

In this chapter, general approaches of the proposed system are shown. Dataset containing spam and legitimate messages has been prepared; the other parts regarding features extraction, selection and machine learning have been offered and presented in detail. Figure (3.1) illustrates the proposed systems for spam detection block diagram. Various sorts of features are collected from other relevant studies besides suggesting new features, which can be important in making distinction between spam and normal messages. Subset of relevant features will be selected by using sequential wrapper feature selector methods with various sorts of gradient boosting methods as classifiers.

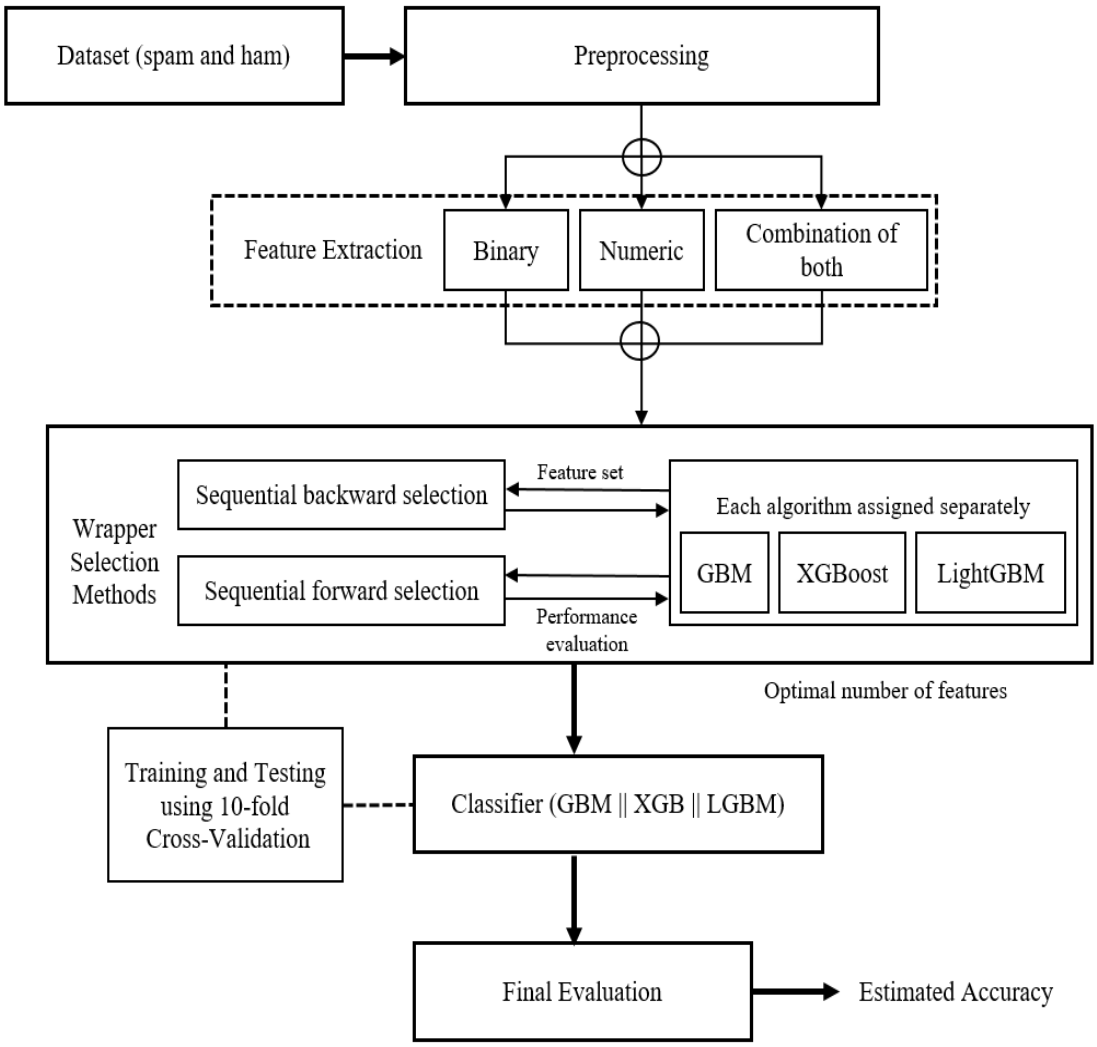


Figure (3.1) Block Diagram of the Proposed System

3.2 Design and Implementation

The proposed system software consists of two main parts, Graphical User Interface (GUI) and the backend. The GUI of the system is created with C# and formed with Windows Forms Application. The Integrated Development Environment (IDE) used for C# programing is visual studio 2017. The application lets the user to input and output data, regarding the purpose of the thesis, for instance, browsing and opening the specified spam dataset to the software. The backend does all of the programing part of machine learning. For instance, reading messages from the dataset and creating vector of feature after extraction and other machine learning phases are all implemented through backend part which is written in python programing.

3.3 Preprocessing

The following steps are essential for the data before feeding it to machine learning models:

- Importing libraries.
- Preparation of dataset.
- Handling missing data in the dataset.
- Categorical data encoding.
- Feature extraction.

3.3.1 Importing Libraries

Usually, importing the libraries that are required during the execution of the program is considered as the first step of preprocessing. A library is primarily a set of modules that could be called upon request because there are tasks in the programming area that do not require to be written separately when they are needed. For that there are different sorts of methods which can be invoked for each explicit task. While the proposed system is mostly written in python programing, therefore libraries are needed to be used. For instance, Pandas library is imported in and assigned a shortcut as "*pd*". Table (3.1) demonstrates important used libraries from preprocessing to feature selection methods and provide short description of each library and sub library.

Table (3.1) Libraries used in The Proposed System

Library	Short Description
NumPy	It is responsible for adding support for metrics and multidimensional arrays
Sklearn	It is machine learning library which is free. It contains various algorithms for classification, regression and clustering including SVM, random forest and gradient boosting.
LabelEncoder	It is a part of sklearn.preprocessing, responsible for encoding labels to 0 and 1.
GradientBoosting Classifier	Improted from sklearn.ensemble packages, it represents gradient boosting machine algorithm for classification
XGBClassifier	It is imported from xgboost libraries which is designed to be highly flexible, portable and efficient to use.
LGBMClassifier	It is imported from lightgbm library which is a gradient framework adapts tree based learning methods.
cross_val_score	Imported from sklearn.model_selection libraries and it is responsible for evaluation of scores via cross-validation.
cross_val_predict	Imported from sklearn.model_selection and provoke cross-validated estimates for each input data point.
confusion_matrix	Imported from sklearn.metrics which is used to evaluate the accuracy regarding classification
SequentialFeature Selector	Imported form mlxtend.feature_selection library which provides useful tools for specific tasks.

3.3.2 Preparation of the Dataset

The lack of representative data is a serious reason to affect the evaluation process and comparison of approaches. Unfortunately, more recent study areas are largely affected by the absence of public data. Therefore, by the assistance of cell phone users or certain websites, SMS dataset could be obtained. In this study, the dataset that is used is public, non-encoded and real SMS spam corpus made available by Tiago et al. [7] and it is the largest dataset which is publically available as of today. The dataset is a collection of spam messages which is counted to be 425 messages that were manually extracted in the website called Grumble text. Additional 450 normal messages were added by Tiago et al. [7] to the dataset which is collected on Caroline Tag's PhD thesis. This dataset is available under the name (SMS Spam Collection v.1) in UCI machine learning repository and known as Tiago's dataset in some studies. This collection includes 5574 messages in total which consist of 4,827 normal SMS messages and 747 cell phone SMS spam short messages. In figure (3.2) the statistics regarding SMS spam corpus v.1 is demonstrated.

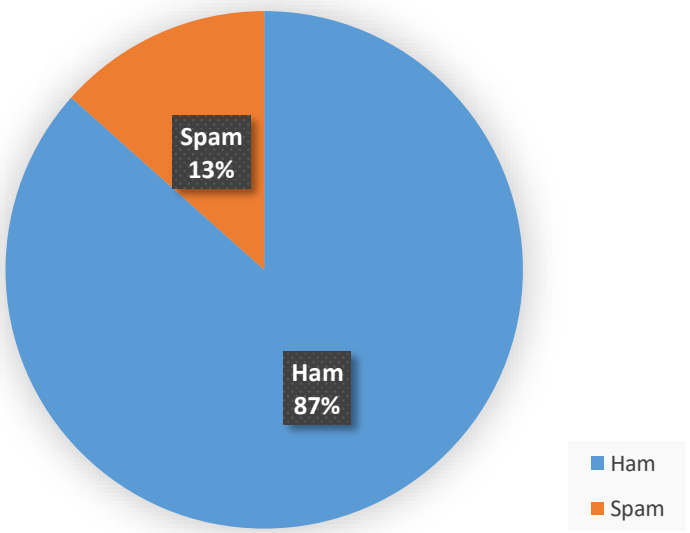


Figure (3.2) SMS Spam Corpus Statistics

Moreover, this SMS spam corpus is not the only dataset in the area, but it can be referred to as the most widely used by other researchers and in this study it will be used throughout the experiments. Table (3.2), shows the first 10 messages that are in the dataset in this study. In addition, for making the spam corpus ready to be used by the proposed system, the following steps implemented that are illustrated in figure (3.3).

Table (3.2) Instance of Messages and Categories in The Dataset used in This Study

Category	Message
ham	Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
ham	Ok lar... Joking wif u oni...
spam	Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's
ham	U dun say so early hor... U c already then say...
ham	Nah I don't think he goes to usf, he lives around here though
spam	FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some fun you up for it still? Tb ok! XxX std chgs to send, £1.50 to rcv
ham	Even my brother is not like to speak with me. They treat me like aids patent.
ham	As per your request 'Melle Melle (Oru Minnaminunginte Nurungu Vettam)' has been set as your callertune for all Callers. Press *9 to copy your friends Callertune
spam	WINNER!! As a valued network customer you have been selected to receivea £900 prize reward! To claim call 09061701461. Claim code KL341. Valid 12 hours only.
spam	Had your mobile 11 months or more? U R entitled to Update to the latest colour mobiles with camera for Free! Call The Mobile Update Co FREE on 08002986030

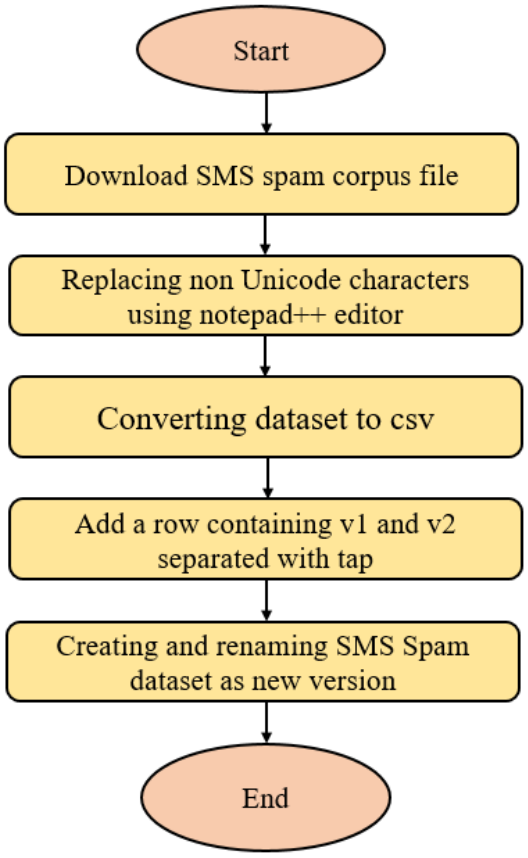


Figure (3.3) The Flow Chart of SMS Spam Corpus Preparation

3.3.3 Handling Missing Data in the Dataset

Missing value is a case when no data value is stored for the variable in an observation. Researchers identify the problem of missing data as common in most of the studies because it has significant effect on the decisions that can be drawn

from the data. It may propose different types of problems. First, the lost data leads to statistical power reduction. Second, the absence of data can cause bias in the parameter estimation. Third, having null values may reduce representatives of samples. In this study, the missing data will be checked right after loading the dataset in C# program.

3.3.4 Categorical Data Encoding

In some cases, the dataset that will be used is in qualitative shape, meaning it has texts as data. Since most of the machine learning models deal with mathematical equation and calculation, it is not easy for those models to interpret and process texts comparing to numbers. For that, encoding from categorical data is required. In this study, SMS spam corpus categories could be found in the class column namely "Spam" and "ham". In order to encode them, Scikit library was imported and the method fit_transform in LabelEncoder class was called for doing the task. After that, the texts in label column converted into numbers as expected, spam became 1 and ham or legitimate encoded to 0. Moreover, all categories were encoded to 0s and 1s after encoding. Figure (3.4) demonstrates preprocessing phases regarding proposed software.

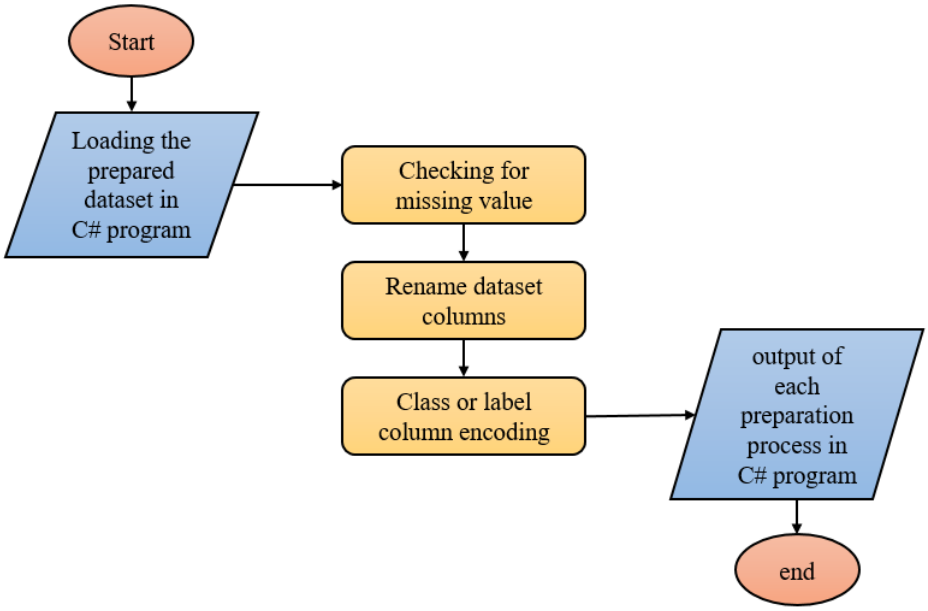


Figure (3.4) Flowchart of Essential Preprocessing Phases

In addition, figure (3.5) shows the preprocessing phase according to software system proposed in this study. It contains checking missing value, encoding spam and ham messages and other useful information regarding the dataset statistics.

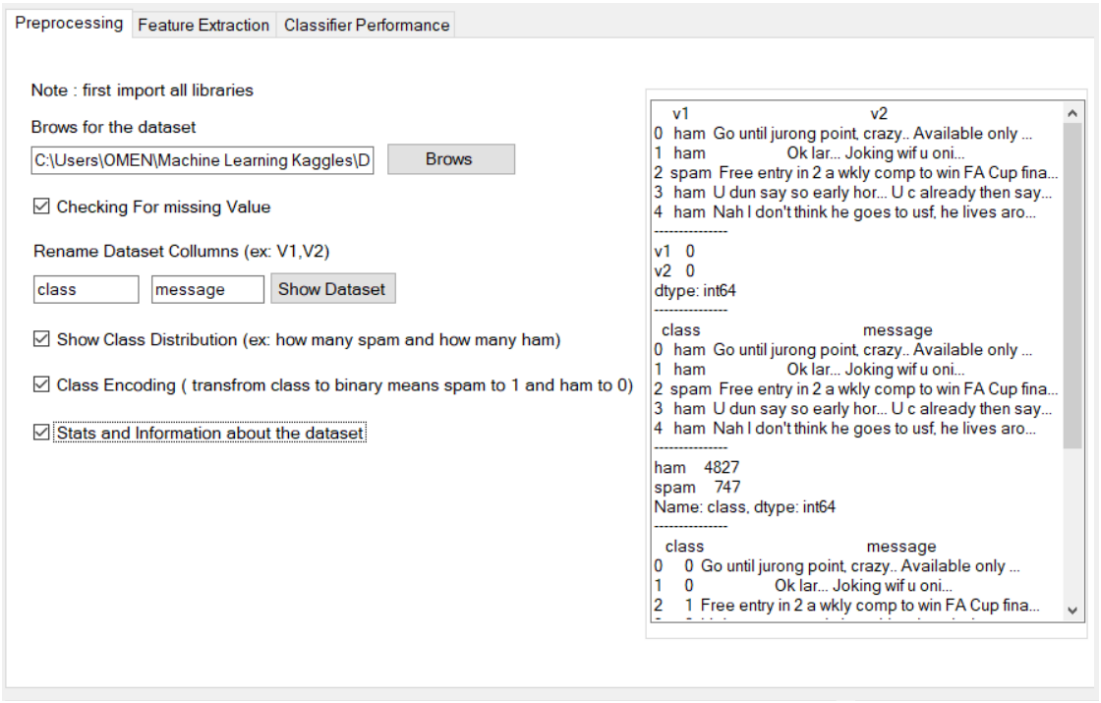


Figure (3.5) Essential Preprocessing Phases

3.3.5 Feature Extraction

In this phase, binary and numeric features were extracted. Most of the features were collected from different relevant studies besides three new features were proposed in this study. Moreover, the aim of using these types of feature is to give a comparison among them, and in terms of the accuracy they lead to separate results, in order to choose the best among them.

3.3.5.1 Binary features extraction

The idea of using binary feature came from [14] which studied spam messages attributes in depth with extraction of 10 structural features. In binary features, each will attain value of 0 or 1 with regard to their existence in the text message. Primarily 18 binary attributes were extracted, 9 of them were collected from [14]. Another 6 features from [15] were collected, in which they used replacement rules in their study in order to optimize the quality of tokens in the dataset. In this study, those replacement rules reformed to features without knowing how well they will affect the proposed model. Finally, the last 3 features, represented in this study, were created by us with the aim of finding out how relevant these three features are. In addition, if the structure of the spam messages is compared to normal messages, there will be some differences that are worth mentioning as features. For instance, the following attributes represent the features of spam and ham separately:

- Math symbols existence – Usually math symbols such as (+, - , <, >, ^ and /) are desired by spammers in order to make the structure of their messages attractive to the recipient. Also, each math symbol may have a purpose, for instance, using plus sign may represent free service in the spam messages like "We tried to call you! reply to our sms for a nokia mobile + free camcorder. Please call now 08000930705". As a result, if one of the math symbols exists in the message, it will be one otherwise it will be zero.

$$feature1 = \begin{cases} 1 & math\ symbol \\ 0 & no\ math\ symbol \end{cases} \quad (3.1)$$

- Existence of Uniform Resource Locator (URL) – The presence of URL in the message can be suspicious because most of the rip offs and stealing occur through URLs. The messages are fashioned in a way that promotes clicking on the link in the body, so it will fish the recipients. For instance, "You've been selected to win a FREE giftcard! Click here for more details http://gx.com/sesjfh". Once, the URL has been clicked on, the recipient may get victimized and harmed in any way including harvesting credential, downloading harmful app that leads to device remote controlling, etc. It is also worth mentioning that hazardousness of URL mostly affects smart phone users. If URL exists in the message, the feature value will be 1, if not it will be 0.

$$feature2 = \begin{cases} 1 & URL \\ 0 & no\ URL \end{cases} \quad (3.2)$$

- Existence of dots – Unlike the presence of URL and math symbol attributes, dots can be legitimate messages attribute. Usually, in normal messages, dots can present formality to the text structure. Dots used to represent the end of a sentence, this includes in chatting and messaging, like "Bro, I will keep you updated. See ya.". Consequently, feature three value will be 1 if dots present in the message, and zero in the absent of dots.

$$feature3 = \begin{cases} 1 & dots \\ 0 & no\ dots \end{cases} \quad (3.3)$$

- Existence of special symbols – Various symbols are inserted into spam texts to make them more attractive to the recipients. For instance, they may use $

sign and £ to represent a prize winning informant message whereas in reality it is nothing but a scam. Likewise, exclamation marks "!" are used for drawing attention of the user, such as "CONGRATS!, YOU WON!". The special symbols that are used for feature four include the following (£, $, !, *, &, #, ~). So feature value will be one if one of the special symbols are presented in the message otherwise it will be 0.

$$feature4 = \begin{cases} 1 & special\ symbol \\ 0 & no\ special\ symbol \end{cases} \quad (3.4)$$

- Existence of emotions – Usually in legitimate massages, people use some sort of symbols to express their emotion while chatting. For instance, happiness is expressed as ":)" whereas for sad face this expression ":(" is used. Additionally, if the person send this ":'(" means crying face or this ":O" means astonished. Those examples of emotions usually are used among chatters like "k :) k :) good :) study well.", so they are good indicator of normal messages. Furthermore, different emotions from various symbols created according to [14], like, (:) , :(, -_- , :p , :v , :* , :o , B-) , :'( ). While other sorts of emotions proposed in this study in order to recognize them more during feature extraction. For instance, (:3 , ;) , ;-) , :-O , :D , xD , <3 , etc.). Thus, feature five value will be 1 if one of the emotions exist in message and 0 otherwise.

$$feature5 = \begin{cases} 1 & emotions \\ 0 & no\ emotions \end{cases} \quad (3.5)$$

- Lowercased words existence – This feature primarily corresponds to the full existence of lower case words in the message. Regular SMS contains more lowercase words than Spam messages; for instance, "it's ok to be normal for once." Repeatedly, feature six value is one if lowercase words exist, and if it does not exist, then it will be 0.

$$feature6 = \begin{cases} 1 & lowercase\ words \\ 0 & non\ lowercase\ words \end{cases} \quad (3.6)$$

- Uppercased words existence – Drawing the attention of recipients via uppercases words is one of the common methods spammers usually do. For instance, they include words like: "CONGRADULATION, YOU ARE THE

WINNER, CLAIM YOUR PRIZE". Moreover, the value of this feature will be 1 if one or more than one uppercase word exists in the message.

$$feature7 = \begin{cases} 1 & uppercase\ words \\ 0 & non\ uppercase\ words \end{cases} \quad (3.7)$$

- Phone number existence – The presence of mobile number is a good indicator of spam messages because spammers use them for stealing personal information. Primarily, spammers persuade the recipients to call the given number in the message. The recipient's confidentiality may go under risk when they dial the provided mobile number in the spam. For instance, the spammer may send a message like "you are the WINNER! For claiming your prize dial 0445506632". So if the mobile number is present in the message, the value of feature eight will be 1, if not then it will be 0.

$$feature8 = \begin{cases} 1 & phone\ number \\ 0 & no\ phone\ number \end{cases} \quad (3.8)$$

- Presence of currency + number – Similar to the existence of special symbols, this feature has currency symbol. However, in this attribute currency must occur with number, the symbol alone will not be counted. Usually, the currency symbol will directly be put to the left of the digits. For instance, if the message includes for instance, "claim $100" the value of this feature will be 1, otherwise it will be 0.

$$feature9 = \begin{cases} 1 & currency + number \\ 0 & no\ currency + number \end{cases} \quad (3.9)$$

- Presence of number with percent – Spammers usually use these phrases in their texts, for instance, "100% Free" or "claim your gift now! Just 10 percent left". This feature can recognize patterns of number and percent symbol or percent word behind it and vice versa. Hence, if this pattern exists in the message, the value of this feature will be 1 otherwise 0.

$$feature10 = \begin{cases} 1 & number\ with\ percent \\ 0 & no\ number\ with\ percent \end{cases} \quad (3.10)$$

- Presence of ring, tel, cal + number – Terms such as ring, tel and call are commonly used by spammer when creating their text messages. Each

specified word can be a good spam indicator when it comes with number. The three terms are chat text language shortened and each gives almost the same meaning. For instance, spammer may include “cal 00954332354 for claiming your PRIZE”. If this pattern occurs in the message, then the value of this feature is equal to 1 and if not then it will be 0.

$$feature11 = \begin{cases} 1 & ring, tel.cal + number \\ 0 & no\ ring, tel.cal + number \end{cases} \quad (3.11)$$

- Presence of date, time – Spammers use date and time in order to pressure on the recipients, to make them subscribe to their fake services. For instance, “the contest will finish in 12 July! Apply now to win the award”. Consequently, the value of this feature will be one if any of date or time occurs in the message, and zero if they are not.

$$feature12 = \begin{cases} 1 & date\ and\ time \\ 0 & no\ data\ and\ time \end{cases} \quad (3.12)$$

- Presence of long string number – Spammers refer to these sorts of number when they try to scam the recipient to send money. These number can be considered as account number were spammer provide in order to receive money like “Send 100 GBP to 932343435453 TO WIN the monthly AWARD from MONZO BANK”. This feature value will be 1 if the account number or long string number is present in the message, and zero if not.

$$feature13 = \begin{cases} 1 & long\ string\ number \\ 0 & no\ long\ string\ number \end{cases} \quad (3.13)$$

- Presence of email – Existence of email could also be a good spam indicator because, in some cases, the spammer may want to trick the recipient into revealing confidential information like name and personal email address. For instance, “You have won 950,000 GBP in Toyota 2015 Mobile Promo, Send name and addresses to toyota2015@gmx.de”. The value of this feature will be one when the email exist in the message; otherwise, it will be 0.

$$feature14 = \begin{cases} 1 & email \\ 0 & no\ email \end{cases} \quad (3.14)$$

- Presence of currency symbol – Two features related to existence of currency are already identified, thus currency as an independent feature which includes

the following symbols (£, $, €, ¥, ₣, ₹, ₩, ₦, and etc.) is proposed in this study. The proposed feature has differences with special symbols feature which contains pound and dollar sign and it is also different with currency + number feature in terms of the fact that this feature is only identifying symbols. Spammer may use currency symbol for drawing attention of recipient. For instance, "win thousands of $$$$$ by just sending ... ". The value of this feature equals to 1 if one of the money symbols exists in the message, unlike special symbol feature apart from having $ and £, if one of these is present in the message (!, *, &, #, ~) its value will be 1.

$$feature15 = \begin{cases} 1 & currency\ symbol \\ 0 & no\ currency\ symbol \end{cases} \quad (3.15)$$

- Presence of service numbers – The presence of short string number or service number is one of the important features proposed in this study. It primarily deals with 4 to 6 long digit numbers which represent service to recipient. To elaborate on the matter, a local example shall be discussed about Asia Cell that has a vast subscriber base in Iraq. Asia Cell service, for instance, is "A special gift from Asia cell to you! Subscribe to Moody service for 3 days FREE of charge by sending 1 to 2188. The service will be renewed automatically with IQD 300 weekly". Here, in the current example, the service number that provided is 2188, which is the four digit number. Spammers also replicate the same idea in order to deceive the recipient by including service numbers in the content of the message, such as, "msg ansr to 8227 and WIN!!!", so primarily the value of this feature can be identified as 1 if service numbers existed in the content of the message otherwise it will be zero.

$$feature16 = \begin{cases} 1 & service\ number \\ 0 & no\ service\ number \end{cases} \quad (3.16)$$

- Presence of currency in word – Another feature that has been proposed in this study is currency written in word which means describing currency in letter to form its word. This feature is also the same as the currency + number feature that described before but a little bit different in format. Here, instead of writing $ symbol in the message, spammer express it in dollar word or shortened word such as USD. As for pound, instead of using £ symbol

spammer also uses pound words such as pound, pence, penny or even p. For instance, spammers form the spam message like “SPECIAL GIFT of 500 pounds of genuine HMV vouchers to be won.” Currency words usually come after the number, so when the currency word comes after the number, the value of this feature will be one if not it will be 0.

$$feature17 = \begin{cases} 1 & currency\ word \\ 0 & no\ currency\ word \end{cases} \quad (3.17)$$

- Presence of spam keywords – In order for spammers to attract the recipient to their spam messages they use special keywords such as free, ringtone, claim, txt, reply, win, video, mobile, award, dating, won, lottery, entitles, and etc. These spam keywords are usually trendy words and make the message more attractive to the user when constructed by the spammer. The value of the feature is 1 when one or more spam keywords exist in the message and it will be 0 when there is no spam keyword.

$$feature18 = \begin{cases} 1 & spam\ keywords \\ 0 & no\ spam\ keywords \end{cases} \quad (3.18)$$

The extraction of structured binary features will be done through using different sorts of Regular Expression (regex). Each regex determines a value of the feature from the identified SMS spam dataset. For instance, if the regex of URL exists in the message column of the dataset, the value of URL feature will be one, otherwise it will be zero.

Furthermore, the extraction of 17 binary features does not require doing any modification to the original text message. They can be extracted directly from the original text message from the dataset. On the other hand, feature 18 which represents spam keywords requires to be converted before its extraction. Since the specified regex of those keywords should be the same as the spam word discovered from the text message. Additionally, all the text messages will be turned into lowercase letters before the extraction of feature 18, because suggested spam keywords are all constructed with lowercase letters. This leads to finding all spam keywords from the text messages whether they are originally uppercase or not.

Primarily, the feature extraction process executes via the backend, which is written in python programming. It starts from importing related libraries, reading the spam dataset afterward, and finally extracting each feature and assigning them as data

frame to create a vector of features. The pseudocode (3.1) demonstrates how each binary feature will be extracted and then assigned to the data frame that holds all the features.

Pseudocode (3.1) Binary Feature Extraction

```
Required:
Get import numpy library and identify as np
Get import pandas library and identify as pd

Initialize
        dataset_path,        // path of the dataset
        df,                  // an empty data frame
        spam_keywords,       // spam terms that are frequently used by
        spammers
        X,y,                 // X: features data frame, y: Labels data frame
        column_message,      // messages column
        column_class,        // labels column (containing spam and ham)
Begin:
        dataset_path READ spam dataset file path
        df ← pd.read_csv(dataset_path) // pandas data frame of dataset csv file
        column_message ← message column of dataset csv file
        column_class ← class column of dataset csv file
        spam_keywords ← array of spam terms for example: free, ringtone …etc.

        // starting extraction of 18 binary features
        IF df[column_message].contains(regex:'Math_Symbols' THEN
            df['feature1'] = 1
        ELSE
            df['feature1'] = 0
        ENDIF
        IF df[column_message].contains(regex: 'URL' THEN
            df['feature2'] = 1
        ELSE
            df['feature2'] = 0
        ENDIF
        IF df[column_message].contains(regex: 'Dots' THEN
            df['feature3'] = 1
        ELSE
            df['feature3'] = 0
        ENDIF
        IF df[column_message].contains(regex: 'Special_Symbols' THEN
            df['feature4'] = 1
        ELSE
            df['feature4'] = 0
        ENDIF

Continue …
```

Pseudocode Continues:

```
// extraction of binary features continues
IF df[column_message].contains(regex: ’Emotion’ THEN
    df['feature5'] = 1
ELSE
    df['feature5'] = 0
ENDIF
IF df[column_message].contains(regex: ’Lowercase_Words’ THEN
    df['feature6'] = 1
ELSE
    df['feature6'] = 0
ENDIF
IF df[column_message].contains(regex:’Uppercase_Words’ THEN
    df['feature7'] = 1
ELSE
    df['feature7'] = 0
ENDIF
IF df[column_message].contains(regex: ’Phone_Number’ THEN
    df['feature8'] = 1
ELSE
    df['feature8'] = 0
ENDIF
IF df[column_message].contains(regex: ’CurrencySymbol+Number’ THEN
    df['feature9'] = 1
ELSE
    df['feature9'] = 0
ENDIF
IF df[column_message].contains(regex: ’Number+Percent’ THEN
    df['feature10'] = 1
ELSE
    df['feature10'] = 0
ENDIF
IF df[column_message].contains(regex: ’RingTelCall+Number’ THEN
    df['feature11'] = 1
ELSE
    df['feature11'] = 0
ENDIF
IF df[column_message].contains(regex: ’DateTime’ THEN
    df['feature12'] = 1
ELSE
    df['feature12'] = 0
ENDIF
IF df[column_message].contains(regex: ’LongStringNumber’ THEN
    df['feature13'] = 1
ELSE
    df['feature13'] = 0
ENDIF
```

Continue …

Pseudocode Continues:

```
// extraction of binary features continues
IF df[column_message].contains(regex: 'Email' THEN
    df['feature14'] = 1
ELSE
    df['feature14'] = 0
ENDIF
IF df[column_message].contains(regex: 'Currency_Symbols' THEN
    df['feature15'] = 1
ELSE
    df['feature15'] = 0
ENDIF
IF df[column_message].contains(regex:'ServiceNumber' THEN
    df['feature16'] = 1
ELSE
    df['feature16'] = 0
ENDIF
IF df[column_message].contains(regex: 'CurrencyAsWord' THEN
    df['feature17'] = 1
ELSE
    df['feature17'] = 0
ENDIF

df[column_message].lower()   // before extracting feature 18, messages will
                                be lowercase letters
IF df[column_message].contains(regex:' spam_keywords' THEN
    df['feature18'] = 1
ELSE
    df['feature18'] = 0
ENDIF

X ← df[['feature1','feature2' ... 'feature18']]        //data frame of features

y ← df[column_class]                                   //data frame of labels (spam and
ham)
```

End;

In addition, the user selects the group of binary features from the GUI of the program, later the backend side will process the extraction of features at once. Through python programing the extraction of features will be implemented. Figure (3.6) demonstrates how the GUI of the program will interact when the user attempts to extract features and shows its output.

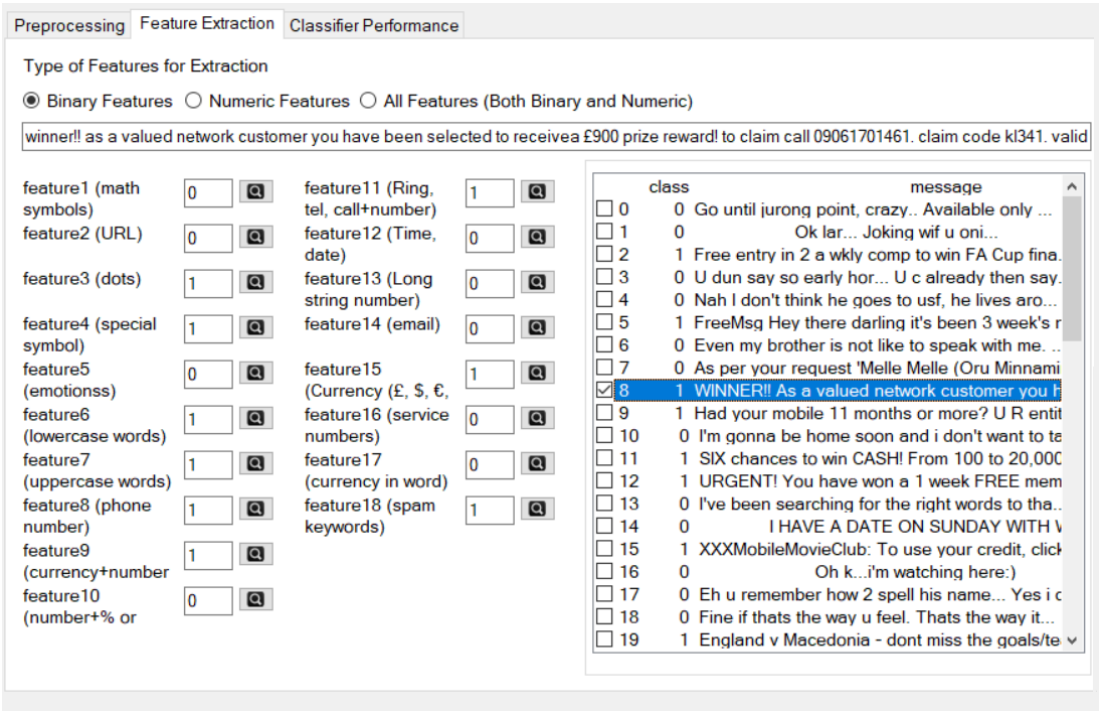


Figure (3.6) Extraction of Binary Features using Proposed System Software

3.3.5.2 Numeric features extraction

The second category of extracted features are numeric features, which represent a numeric data of the attributes in the feature vector. Numeric data will be attained by counting the frequently used attribute in each message. In this study, 8 numeric features were extracted and all of them are collected from relevant studies. Unlike binary features where the value of feature is either 1 or 0, numeric features can be any positive integer number. The following features are representing numeric features and each may refer to spam or ham:

- Message Length – It contains the total length of the whole message, including symbols, special characters, spaces, emojis, etc. Spammers usually present their words with a more extended number of characters comparing to regular messages. This numeric feature primarily counts the length of the message then represents it in numbers. For instance, the feature value for this message: "Customer service annoncement.You have a New Years delivery waiting for you. Please call 07046744435 to arrange delivery" equals to 119.
- Count of Slashes – Counting the number of slash contained in the message can be a good indicator for spam messages. In this example "Isabela (25/F) is

inviting for friendship. Reply YES/175 or NO/175 See her: www.SMS.ac/u/isabela2k9 STOP? Send STOP FRND to 62468" the value of current feature is 5. It shows that spammers use this type of punctuation specifically in some of the cases.

- Count of non-alphanumeric chars – The value of this feature can be considered by counting various types of non-alphanumeric characters included in the message. For instance, it will count different sorts of characters such as comma (,), colon (:), semicolon (;), dash (-), apostrophe (’), quotation marks (“), period (.), question mark (?), parentheses (()) and etc. For instance, “Dear Walmart shopper, last month you won ‘1000 USD’, Claim it within 24 hr (No2Cancel)” the value of this feature is 6.
- Count of alphanumeric words – Alphanumeric is the term deals with encompassing letters and numerals in the presented language set. If English language is taken as an instance, alphanumeric words can be composed of the combination of 26 alphabetic characters and 0 to 9 numbers. Additionally, counting the number of these words can also become another feature which is also presented in [8]. In this example, “Reply Y2sub and N2un” the value of this feature is 2.
- Count of numeric words – Included numbers or digits that present in the text can be counted and become the value of this feature. Principally, this feature only target numbers, punctuation and letters not covered. Spammers most of the time incorporate numeric characters in their text messages, for instance, “10/31 Bulls; 87, Kings 75 3 qtr. 1:02, Noah; 12pts 17 reb. Full info www.bulls.com”.
- Count of uppercase letters – Like the uppercase words feature, counting uppercase letters can be considered a good indicator of spam messages. Additionally, the value of this feature obtained by counting the number of uppercase letters. For instance, in "Claim YOUR Cash!!!" the feature value is equal to 6.
- Count of words – The word is a letter or a group of letters separated by space regarding the text message. This feature value attained by counting the number of words that formed the message body. For instance, in "Hello world, ML is Not so Hard To learn!!" the value of the feature is nine.

- Count of spam keywords – this feature is the same as the presence of spam keywords feature in terms of concept, but instead of putting 1 as the value of the feature where the keyword exists, it will count the spam keywords in the message as the value of the feature. Moreover, the keywords are the same trendy keywords that mentioned before.

Numeric feature extraction is the same as binary feature extraction in terms of using various sorts of regex. However, the value of numeric features will be natural number when specified regex is found from the message in the dataset. For instance, the value of non-alphanumeric words feature will be between zero and any natural number when non-alphanumeric words regex is found in the message. Mainly the value of the feature is discovered by counting the number of occurred regex from the text message.

Furthermore, the extraction of counting spam keywords feature needs a bit of modification to the message. Chiefly, it should be converted to lowercase letters before extraction of feature 8 like mentioned in binary feature extraction. Pseudocode (3.2) demonstrates the process of numeric feature extraction in the proposed system. Figure (3.7) illustrates how numeric feature extraction will be implemented using the GUI of the proposed system.

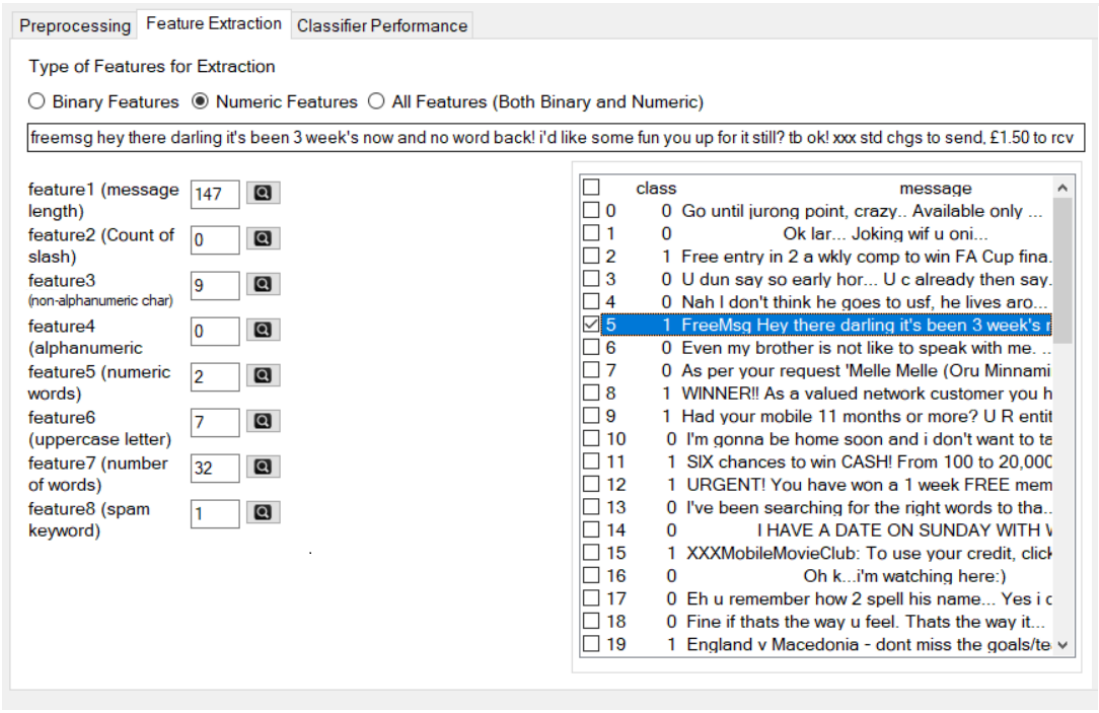


Figure (3.7) Extraction of Numeric Features using Proposed System Software

Pseudocode (3.2) Numeric Features Extraction

Required:
Get import numpy library and identify as np
Get import pandas library and identify as pd

Initialize
dataset_path, // path of the dataset
df, // an empty data frame
spam_keywords, // spam terms that are frequently used by spammers
X,y, // X: features data frame, y: Labels data frame
column_message, // corresponds to messages column from the dataset
column_class, // labels column (containing spam and ham)

Begin:
dataset_path READ spam dataset file path
df ← pd.read_csv(dataset_path) // pandas data frame of dataset csv file
column_message ← message column of dataset csv file
column_class ← class column of dataset csv file
spam_keywords ← array of spam terms for example: free, ringtone …etc.

// starting extraction of 8 binary features
df['feature1'] = df[column_message].length() // get the length message

df['feature2'] = count of df[column_message].string.findall(Regex: 'Slash'*)*

df['feature3'] = count of df[column_message].string.findall(Regex: 'Punctuation'*)*

df['feature4'] = count of df[column_message].string.findall(Regex: 'AlphanumericWords'*)*

df['feature5'] = count of df[column_message].string.findall(Regex: 'NumericWords'*)*

df['feature6'] = counting the capital letters from df['message']

df['feature7'] = count of df[column_message].string.findall(Regex: 'NumOfWords'*)*

df[column_message].lower() // before extracting feature 8, messages will be lowercase letters
df['feature8'] = count df[column_message].string.findall(Regex: spam_keywords)

X ← df[['feature1','feature2' … 'feature8']] //data frame of features

y ← df[column_class] //data frame of labels (spam and ham)

End;

3.3.5.3 All features extraction

In the proposed system, extracted features are grouped as binary, numeric and all features. All features here means containing both binary and numeric features which are all have been explained in this study. It should also be noted that a bit of modification should be done before spam keywords features extraction by replacing the whole messages in the dataset to lowercase letters; because the trigger keywords will be searched in small letters in the messages column. After that, extraction of features will be done by creating a feature vector which includes label, messages and features column. Each feature column contains its extracted value with regard to the corresponding message. Table (3.3) demonstrates all features, their names, values and references related to them.

Table (3.3) Description of Extracted Feature

Feature No.	Feature name	Type	Values	Reference
Feature1	Message length	Numeric	Natural number	[14][10]
Feature2	Count of Slash "/"	Numeric	Natural number	[8]
Feature3	Non-alphanumeric characters	Numeric	Natural number	[10] (presented as char ratio)
Feature4	Count of Alphanumeric words	Numeric	Natural number	[8]
Feature5	Count of numeric words	Numeric	Natural number	[8]
Feature6	Count of Uppercase letter	Numeric	Natural number	[10] (presented as char ratio)
Feature7	Number of words (terms)	Numeric	Natural number	[8][10]
Feature8	Math symbol	Binary	{1,0}	[14]
Feature9	Presence of URL	Binary	{1,0}	[14][10][15] [8]
Feature10	Presence of dots	Binary	{1,0}	[14]
Feature11	Special symbol	Binary	{1,0}	[14]
Feature12	Emotions	Binary	{1,0}	[14] [8]
Feature13	Lowercase words	Binary	{1,0}	[14]
Feature14	Uppercase words	Binary	{1,0}	[14]
Feature15	Phone number	Binary	{1,0}	[14]
Feature16	Currency (£, $, €, ¥) + number	Binary	{1,0}	[15]
Feature17	Number+% or number+percent	Binary	{1,0}	[15]
Feature18	Ring,tel,cal + number	Binary	{1,0}	[15]
Feature19	Date and time	Binary	{1,0}	[15]
Feature20	Long string number	Binary	{1,0}	[15]
Feature21	Email	Binary	{1,0}	[15]
Feature22	Presence Currency (£, $, €, ¥ …etc.)	Binary	{1,0}	Suggested in the study
Feature23	Service number	Binary	{1,0}	Suggested in the study
Feature24	Currency in word	Binary	{1,0}	Suggested in the study
Feature25	Count of spam keywords	Numeric	Natural number	[8]
Feature26	Presence of spam keywords	Binary	{1,0}	[14]

Moreover, feature extraction will be divided into two main branches according to the feature type. However, in this study, all the features are added in one group because our aim is to find the best optimal feature set that best represents the problem in terms of accuracy. Figure (3.8) illustrates the whole feature extraction in flowchart. Starting from reading the python file that do the extraction of feature from background, after that loading the result to the proposed program where the user can see.

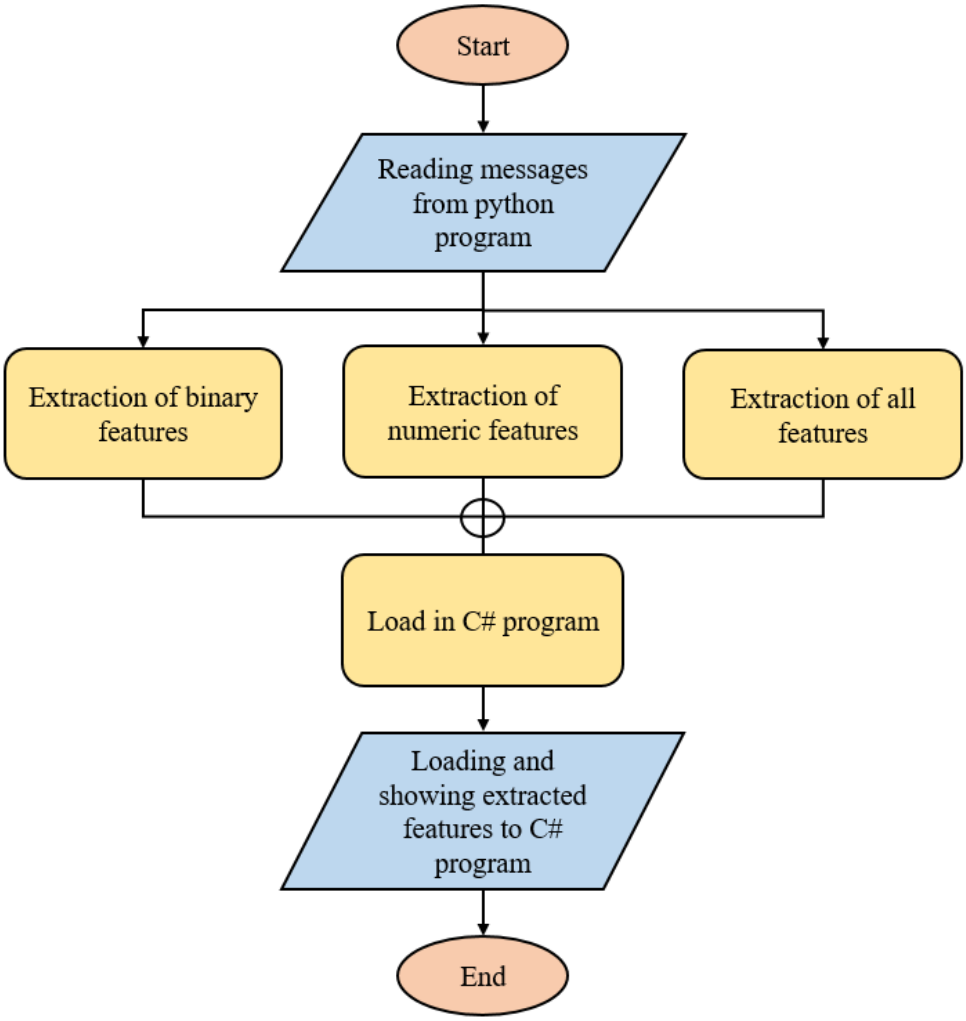


Figure (3.8) General Feature Extraction Flow Chart

According to the GUI, the user selects the group of features that wants to be extracted. Each feature labeled with number and description of it. The value of the feature will be shown from the GUI where the user clicks on the message. Figure (3.9) demonstrates extraction of all features regarding the GUI of the program.

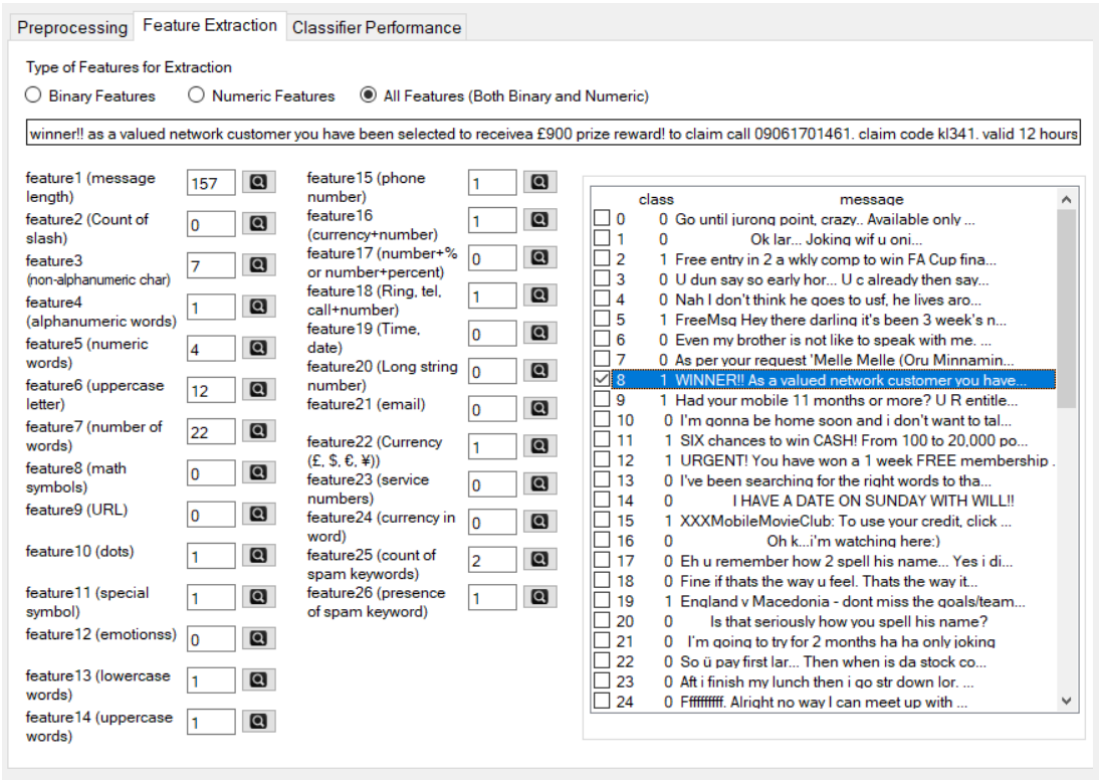


Figure (3.9) Extraction of All Features using Proposed System Software

3.4 Feature Selection Using Sequential Wrapper Feature Selector

In this section, sets of most relevant features have been selected for a better performing model. In other words, they can lead to remove and identify irrelevant, unneeded and even redundant features in the dataset that do not advance in accuracy or it may lead to decrement in the accuracy of the predictive model. The subset of the selected features will be the inputs for the classifier for better performance, faster and more cost effective models.

As mentioned in chapter 2, feature selection algorithms have different types, such as filter, wrapper, and embedded methods. While wrapper method can be classified as sequential feature selection or heuristic search algorithm, in this thesis, two types of sequential feature selection methods such as forward and backward selection were applied. The main purpose of these types of selection algorithms is to choose a feature subset that is most related to the learning problem.

In this study, from the practical point of view each time when the feature selection methods SFS and SBS are used, python library has been imported that had the

methods of implementation. The process of feature selection starts right after the extraction of features. For this process, a sum of 26 features will be passed to both SFS and SBS separately. Both SBS and SFS share common parameters like classifier model, which in this study contains gradient boosting algorithms. Each algorithm parameter will be passed to SBS or SFS. The parameter which composed of scoring type, accuracy will be assigned to it because these feature selection methods depend on accuracy score for the determination of best features in each iteration. Then, the last important parameter is cross validation for training and testing on the same dataset, it will be valued only on 10 because it is the most common. The result of these methods will be stored in text file, and then the final iteration will be in C# form to the user. Figure (3.10) explains the flowchart of sequential feature selection process. Figure (3.11) demonstrates proposed system user interface for feature selection and its parameters.

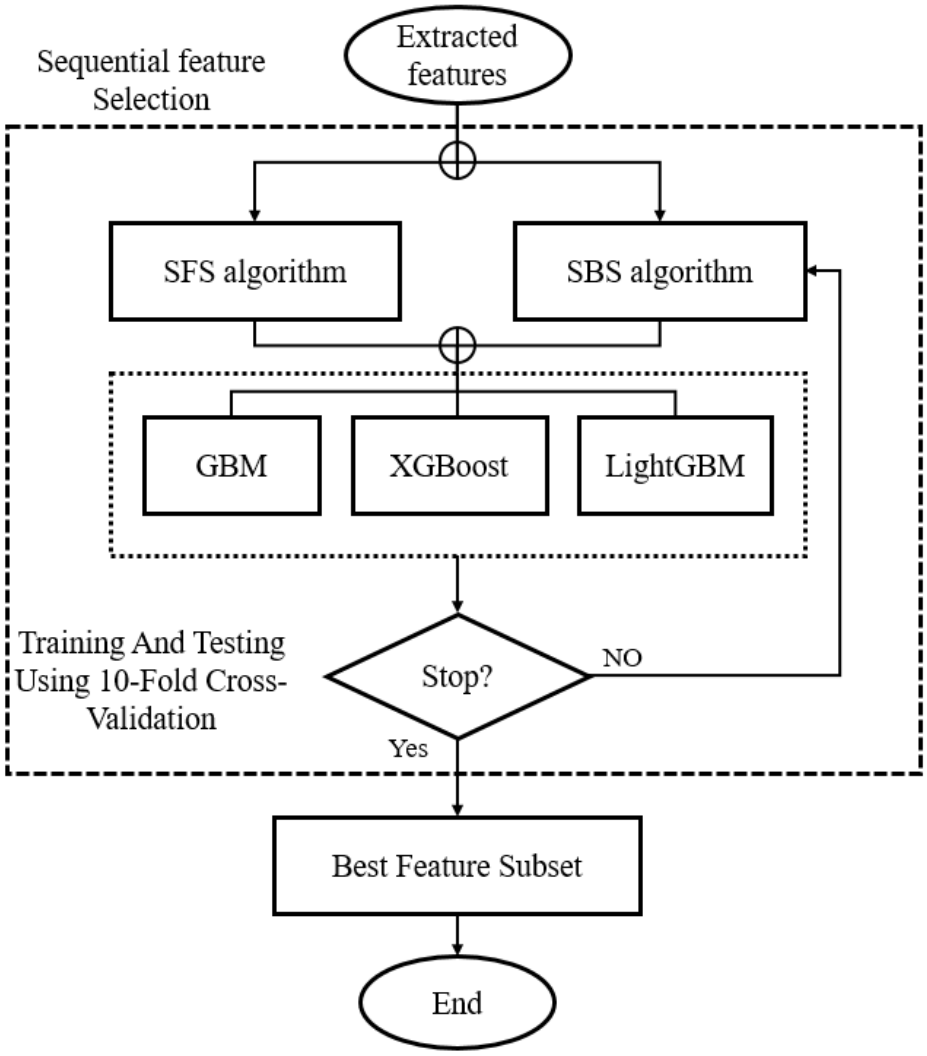


Figure (3.10) Flowchart of Sequential Feature Selection Method

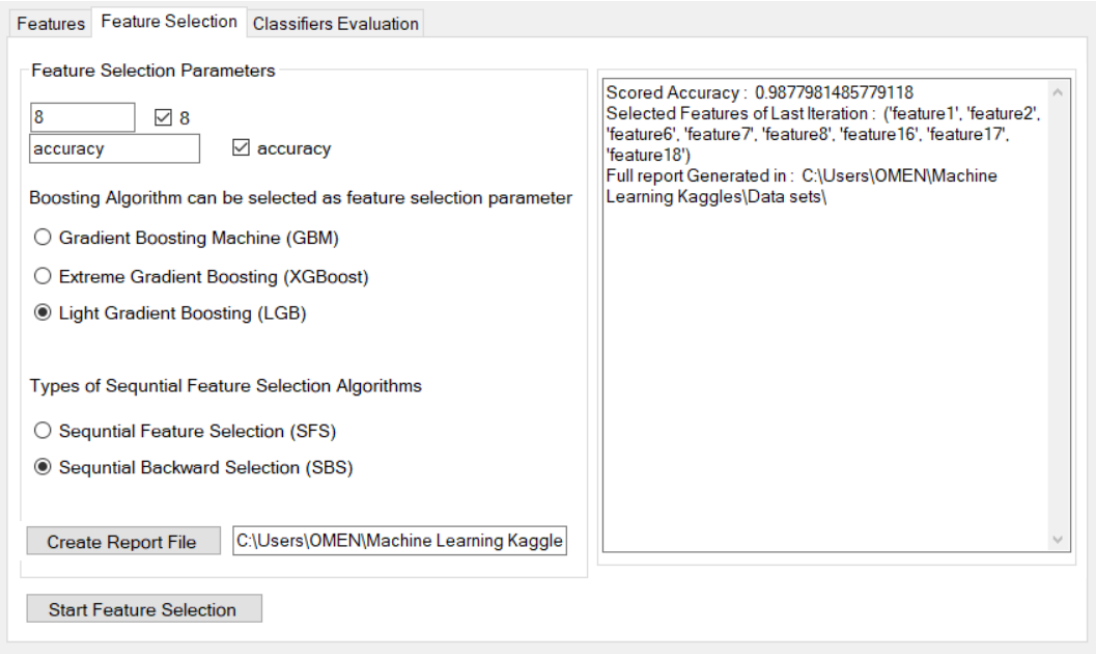


Figure (3.11) Proposed System GUI for Sequential Feature Selection Methods

3.5 Classification of Spam and Ham Messages

In this study, even though wrapper methods were used as feature selection methods, yet different classifiers were also applied during the process of feature selection. The reason why these methods were applied was to evaluate a subset of features in each iteration of the attribute selection method. In this study, the classifiers that will be implemented includes GBM, XGBoost and LightGBM classifiers. Each will be imported according to their specific python library then assigned to the SFS and SBS algorithms as a parameter. In other scenarios of this study, classifiers were compared, taking accuracy as the primary metric. Then other evaluation metrics will be used for evaluating the boosting classifiers besides taking time.

3.5.1 GBM Classifier

Machine learning library such as Scikit-learn provides a library for ensemble methods. GBM classifier can be implemented from “sklearn.ensemble”, the module contains ensemble-based approaches for classification. In this proposed system, GBM classifier will be used with its default parameters, means no modified value of parameter will be passed to it. The pseudocode (3.3) demonstrates process of implementing GBM for classification in the proposed study.

Pseudocode (3.3) classification process of GBM model

Required:

Get GradientBoostingClassifier function from sklearn ensemble library

Get cross validation score function from sklearn model selection library

---

Initialize

X, // feature vector created with datafeame of features from the dataset

y, // class variable contains dataset labels spam and ham

GBMModel, // gradient boosting machine model variable

CV, // cross-validation variable parameter from models classifier

scores, // variable that assigns the result of cross-validation function

Begin:

X ← data frame of features

y ← data frame of labels (spam and ham)

// classification using cross validation

GBMModel ← GradientBoostingClassifier(default params.)

scores ← cross_validation_score(GBMModel, X, y, CV=10, accuracy scoring type)

Compute:

scores mean // finding the mean of the scores which will be the accuracy // of the model using 10-fold cross-validation

***Print** scores mean // outputting the mean of cross validation scores*

End;

3.5.2 XGBoost Classifier

Developers created the python package of XGBoost algorithm that is publically available. In this study, as a first step XGBoost package is downloaded, and then installed into specified directory. Next, in this proposed system, XGBoost classifier will be imported from installed xgboost library. XGBClassifier will be used with its default parameters without tuning any of them. The pseudocode (3.4) demonstrates process of implementing XGBoost for classification in this proposed study.

Pseudocode (3.4) classification process of XGBoost model

Required:

Get XGBClassifer function from xgboost library

Get cross validation score function from sklearn model selection library

---

Initialize

X, // feature vector created with datafeame of features from the dataset

y, // class variable contains dataset labels spam and ham

XGBModel, // extreme gradient boosting model variable

CV, // cross-validation variable parameter from models classifier

scores, // variable that assigns the result of cross-validation function

Begin:

X ← data frame of features

y ← data frame of labels (spam and ham)

// classification using cross validation

XGBModel ← XGBClassifer(default params.)

scores ← cross_validation_score(XGBModel, X, y, CV=10, accuracy scoring type)

Compute:

scores mean // finding the mean of the scores which will be the accuracy // of the model using 10-fold cross-validation

***Print** scores mean // outputting the mean of cross validation scores*

End;

3.5.3 LightGBM Classifier

LightGBM is a tree-based learning model like GBM and XGBoost that will be used to filtrate spam and ham messages. In this study, using LightGBM algorithm requires downloading its package and dependencies. In addition, for importing LightGBM's classifier, it is essential to install lightgbm package on the host machine. Totally like GBM and XGBoost, LGBMClassifier will be implemented without tuning any parameters. Means default parameters will be passed to it when training and testing

applied using 10-fold cross-validation. The pseudocode (3.5) demonstrates the process of implementing LightGBM for classification in this proposed study.

Pseudocode (3.5) classification process of LightGBM model

Required:

Get LGBMClassifier function from lightgbm library

Get cross validation score function from sklearn model selection library

---

Initialize

X, // feature vector created with datafeame of features from the dataset

y, // class variable contains dataset labels spam and ham

LGBMModel, // light gradient boosting machine model variable

CV, // cross-validation variable parameter from models classifier

scores, // variable that assigns the result of cross-validation function

Begin:

X ← data frame of features

y ← data frame of labels (spam and ham)

// classification using cross validation

XGBModel ← LGBMClassifier(default params.)

scores ← cross_validation_score(LGBMModel, X, y, CV=10, accuracy scoring type)

Compute:

scores mean // finding the mean of the scores which will be the accuracy // of the model using 10-fold cross-validation

***Print** scores mean // outputting the mean of cross validation scores*

End;

Moreover, the classification using the three boosting algorithms can result in different accuracy. The proposed system software shows the result of each classifier when specific feature set is selected. Figure (3.12) shows the results of classifications via the software system.

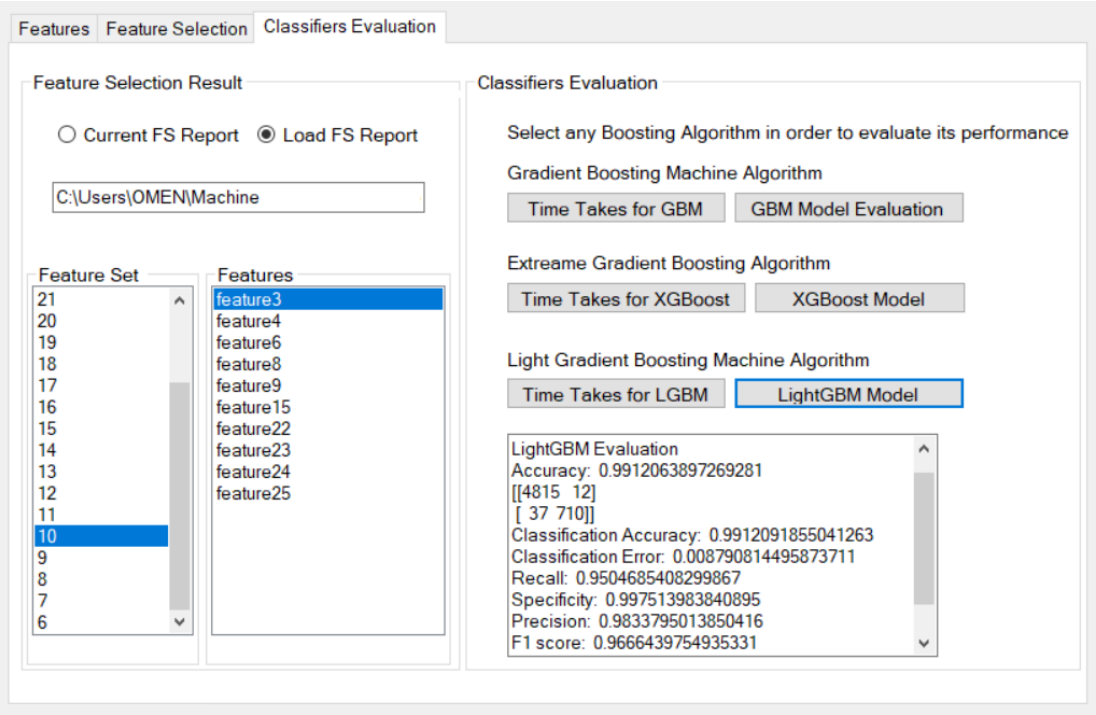


Figure (3.12) Gradient Boosting Classifiers Evaluation using Selected Feature Set

Chapter Four
Experiments and Results

4.1 Introduction

One of the important phases of research work is accomplishing accurate results for a system. In this chapter, the number of experiments is demonstrated along with explaining the test outcomes. According to the tests, three different gradient boosting algorithms were used for determining which one leads to a better accuracy in a shorter period of time. Machine learning algorithms alone are not enough to obtain a better accuracy; relevant features are even more important. Consequently, two wrapper sequential feature methods are implemented for selecting relevant features and eliminating redundant attributes in order to obtain which method leads to a better accuracy with different learning algorithms. Moreover, the following tests have been applied to reach a better accuracy:

- First, in this proposed system set of binary, numeric and combination of both numeric and binary features were extracted. Then, three boosting algorithms with each group of features have been tested in order to determine which group leads to a better performance.
- Second, the proposed system used two sequential wrapper feature selection methods with regard to boosting algorithms in order to determine which feature selection method leads to optimal feature subset with better accuracy.
- Third, the optimal feature subset was determined by accuracy and evaluation metrics that concerned with Spam filtering problem.
- Fourth, measuring time and accuracy for optimal selected features were done with regard to each gradient boosting model. The learning algorithms were compared in terms of accuracy, time and other evaluation metrics. Then, the optimal learning model was chosen.

Furthermore, one of the most significant tests that was conducted is when features were evaluated and selected using SFS and SBS methods. These wrapper methods lead to choose optimal feature sets, which led to higher accuracy in a shorter period of training and testing time. The experiment output was mostly based on the impact of features, which were created and collected based on spam and ham messaging

behaviors. The suggested features were evaluated in the end to see if they were relevant to the algorithm that gave the best accuracy or not. Throughout this chapter, all the experiment results will be demonstrated in tables and figures.

All the tests that have been performed in this study were done on the HP Omen computer with the following specifications demonstrated in table (4.1).

Table (4.1) Hardware and Software Specifications of The Computer used for The Experiments in This Study

Computer Hardware and Software	Specifications
Processor	Intel Core i7, 7th generation
Processor number of sockets, cores and threads	1,4 and 8
Processor information (socket specification)	Intel(R) Core(TM) i7-7700HQ CPU @ 2.80GHz
Memory type and size	DDR4 and 24 GBytes
Operating system (software)	Microsoft Windows 10 (10.0) Home 64-bit (Build 18362)

4.2 Involved Parameters of Sequential Feature Selection Methods

In this study, for implementing SFS and SBS methods, python library of useful tools, for the day-to-day data science tasks called Mlxtend (machine learning extensions), was used. At the beginning, the boosting classifier is initialized in order to be used later as a learning parameter in the selection method. Then, the sequential feature selector methods were imported from Mlxtend library. After that, different parameters were passed to the mentioned selection methods. The first parameter value is one of these learning methods: 'LGBMClassifier',' XGBClassifier' or 'GradientBoostingClassifier'. Furthermore, parameter K_feature value is a positive integer number which stopped the algorithm at a specified K number of feature while the algorithm was SFS or SBS. In SFS, the algorithm starts from one feature to the specified K number of feature while SBS starts from the whole set of given feature and stops at the K number of feature, then the forward parameter will be 'True' if SFS is implemented whereas for SBS its value will be 'False'. Although, floating parameter will be 'False' in both SFS and SBS because in this research, the concentration is not on Sequential Forward Floating Selection (SFFS) or Sequential Backward Floating Section (SBFS). Moreover, the scoring parameter value is 'accuracy' because the performance computation for selecting the features depends on the accuracy obtained in each iteration. Last but not least, cross-validation

parameter value was assigned with number 10 which stands for 10 fold cross-validation.

4.3 Involved Parameters of Gradient Boosting Algorithms

In this study, the default parameter of each boosting algorithms was used according to python programing. When the algorithm is primarily identified as the classifier model, no values will be passed to its parameters. In that case, the classifier function allows parameters to have their default values. Table (4.2) demonstrates algorithms such as GBM, XGBoost and LightGBM obtaining the following values to their parameters in default state.

Table (4.2) Parameter Names and Their Default Values Regarding Gradient Boosting Classifiers

GBM		XGBoost		LightGBM	
Parameter	**Value**	**Parameter**	**Value**	**Parameter**	**Value**
criterion	'friedman_mse'	booster	'gbtree'	boosting_type	'gbdt'
Init	None	base_score	0.5	class_weight	None
loss	'deviance'	colsample_bytree	1	colsample_bytree	1.0
max_features	None	colsample_bylevel	1	importance_type	'split'
learning_rate	0.1	learning_rate	0.1	learning_rate	0.1
max_depth	3	max_depth	3	max_depth	-1
max_leaf_nodes	None	max_delta_step	0	min_child_samples	20
min_impurity_decrease	0.0	min_child_weight	1	min_child_weight	0.001
min_impurity_split	None	missing	None	min_split_gain	0.0
n_estimators	100	n_estimators	100	n_estimators	100
min_samples_leaf	1	n_jobs	1	n_jobs	-1
min_samples_split	2	nthread	None	num_leaves	31
min_weight_fraction_leaf	0.0	objective	'binary:logistic'	Objective	None
random_state	None	random_state	0	random_state	None
n_iter_no_change	None	reg_alpha	0	reg_alpha	0.0
presort	'auto'	reg_lambda	1	reg_lambda	0.0
tol	0.0001	Silent	True	Silent	True
subsample	1.0	subsample	1	subsample	1.0
validation_fraction	0.1	scale_pos_weight	1	subsample_for_bin	200000
verbose	0	seed	None	subsample_freq	0
warm_start	False	gamma	0		

4.4 Training and Testing Using 10-Fold Cross-Validation

There are number of ways for evaluating performance of learning models. The common ways contain training accuracy, testing accuracy (Train/Test split) and K-fold cross-validation. In this study, 10-fold cross validation was used over the other two because it has more accurate estimate of out-of-sample accuracy and higher efficiency in data usage. Accordingly, the dataset of 5574 messages will be split into 10 equal partitions, using one fold as testing set and the training set will contain the union of other remaining folds. In the first iteration, the first 9 folds will be in training set while the tenth fold will be used as testing. In the second iteration, ninth fold will be used as testing and the other remaining folds will be used as training and so on.

Moreover, cross-validation is used in different experiments in this study such as finding best feature group and better selection method by accuracy of the model. For measuring the effectiveness of learning algorithm in classification problem such as spam filtering, accuracy alone is not enough because the dataset used in this study is highly imbalanced dataset. The following metrics that are based on confusion matrix, accuracy, precision, etc. will be focused on in this binary classification problem.

4.4.1 Confusion Matrix for Spam Filtering

For finding correctness of the model, confusion matrix is the best choice. It is a table that usually used for defining classification models performance. Table (4.3) demonstrates confusion matrix for binary classification. It is usually used for classification problems. One of the important points of depending on confusion matrix is that almost all of the other metrics are based on it. Confusion matrix observes the following metrics:

- True Positive (TP): in these scenarios, message was a spam and also truly labeled as spam.
- True Negative (TN): these are the cases were the text message was not a spam and also labeled as not a spam.
- False Positive (FP): in these sorts of scenarios, the SMS was not a spam, but it was labeled as spam.
- False Negative (FN): these are the cases were the message was spam, but was not labeled as spam, it is predicted as normal message.

Table (4.3) Confusion Matrix

Class	**Predicted Class**	
Actual class	Negative class (N)	Positive class (P)
Negative class (N)	TN	FP
Positive class (P)	FN	TP

4.5 Test Results for Features Category Selection

In this experiment, the three categories of feature sets containing binary, numeric and combination of both were separately extracted and created vectors of features. Then, they were directly fed into the gradient boosting algorithms such as GBM, XGboost and LightGBM. For finding the accuracy of each classifier, 10-fold cross-validation was used as the models evaluation procedure. Classification accuracy which is the percentage of correct prediction was the outcome that the three categories of features mainly depended on when they got compared. Usually in spam filtering problems, besides the accuracy of the model, confusion matrix should also be considered, which identifies the classification performance. Yet, in this stage, the main concentration was on accuracy only. The following tables of (4.4), (4.5), and (4.6) demonstrate the results of the comparison among binary, numeric and combination of both features. In addition, the results of accuracy of this model depended on the different boosting classifiers. Finally, the category of feature set was chosen that led in a better accuracy in the upcoming experiments.

Table (4.4) GBM Accuracy Result of Different Categories of Features

Evaluation metrics	**Binary Features**	**Numeric features**	**Combination of both**
Accuracy	0.9882	0.9788	0.9901
TP	705	668	703
TN	4803	4788	4816
FP	24	39	11
FN	42	79	44

Table (4.5) XGBoost Accuracy Result of Different Categories of Features

Evaluation metrics	**Binary Features**	**Numeric features**	**Combination of both**
Accuracy	0.9874	0.9776	0.9896
TP	702	661	696
TN	4802	4788	4820
FP	25	39	7
FN	45	86	51

Table (4.6) LightGBM Accuracy Result of Different Categories of Features

Evaluation metrics	Binary Features	Numeric features	Combination of both
Accuracy	0.9871	0.9824	0.9891
TP	696	681	698
TN	4806	4795	4815
FP	21	32	12
FN	51	66	49

Furthermore, the results from figure (4.1) show that when both numeric and binary are combined as feature set, the classification accuracy records the best regarding each boosting algorithm. According to the outcome, GBM's accuracy score tops both XGBoost and LightGBM, when feature set is binary and contains both; while LighGBM scores best when it is numeric feature. Thus, the feature set that holds all features will be chosen because of its better scoring of accuracy rather than the binary features for later experiments.

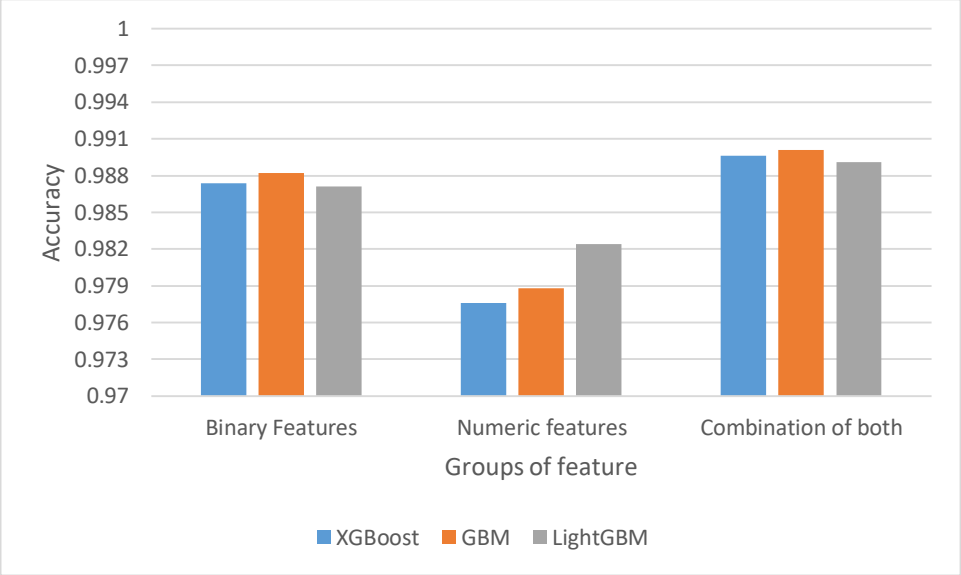


Figure (4.1) Evaluation of Three Gradient Boosting Classifiers using Different Categories of Features

4.6 Test Results for Determining Accurate Feature Selection Method

After selecting the group of feature that will be fed into sequential wrapper feature selection method, another test which determines better outcome of accuracy among SFS and SBS was implemented. Therefore, the feature vector was created from the extraction of all features which proved to lead to a better accuracy. Firstly, in this experiment, SFS was tested for selection of features and, after that SBS was tested.

The aim of these tests is to compare the SFS to the SBS methods in terms of leading to better performance regarding each learning model.

4.6.1 SFS Using Each Gradient Boosting Algorithm

The method SFS can be used when forward parameter equals to 'true' from sequential feature selector algorithm. Accordingly, the other parameters should be tuned properly, for instance, CV parameter to 10, scoring to 'accuracy' and k_features to 12. When k_features equal to 12, it means that this method will start from feature 1 and adds up 1 feature till it reaches 12 then it stops. Moreover, 12 features are chosen as a limit because if a higher number of features are selected, then the time will increase. The less number of features, the less time it takes to finish the classification. Additionally, when a higher number of k_features is taken, accuracy does not score better results.

In addition to assigning values to the parameters, classifier must be fed to the SFS method. Mainly, in this proposed system, algorithms such as GBM, XGboost and LightGBM will be assigned as a classifier separately in order to determine which one scores the best accuracy and determine the optimal feature subset.

4.6.1.1 SFS using GBM model

In this test, GBM was assigned as classifier in SFS algorithm. Different outcome is obtained as it is shown in table (4.7) and illustrated in figure (4.2). GBM parameters are all defaults, in other words, no parameter tuning was done regarding the classifier. In contrast, SFS was tuned with cross-validation of 10, k_feature equals to 12 and scoring parameter to accuracy.

Table (4.7) Accuracy Score of Selected Features in Each Iteration using SFS algorithm and GBM classifier

No. of features	Feature subsets	Avg. Accuracy Score
1	f5	0.94636
2	f5,f25	0.96788
3	f5,f15,f25	0.97668
4	f5, f9, f15, f25	0.98208
5	f5,f9,f15,f23,f25	0.98547
6	f5,f6,f9,f15,f23,f25	0.98762
7	f5,f6,f7,f9,f15,f23,f25	0.9887
8	f5,f6,f7,f9,f10,f15,f23,f25	0.9887
9	f5,f6,f7,f9,f10,f15,f17,f23,f25	0.98924
10	f5,f6,f7,f9,f10,f15,f17,f23,f24,f25	0.98941
11	f5,f6,f7,f9, f10,f13,f15,f17,f23,f24,f25	0.98995
12	f2,f5,f6,f7, f9,f10,f13,f15,f17,f23,f24,f25	0.99013

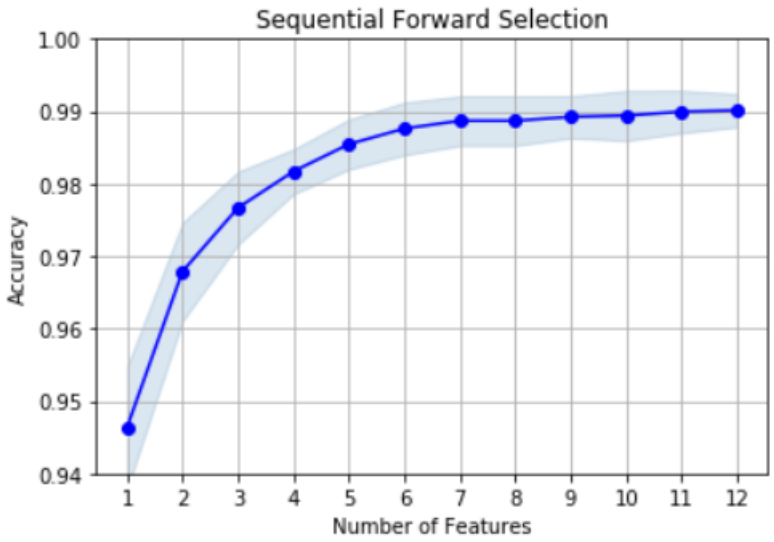


Figure (4.2) Accuracy Score of Different Feature Sets using GBM Regarding SFS

4.6.1.2 SFS using XGBoost model

In this experiment, XGBoost was assigned to classifier parameter of SFS algorithm. In each iteration, cross-validation of 10-fold is performed using XGBoost Classifier. Because the training and testing in each iteration done via tuning CV parameter of SFS. Moreover, the outcome is demonstrated in the table (4.8); it contained the feature subsets of each iteration when the SFS method was used, and its classifier parameter tuned with XGBoost algorithm with its default parameters.

Table (4.8) Accuracy Score of Selected Features in Each Iteration According to SFS algorithm and XGBoost classifier

No. of features	Feature subsets	Avg. Accuracy Score
1	f5	0.94636
2	f5,f25	0.96717
3	f5,f15,f25	0.97649
4	f5,f15,f23,f25	0.98224
5	f5,f9,f15,f23,f25	0.98439
6	f5,f6,f9,f15,f23,f25	0.98744
7	f3,f5,f6,f9,f15,f23,f25	0.98906
8	f3,f5,f6,f8,f9,f15,f23,f25	0.98977
9	f3,f5,f6,f8,f9,f13,f15,f23,f25	0.99031
10	f3,f5,f6,f8,f9,f13,f15,f18,f23,f25	0.99049
11	f3,f5,f6,f8,f9,f10,f13,f15,f18,f23,f25	0.99067
12	f3,f5,f6,f8,f9,f10,f13,f15',f17,f18,f23,f25	0.99067

Additionally, figure (4.3) illustrates how SFS iterations score accuracy using subsets of features. It illustrates that in every iteration, when a feature added, it led to the best

accuracy in the feature subset. Then the outcome accuracy increased gradually in every feature subset.

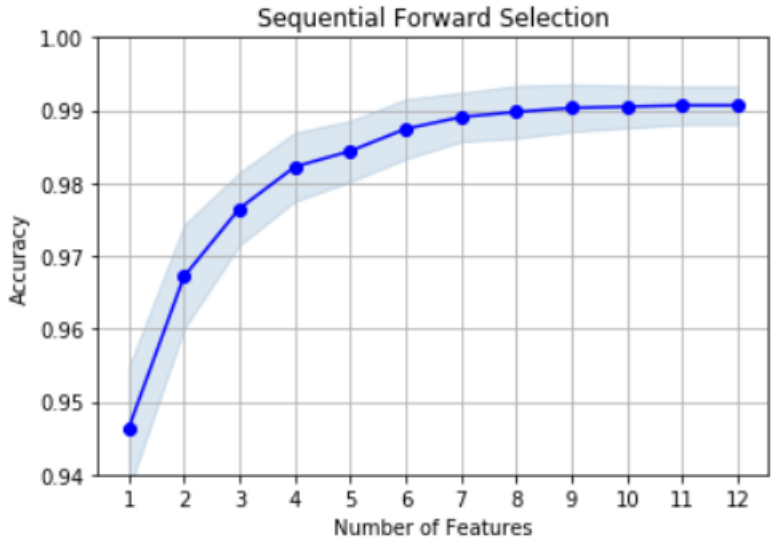


Figure (4.3) Accuracy Score of Different Feature Sets using XGBoost Regarding SFS

4.6.1.3 SFS using LightGBM model

The last algorithm that was assigned in SFS classifier parameter is LightGBM. Like GBM and XGBoost default parameter passed to the LightGBM classifier. This algorithm also recorded different outcomes which is shown in table (4.9).

Table (4.9) Accuracy Score of Selected Features in Each Iteration According to SFS algorithm and LightGBM classifier

No. of features	Feature subsets	Avg. Accuracy Score
1	f5,	0.94636
2	f5, f25	0.96735
3	f5, f15, f25	0.9765
4	f5, f9, f15, f25	0.9817
5	f5, f6, f9, f15, f25	0.98582
6	f5, f6, f9, f15, f23, f25	0.98834
7	f3, f5, f6, f9, f15, f23, f25	0.98995
8	f3, f5, f6, f9, f15, f23, f24, f25	0.99085
9	f3, f5, f6, f9, f15, f17, f23, f24, f25	0.99085
10	f3, f5, f6, f9, f15, f17, f18, f23, f24, f25	0.99085
11	f3, f5, f6, f9, f15, f17, f18, f19, f23, f24, f25	0.99085
12	f3, f5, f6, f9, f15, f17, f18, f19, f20, f23, f24, f25	0.99085

In figure (4.4), accuracy score gradually increases every time a feature added to the feature subset in each iteration. The curve from the figure also illustrate that these subsets 8,9,10 and 11, score the best accuracy comparing to other subsets.

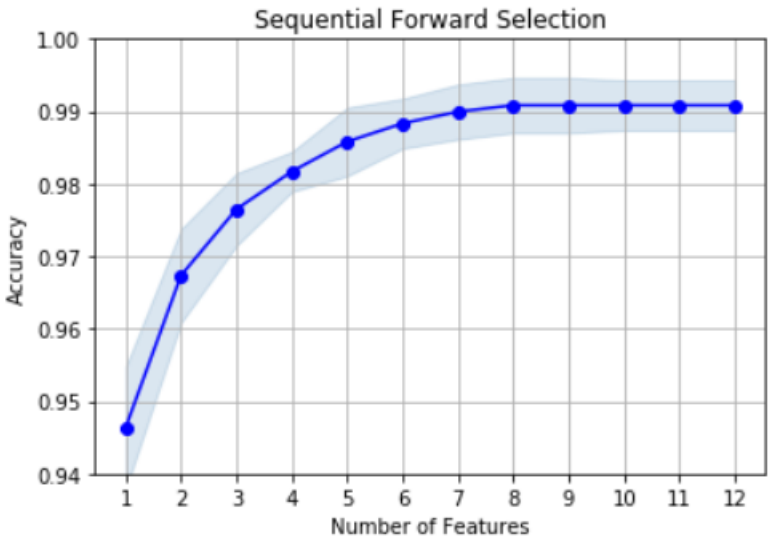


Figure (4.4) Accuracy Score of Different Feature Sets using LightGBM Regarding SFS

4.6.2 SBS Using Each Gradient Boosting Algorithm

Opposite to SFS method, SBS will be put in action when parameter forward equals to 'false' from sequential feature selector method. Similar to SFS, these parameters should be tuned accordingly, CV parameter to 10 and scoring to 'accuracy' but k_features to 6. Number 6 is used because SBS starts from 26 feature and continues to eliminate less relevant features until it reaches 6, in that case, SBS stops after 20 iteration. Moreover, the optimal number of features, which are the reason for better accuracy, will be demonstrated.

The final parameter which is fundamental to accuracy score in each iteration is classifier parameter. It was assigned with the following values separately: GBM, XGboost and LightGBM. Each outcome regarding each algorithm was compared and the best one which led to a better accuracy was determined with optimal feature set.

4.6.2.1 SBS using GBM model

After finishing SFS experiments, SBS was implemented on different boosting algorithms. Starting with GBM that led to different accuracy in each iteration of SBS method besides the remaining features in subsets demonstrated in table (4.10). The results are also illustrated in figure (4.5).

Table (4.10) Accuracy Score of Selected Features in Each Iteration According to SBS depending on GBM Classifier

No. of features	Feature subsets	Avg. Accuracy Score
26	f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f12,f13,f14,f15,f16,f17,f18,f19,f20,f21, f22,f23,f24,f25,f26	0.99013
25	f1,f2,f3,f4,f5,f6,f8,f9,f10,f11,f12,f13,f14,f15,f16,f17,f18,f19,f20,f21,f22 ,f23,f24,f25,f26	0.99084
24	f1,f2,f3,f4,f5,f6,f8,f9,f10,f11,f12,f13,f15,f16,f17,f18,f19,f20,f21,f22,f23 ,f24,f25,f26	0.99103
23	f1,f2,f3,f4,f5,f6,f8,f9,f10,f11,f12,f13,f15,f16,f17,f18,f19,f20,f21,f23,f24 ,f25,f26	0.99103
22	f1,f2,f3,f5,f6,f8,f9,f10,f11,f12,f13,f15,f16,f17,f18,f19,f20,f21,f23,f24,f2 5,f26	0.99121
21	f1,f2,f3,f5,f6,f8,f9,f10,f11,f12,f13,f15,f16,f17,f18,f19,f20,f21,f23,f24,f2 5	0.99121
20	f1,f2,f3,f5,f6,f8,f9,f10,f11,f12,f13,f15,f16,f17,f18,f19,f21,f23,f24,f25	0.99121
19	f1,f2,f3,f5,f6,f8,f9,f11,f12,f13,f15,f16,f17,f18,f19,f21,f23,f24,f25	0.99103
18	f1,f2,f3,f5,f6,f8,f9,f11,f12,f13,f15,f16,f17,f18,f19,f23,f24,f25	0.99103
17	f1,f2,f3,f5,f6,f8,f9,f11,f13,f15,f16,f17,f18,f19,f23,f24,f25	0.99103
16	f1,f2,f3,f5,f6,f8,f9,f11,f13,f15,f16,f17,f19,f23,f24,f25	0.99103
15	f1,f2,f3,f5,f6,f8,f9,f11,f13,f15,f16,f17,f23,f24,f25	0.99085
14	f1,f2,f3,f5,f6,f8,f9,f11,f15,f16,f17,f23,f24,f25	0.99085
13	f1,f3,f5,f6,f8,f9,f11,f15,f16,f17,f23,f24,f25	0.99085
12	f1,f3,f5,f6,f8,f9,f15,f16,f17,f23,f24,f25	0.99103
11	f1,f3,f6,f8,f9,f15,f16,f17,f23,f24,f25	0.99031
10	f3,f6,f8,f9,f15,f16,f17,f23,f24,f25	0.99121
9	f3,f6,f9,f15,f16,f17,f23,f24,f25	0.99085
8	f3,f6,f9,f15,f16,f23,f24,f25	0.99049
7	f3,f6,f9,f15,f23,f24,f25	0.99013
6	f3,f6,f9,f15,f23,f25	0.98941

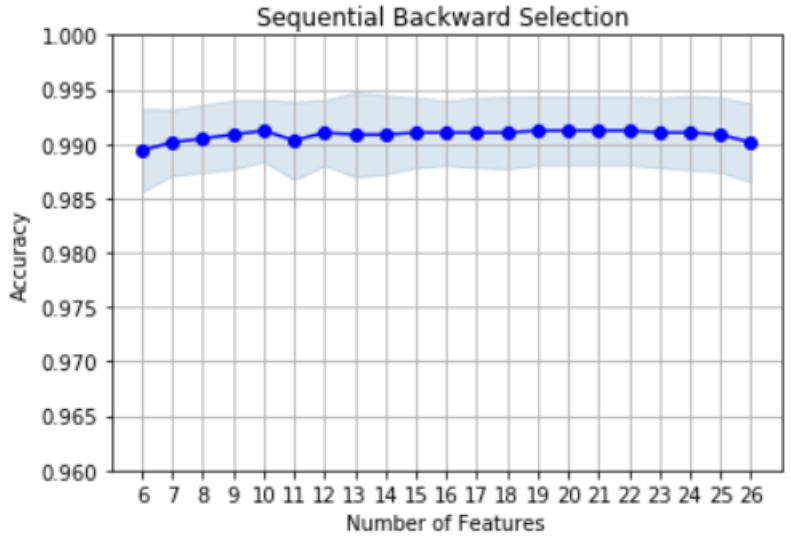


Figure (4.5) Numbers of Features Accuracy Score using GBM Regarding SBS

4.6.2.2 SBS using XGBoost model

XGBoost scored the following outcomes shown in table (4.11) when it was assigned in classifier parameter of SBS method. The results are illustrated in figure (4.6).

Table (4.11) Accuracy Score of Selected Features in Each Iteration According to SBS depending on XGBoost Classifier

No. of features	Feature subsets	Avg. Accuracy Score
26	f1,f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f12,f13,f14,f15,f16,f17,f18,f19,f20,f21, f22,f23,f24,f25,f26	0.98959
25	f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f12,f13,f14,f15,f16,f17,f18,f19,f20,f21,f22 ,f23,f24,f25,f26	0.99013
24	f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f13,f14,f15,f16,f17,f18,f19,f20,f21,f22,f23 ,f24,f25,f26	0.99031
23	f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f13,f14,f15,f16,f17,f18,f19,f20,f21,f22,f23 ,f24,f25	0.99031
22	f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f13,f14,f15,f16,f17,f18,f19,f20,f22,f23,f24 ,f25	0.99031
21	f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f13,f14,f15,f16,f17,f18,f19,f22,f23,f24,f25	0.99031
20	f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f13,f14,f15,f16,f17,f18,f22,f23,f24,f25	0.99031
19	f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f13,f14,f15,f16,f18,f22,f23,f24,f25	0.99031
18	f2,f3,f4,f5,f6,f7,f8,f9,f10,f11,f13,f14,f15,f18,f22,f23,f24,f25	0.99031
17	f3,f4,f5,f6,f7,f8,f9,f10,f11,f13,f14,f15,f18,f22,f23,f24,f25	0.99013
16	f3,f4, f5,f6,f7,f8,f9,f10,f13,f14,f15,f18,f22,f23,f24,f25	0.99031
15	f3,f4, f5,f6,f7,f9,f10,f13,f14,f15,f18,f22,f23,f24,f25	0.99031
14	f3,f4,f6,f7,f9,f10,f13,f14,f15,f18,f22,f23,f24,f25	0.99031
13	f3,f4,f6,f7,f9,f13,f14,f15,f18,f22,f23,f24,f25	0.99049
12	f3,f4,f6,f9,f13,f14,f15,f18,f22,f23,f24,f25	0.99049
11	f3,f4,f6,f9,f13,f14,f15,f22,f23,f24,f25	0.99067
10	f3,f4,f6,f9,f14,f15,f22,f23,f24,f25	0.99049
9	f3,f6,f9,f14,f15,f22,f23,f24,f25	0.99067
8	f3,f6,f9,f15,f22,f23,f24,f25	0.99103
7	f3,f6,f9,f15,f23,f24,f25	0.99049
6	f3,f6,f9,f15,f23,f25	0.98977

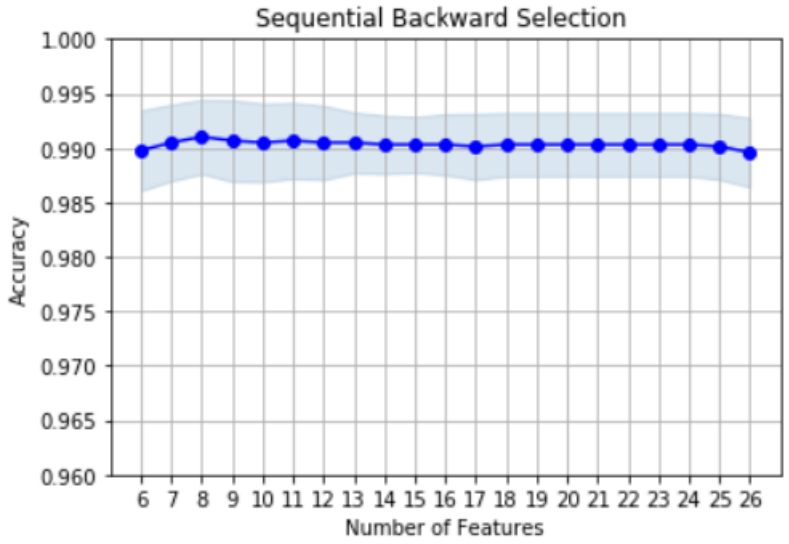


Figure (4.6) Numbers of Features Accuracy Score using XGBoost Regarding SBS

4.6.2.3 SBS using LightGBM model

LightGBM was assigned as a classifier parameter in SBS algorithm, and it obtained results that are demonstrated in table (4.12). Cross-validation parameter value was 10 and 6 assigned as a value number of feature parameter. That means the SBS method stopped at feature set including 6 features. LightGBM's classifier assigned with its default parameters, means no tuning performed regarding the boosting algorithm.

Table (4.12) Accuracy Score of Selected Features in Each Iteration According to SBS depending on LightGBM Classifier

No. of features	Feature subsets	Avg. Accuracy Score
26	f1,f2,f3,f4, f5,f6,f7,f8,f9,f10,f11,f12,f13,f14,f15,f16,f17,f18,f19,f20,f21, f22,f23,f24,f25,f26	0.98906
25	f1,f2,f3,f4, f5,f6,f7,f8,f9,f10,f11,f12,f13,f15,f16,f17,f18,f19,f20,f21,f22, f23,f24,f25,f26	0.98959
24	f1,f2,f3,f4, f5,f6,f7,f8,f9,f10,f11,f12,f13,f15,f16,f17,f18,f19,f20,f21,f22, f23,f24,f25	0.98959
23	f1,f2,f3,f4, f5,f6,f7,f8,f9,f10,f11,f12,f13,f15,f16,f17,f18,f19,f20,f22,f23, f24,f25	0.98959
22	f1,f2,f3,f4, f5,f6,f7,f8,f9,f10,f11,f12,f13,f15,f16,f17,f18,f19,f22,f23,f24, f25	0.98959
21	f1,f2,f3,f4, f5,f6,f7,f8,f9,f10,f11,f12,f13,f15,f16,f17,f18,f22,f23,f24,f25	0.98959
20	f1,f2,f3,f4, f5,f6,f7,f8,f9,f10,f11,f12,f13,f15,f16,f18,f22,f23,f24,f25	0.989594
19	f1,f2,f3,f4,f6,f7,f8,f9,f10,f11,f12,f13,f15,f16,f18,f22,f23,f24,f25	0.98959
18	f1,f2,f3,f4,f6,f7,f8,f9,f10,f11,f12,f13,f15,f18,f22,f23,f24,f25	0.99013
17	f1,f2,f3,f4,f6,f8,f9,f10,f11,f12,f13,f15,f18,f22,f23,f24,f25	0.99049
16	f1,f2,f3,f4,f6,f8,f9,f10,f11,f12,f13,f15,f22,f23,f24,f25	0.99067
15	f1,f2,f3,f4,f6,f8,f9,f10,f11,f13,f15,f22,f23,f24,f25	0.99103
14	f1,f2,f3,f4,f6,f8,f9,f10,f13,f15,f22,f23,f24,f25	0.99049
13	f1,f2,f3,f4,f6,f8,f9,f10,f15,f22,f23,f24,f25	0.99049
12	f2,f3,f4,f6,f8,f9,f10,f15,f22,f23,f24,f25	0.99048
11	f2,f3,f4,f6,f8,f9,f15,f22,f23,f24,f25	0.99084
10	f3,f4,f6,f8,f9,f15,f22,f23,f24,f25	0.99121
9	f3,f4,f6,f8,f9,f15,f22,f23,f25	0.99139
8	f3,f6,f8,f9,f15,f22,f23,f25	0.98995
7	f3,f6,f9,f15,f22,f23,f25	0.99013
6	f3,f6,f9,f15,f23,f25	0.98995

The outcomes illustrated in figure (4.7) show that, feature subsets such as 9, 10 and 11 recorded the best accuracy comparing to other subsets. According to the curve, in every iteration when a feature is eliminated, accuracy score may remain the same or improvement may happen. When accuracy stay the same, means having the particular feature in the subset is not effective to the accuracy so it will be eliminated. Consequently, eliminating less effective features in the subset, helps in performance improvement of the model.

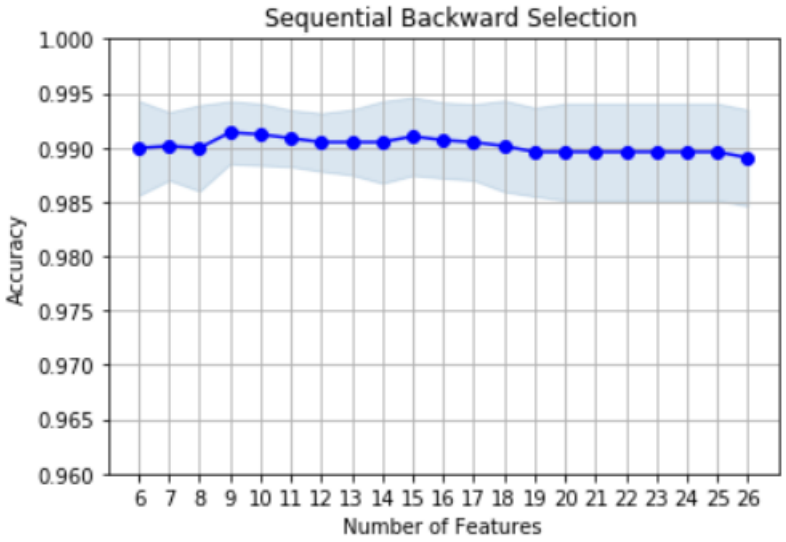


Figure (4.7) Numbers of Features Accuracy Score using LightGBM Regarding SBS

4.6.3 Comparison between SFS and SBS Results According to Each Algorithm

After finishing the selection of features, according to the three gradient boosting algorithms separately, SFS and SBS were compared and obtained different accuracy among each feature set. From the results demonstrated in the following tables, SBS recorded better results when any boosting classifier was selected comparing to SFS in terms of number of features and accuracy.

The accuracy GBM algorithm scored using SBS was better than the score obtained using SFS. In each feature set of 8, 9, 10 and 11 accuracy score of SBS is better than SFS. Table (4.13) demonstrates the result of both SBS and SFS comparison using GBM as classifier.

Table (4.13) Comparison between SBS and SFS Accuracy using GBM Classifier

Number of features	SFS Accuracy	SBS Accuracy
8	0.9887	0.99049
9	0.98924	0.99085
10	0.98941	0.99121
11	0.98995	0.99031

Figure (4.8) illustrates the result of GBM classification and comparison between SBS and SFS. It illustrate that SFS scores the best result when the feature subset is 11, while SBS worst accuracy score by 11 features scores better results acquired by SFS. As a result, the accuracy score of SBS will be the optimal when GBM is taken as a classifier parameter.

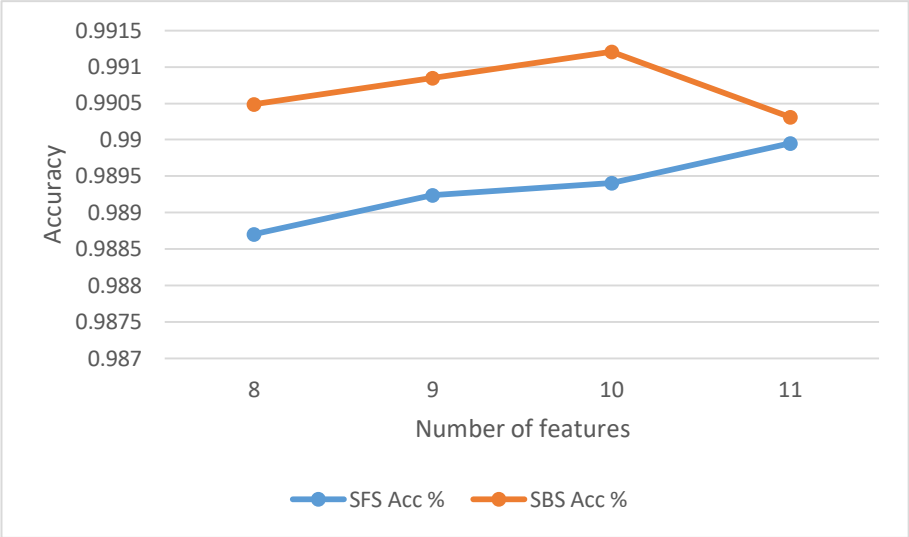


Figure (4.8) GBM's Most Accurate Feature Set of SBS and SFS Comparison

Moreover, the accuracy XGBoost scored via SBS was higher than SFS when feature 8 and 9 compared. Although, feature sets of 10 and 11 scored the same accuracy when SBS and SFS compared. Table (4.14) demonstrates the result of comparing SBS and SFS regarding XGBoost classifier. Figure (4.9) illustrates the accuracy result SBS and SFS scores when XGBoost is the Classifier.

Table (4.14) Comparison between SBS and SFS Accuracy using XGBoost Classifier

Number of features	SFS Accuracy	SBS Accuracy
8	0.98977	0.99103
9	0.99031	0.99067
10	0.99049	0.99049
11	0.99067	0.99067

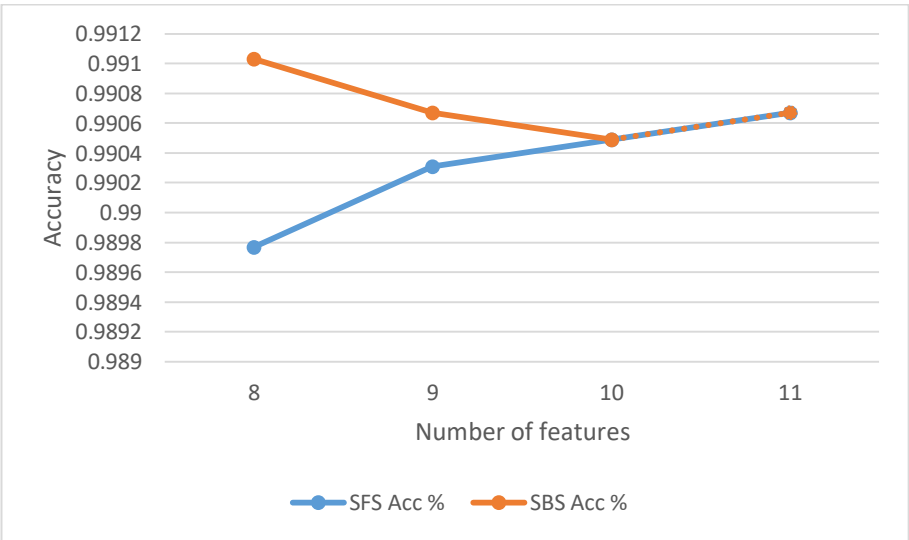


Figure (4.9) XGBoost's Most Accurate Feature Set of SBS and SFS Comparison

Lastly, LightGBM classifer scored better accuracy via SBS when feature 9 and 10 is considered. Whereas SFS scores higher when 11 feature is taken as instance of comparison. According to the overall results SBS is better feature selector than SFS when LightGBM is concerned. Table (4.15) demonstrates the result of comparing both SBS and SFS via LightGBM classifier. Figure (4.10) is the illustration of both accuracy results SBS and SFS scored.

Table (4.15) Comparison between SBS and SFS Accuracy using LightGBM Classifier

Number of features	SFS Accuracy	SBS Accuracy
8	0.99085	0.98995
9	0.99085	0.99139
10	0.99085	0.99121
11	0.99085	0.99084

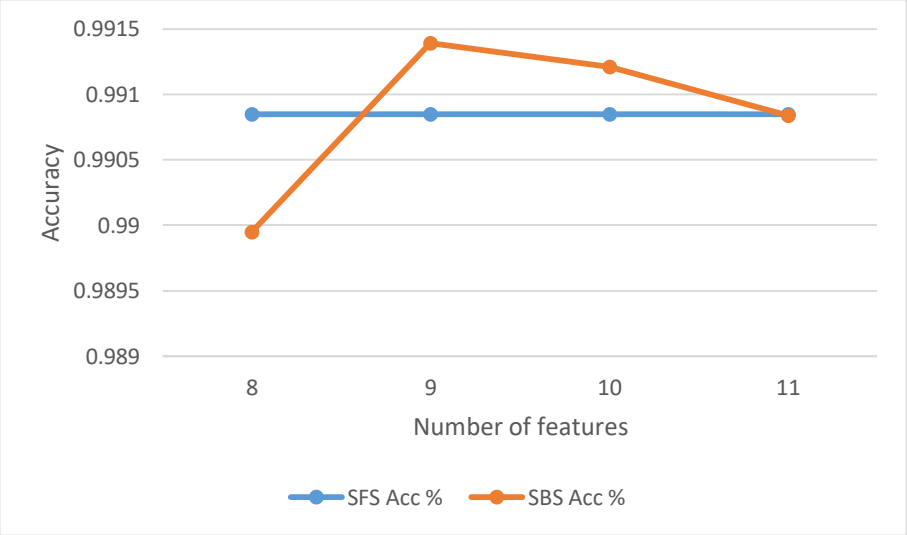


Figure (4.10) LightGBM's Most Accurate Feature Set of SBS and SFS Comparison

4.7 Test Results of Optimal Feature Set

After selecting the best features set by using SBS and SFS methods, the accuracy metric of selected attributes scored various results regarding each learning algorithm. The accuracy of the best selected features became a measure to compare SBS to the SFS, which consequently led the SBS to a better scoring of accuracy. In this test, the aim was to determine the optimal number of features among the learning algorithms, and SBSs best results were selected according to each classifier.

4.7.1 GBM with Optimal Feature Set

According to the results GBM obtained, ten features led to the best scoring of accuracy, recall and precession. Table (4.16) demonstrates the final results among the several outcomes of each feature set obtained during classification. Furthermore, the time for the optimal feature set which are ten scored the third place in terms of time-taking for classification by using 10-fold cross-validation. Table (4.17) shows the classification time of best features subsets. Figure (4.11) illustrate the comparison of feature sets regarding to the classification metrics. In figure (4.12) the classification time of four optimal feature set regarding GBM is illustrated.

Table (4.16) Comparing GBM Performance via Four Best Feature Subsets

No. of features	Accuracy	Recall	Specificity	Precision	F-measure
8	0.99049	0.94645	0.99731	0.98194	0.96387
9	0.99085	0.94779	0.99751	0.98333	0.96524
10	0.99121	0.94913	0.99772	0.98472	0.9666
11	0.99031	0.94244	0.99772	0.98462	0.96306

Table (4.17) Comparing GBM Classifier According to Classification Time and Feature Type

No. of features	Features Type		Avg. Time in Sec.
	No. binary feature	No. numeric feature	
8	5	3	1.739
9	6	3	1.805
10	7	3	1.941
11	7	4	2.112

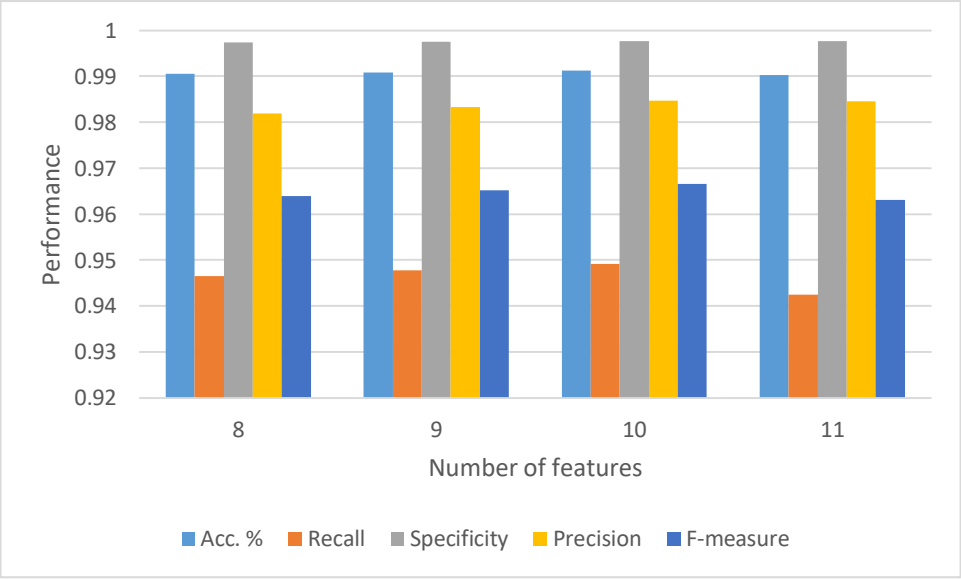


Figure (4.11) Evaluation Metric Scores Regarding Best Feature Sets using GBM Classifier

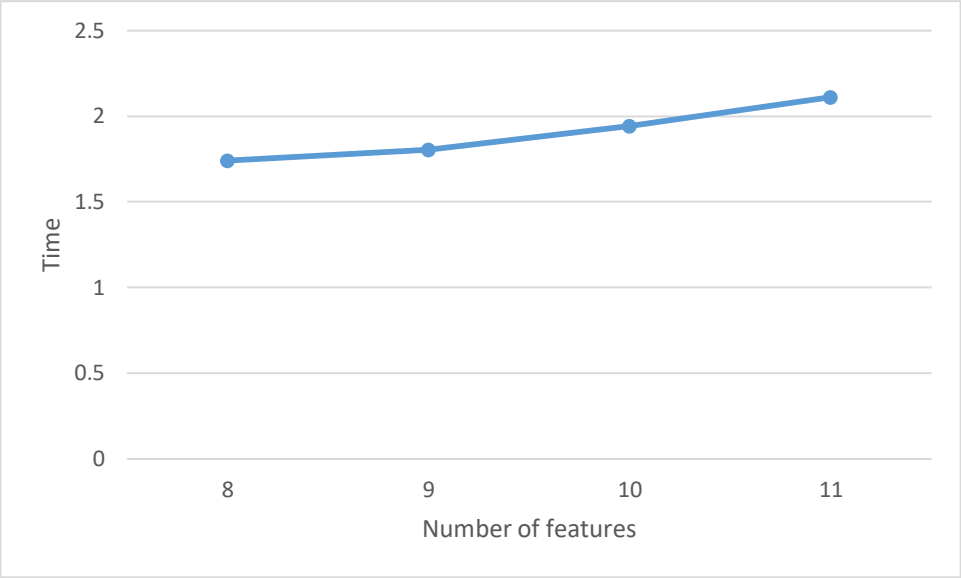


Figure (4.12) GBM Classification Time Scores of Best Feature Subsets

4.7.2 XGBoost with Optimal Feature Set

The results from table (4.18) demonstrate that XGBoost classifier has various scores of accuracy, recall and precision regarding each feature set. It is hard to determine optimal feature set among these four since feature set of eight scores are in a higher accuracy with best recall, while nine features have a higher precision. Additionally, in a case like that, f-measure will be taken into consideration. Thus, eight features were determined as the best among the other subsets because it is higher in accuracy, f-measure takes less time compared to nine features. Table (4.19) demonstrates the output of time of the four best subsets of features according to XGBoost algorithm.

Table (4.18) Comparing XGBoost Performance via Four Best Feature Subsets

No. of features	Accuracy	Recall	Specificity	Precision	F-measure
8	0.99103	0.94779	0.99772	0.9847	0.96589
9	0.99085	0.94378	0.99815	0.98739	0.96453
10	0.99049	0.94244	0.99793	0.98599	0.96382
11	0.99013	0.93976	0.99793	0.98596	0.96448

Table (4.19) Comparing XGBoost Classifier According to Classification Time and Feature Type

No. of features	Features Type		Avg. Time in Sec.
	No. binary feature	No. numeric feature	
8	5	3	2.891
9	6	3	3.118
10	6	4	3.27
11	7	4	3.388

Moreover, the comparison among the four optimal feature sets of XGBoost using classification metrics illustrated in figure (4.13). In figure (4.14) illustration of only

classification time has considered for comparing among the optimal feature sets regarding XGBoost. It also illustrate that, less number of feature takes less time for classification while cross-validation of 10 fold is implemented.

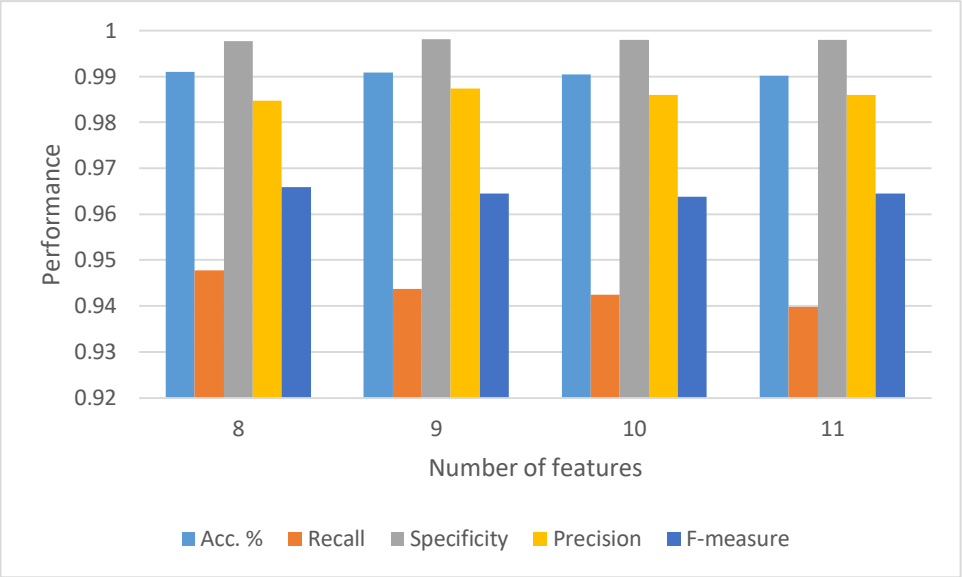


Figure (4.13) Evaluation Metric Scores Regarding Best Feature Sets using XGBoost Classifier

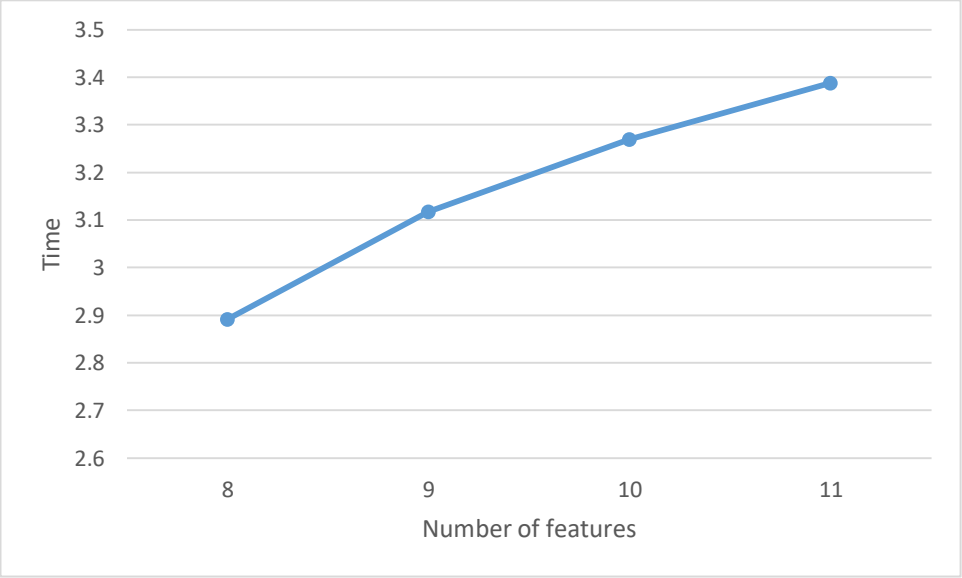


Figure (4.14) XGBoost Classification Time Scores of Best Feature Subsets

4.7.3 LightGBM with Optimal Feature Set

The results from table (4.20) show that LightGBM has performed the best when feature number was either nine or ten because they obtained the best accuracy and recall. However, if other metrics such as specificity and precision were used, nine number of feature will perform the best, and it will be the optimal feature set for the

learning model. Although, from table (4.21) nine and ten number of features are also taking a time that is close to each other when cross validation is performed for the accuracy measurement. Nevertheless, still nine features take less time for training and testing with 10-fold cross-validation compared to ten. Thus, nine numbers of features can be taken as the optimal number feature regarding lightGBM, while ten features can be the second optimal feature set. Figure (4.15) and (4.16) illustrate the outcomes of the experiments.

Table (4.20) Comparing LightGBM Performance via Four Best Feature Subsets

No. of features	Accuracy	Recall	Specificity	Precision	F1-measure
8	0.98995	0.94244	0.99731	0.98187	0.96175
9	0.99139	0.95047	0.99772	0.9847	0.9673
10	0.99121	0.95047	0.99751	0.98338	0.96664
11	0.99085	0.94913	0.99731	0.98199	0.96528

Table (4.21) Comparing LightGBM Classifier According to Classification Time and Feature Type

No. of features	Features Type		Avg. Time in Sec.
	No. binary feature	No. numeric feature	
8	5	3	1.238
9	5	4	1.263
10	6	4	1.266
11	6	5	1.275

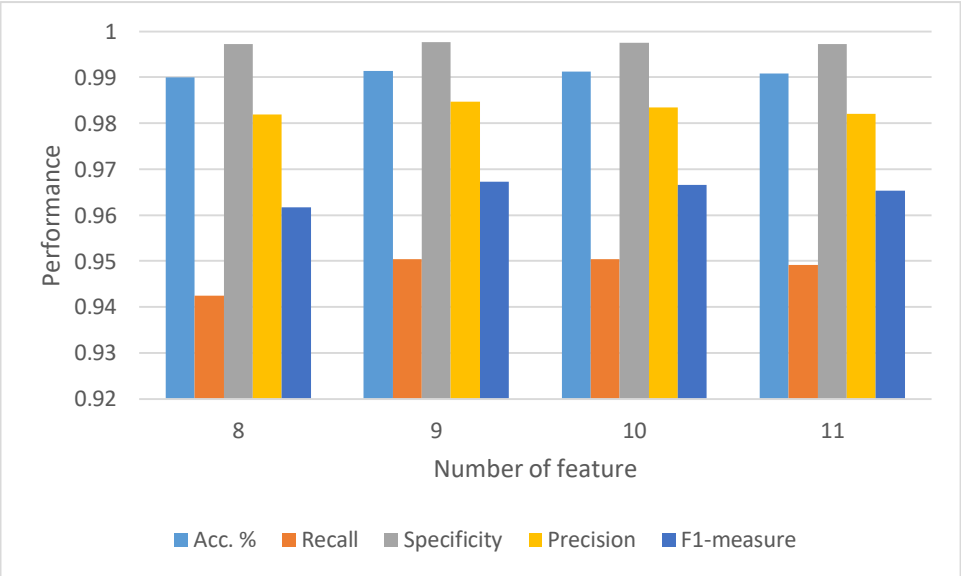


Figure (4.15) Evaluation Metric Scores Regarding Best Feature Sets using LightGBM Classifier

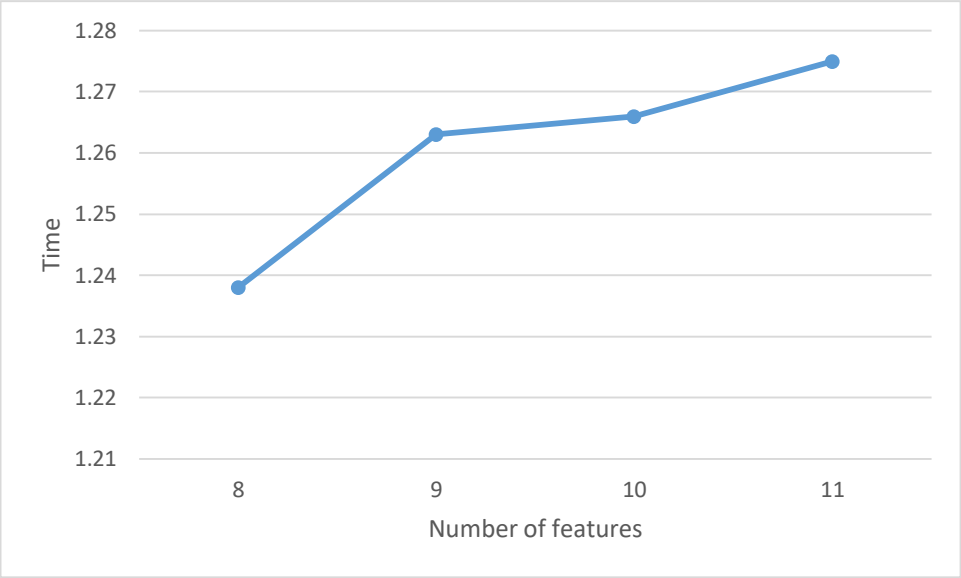


Figure (4.16) LightGBM Classification Time Scores of Best Feature Subsets

4.7.4 Comparison between Gradient Boosting Algorithms Results

The three boosting algorithms, which were used in this study, have scored different outcomes regarding accuracy, evaluation metrics and time. After finding the best feature set for each boosting classifier, better classifier was determined. Despite the fact that all three algorithms are almost the same in terms of accuracy, still there was one classifier chosen for this problem of spam detections' model. Moreover, according to the analysis of this study, the better results are achieved by LightGBM algorithm when the three algorithms are compared to each other. LightGBM scored 99.139% of accuracy when feature set was nine which is higher than the other two algorithms. In addition, it also scored 99.121% when ten features were used, which equals to GBM in accuracy but higher in recall. The recall which LightGBM scored was the highest when either nine or ten features were taken as optimal feature set. Table (4.22) demonstrates the comparison between the three boosting classifier. Figure (4.17) illustrates the comparison of the three classifiers.

Table (4.22) Comparison among Algorithms using Accuracy and Evaluation metrics

Algorithm	No. of features	Accuracy	Recall	Specificity	Precision	f-measure
GBM	10	0.99121	0.94913	0.99772	0.98472	0.9666
XGBoost	8	0.99103	0.94779	0.99772	0.9847	0.96589
LightGBM	9	0.99139	0.95047	0.99772	0.9847	0.9673
	10	0.99121	0.95047	0.99751	0.98338	0.96664

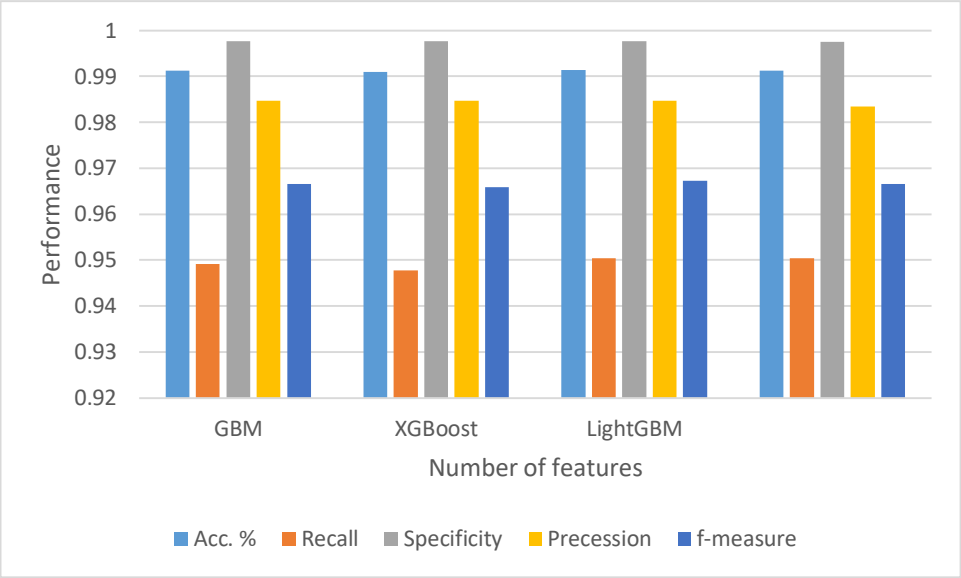


Figure (4.17) Comparison between Gradient Boosting Classifiers Performance

The lightGBM classifier is not only more precise in terms of accuracy and other evaluation metrics, but it also tops the other two algorithms in terms of classification time. It primarily scores the best classification time when 10-fold cross-validation is used. According to table (4.23) LightGBM took less time for training and testing, and figure (4.18) illustrate the result.

Table (4.23) Comparison among Algorithms using Classification Time

Algorithm	No. of features	Features Type		Avg. Time in Seconds
		No. binary feature	No. numeric feature	
GBM	10	7	3	1.941
XGBoost	8	5	3	2.891
LightGBM	9	5	4	1.263

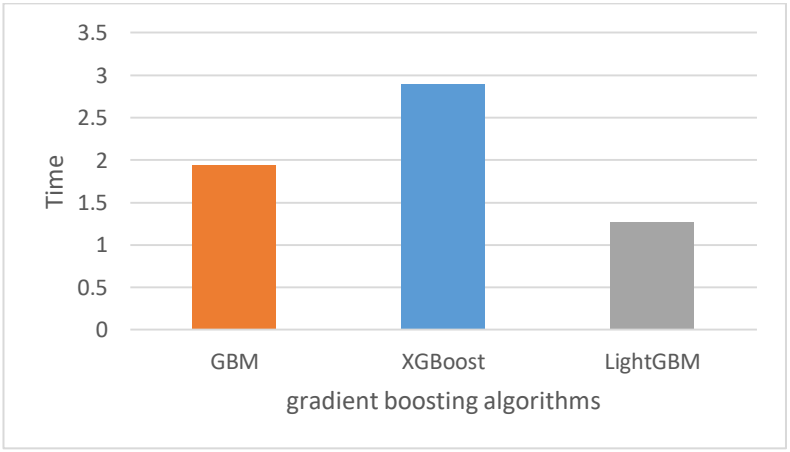


Figure (4.18) Comparison between Gradient Boosting Classifiers Regarding Classification Time

4.8 Determining Relevant Features

After selecting the model that best filters spam and normal messages, which is LightGBM, its feature set will be analyzed in order to show the three suggested features in this study are relevant or not. According to the experiments nine as first optimal feature set and ten as second can lead to best performance from the model. Table (4.24) demonstrates the feature name and its description regarding the model's best performance. The three features proposed in this study can be considered as relevant features to spam problem because all three are included in the second best feature set while just two of them are part of the very best feature set.

Table (4.24) Optimal Feature Set of the Proposed Model

Best feature subset (9)	Second best feature subset (10)	Feature description	Feature type
Feature3	Feature3	Count of punctuations	numeric
Feature4	Feature4	Count of Alphanumeric words	numeric
Feature6	Feature6	Count of Uppercase letter	numeric
Feature8	Feature8	Presence of Math symbol	binary
Feature9	Feature9	Presence of URL	binary
Feature15	Feature15	Phone number	binary
Feature22	Feature22 (proposed)	Currency (£, $, €, ¥…etc.)	binary
Feature23	Feature23 (proposed)	Service numbers	binary
Feature25	Feature25	Count of spam words	numeric
	Feature24 (proposed)	Currency in word	binary

In addition, when the best feature set regarding XGBoost is taken for determining the proposed features relevancy of SMS spam problem, the three suggested features are existing when 99.10 scored by XGBoost. However, only two features of suggested features remain relevant in the best feature set when GBM used as classifier model scoring 99.12. Table (4.25) reveal the best feature set names regarding GBM and XGBoost.

Table (4.25) Optimal Feature Set of XGBoost and GBM classifiers

GBM Best Scoring Feature Set		XGBoost Best Scoring Feature Set	
Feature subset (10)	**Feature Type**	**Feature subset (8)**	**Feature type**
Feature3	Numeric	Feature3	Numeric
Feature6	Numeric	Feature6	Numeric
Feature8	Binary	Feature9	Binary
Feature9	Binary	Feature15	Binary
Feature15	Binary	Feature22	Binary
Feature16	Binary	Feature23	Binary
Feature17	Binary	Feature24	Binary
Feature23	Binary	Feature25	Numeric
Feature24	Binary		
Feature25	Numeric		

4.9 Proposed System's Comparative Study

To compare with other studies, the latest papers and most relevant ones have been chosen. Additionally, the comparison between this proposed system and other studies are based on accuracy and evaluation metrics according to the results of other studies. Table (4.26) shows the comparison between this proposed system and the most relevant study.

Table (4.26) Comparison with the Most Relevant Study [12]

ML algorithms	Feature type	Acc.	TP	TN	FP	FN
LightGBM (Proposed method)	9 structural features	99.13	710	4816	11	37
GentleBoost [12]	Textual Features (32 words)	98.30	677	4800	25	70
TotalBoost [12]		98.17	682	4788	37	65
LogitBoost [12]		98.24	664	4810	15	83
LPBoost [12]		97.22	637	4780	45	110
AdaBoost [12]		93.95	416	4819	6	331
RobustBoost [12]		91.04	253	4820	5	494
RusBoost [12]		91.04	253	4820	5	494

Moreover, taking the most recent studies for comparison with this proposed system will be demonstrated in table (4.27). The outcome shows that LightGBM out performs the other study's approach with accuracy of 99.13%. However, the proposed system scores 0.98 of precision and 0.97 as F1-measure which is equal to the score of the other study's approach [17] has obtained.

Table (4.27) Comparative Study with the Most Recent Studies

ML algorithms	Feature type	Acc. %	Precision	F-measure
LightGBM (Proposed method)	9 structural features	99.13	0.98	0.97
Random Forest [17]	Textual features (TF-IDF)	97.50	0.98	0.97

Chapter Five
Conclusions and Suggestions for Future Work

5.1 Conclusions

SMS Spamming is considered as one of the momentous problems that are faced by the mobile users because of its low cost that made it easy for anyone to use it. Beside its benefits from business point of view, it can also frustrate mobile users with many problems. Detection of SMS spam messages is a big challenge nowadays. In this study, structural features that impact the process of classifying spam and normal messages are investigated. The idea of some of those features came from relevant studies in addition to creating and suggesting new ones. Extraction of 26 features performed containing 18 binary and 8 numeric features. Moreover, relevant features that lead to a better accuracy for filtering spam and ham messages determined using wrapper algorithm. For evaluating this study's approach, different evaluation metrics were used. In addition, classification time has also depended on when different models are compared because spam filtering should take less time in real world example.

The results of the experiments and tests of this study led to some conclusions which are summarized in the following points:

- The 18 binary and 8 numeric feature sets separately do not lead to a better performance compared to the feature set when both types of features are included.
- Wrapper approaches such as sequential feature selector algorithms are a good step when there is a duplicate or irrelevant feature to the machine learning problem. The results of the experiments show that SBS leads to select better relevant features compared to SFS because it determines less number of features with a higher performance. Furthermore, it appears that SBS is a good choice when it comes to find the best relevant features in a problem of spam detection. Since SBS selects the feature set with the highest accuracy.
- Gradient boosting classifiers such as GBM, XGBoost and LightGBM scores good performance in SMS spam classification problem, while the dataset is imbalanced and none of them is tuned with their best tuning parameters.

- In this study, 3 new features were proposed. Currency symbols, service numbers and currency as a word. These features were selected as relevant to SMS spam problem in the optimal feature subset which is determined by the wrapper SBS algorithm when the three classifiers such as GBM and LightGBM scored their best accuracy.
- The second most important conclusion is determining LightGBM as the best algorithm for classifying SMS spam messages. LightGBM achieved % 99.139 with 9 features that tops the other two algorithms in terms of accuracy and metrics that matter to SMS spam detection.
- Another important conclusion is drawn when the classification time was measured for each gradient boosting algorithm and LightGBM scored the best. While the training and testing within 10-fold cross-validation was performed for all the three algorithms, the LightGBM took shorter time than the other two in problem of classification.

5.2 Suggestions for Future Work

The following points are suggested for future study:

- Using the proposed model for developing a mobile application for SMS Spam detection because of its fast and accurate performance in detection of SMS spam messages.
- Creating more effective features which defines a spam SMS.
- Using black and white list number technique with the proposed system for filtering spam and ham messages.
- Improving the proposed system by making it more dynamic in terms of adding features and adding more machine learning algorithms or deep learning, and using different feature selection algorithms to select optimal feature subset.
- Creating a bigger dataset of SMS spam messages and making it available publicly in order to let other researchers use it.

References

[1] S. J. Delany, M. Buckley, and D. Greene, “SMS spam filtering: Methods and data,” *Expert Systems with Applications*, vol. 39, no. 10. pp. 9899–9908, Aug-2012, doi: 10.1016/j.eswa.2012.02.053.

[2] T. Almeida, “Towards SMS Spam Filtering: Results under a New Dataset,” *Int. J. Inf. Secur. Sci.*, vol. 2, no. 1, pp. 1–18, 2016.

[3] J. M. G. Hidalgo, G. C. Bringas, E. P. Sánz, and F. C. García, “Content based SMS spam filtering,” in *Proceedings of the 2006 ACM Symposium on Document Engineering, DocEng 2006*, 2006, vol. 2006, pp. 107–114, doi: 10.1145/1166160.1166191.

[4] Keymessagesapp.com, “Block Text, SMS, Spam Blocker - Key Messages - Apps on Google Play,” 2019. [Online]. Available: https://play.google.com/store/apps/details?id=com.smsBlocker&hl=en. [Accessed: 24-Jan-2020].

[5] “Mr. Number - Caller ID & Spam Protection - Apps on Google Play.” [Online]. Available: https://play.google.com/store/apps/details?id=com.mrnumber.blocker&hl=en. [Accessed: 24-Jan-2020].

[6] D. Puniškis, R. Laurutis, and R. Dirmeikis, “An artificial neural nets for spam e-mail recognition,” *Electron. Electr. Eng.*, vol. 5, no. 5, pp. 1215–1392, 2006.

[7] T. A. Almeida, J. M. G. Hidalgo, and A. Yamakami, “Contributions to the study of SMS spam filtering: New collection and results,” in *DocEng 2011 - Proceedings of the 2011 ACM Symposium on Document Engineering*, 2011, pp. 259–262, doi: 10.1145/2034691.2034742.

[8] K. Yadav, P. Kumaraguru, A. Goyal, A. Gupta, and V. Naik, “SMSAssassin: Crowdsourcing driven mobile-based system for SMS spam filtering,” in *HotMobile 2011: The 12th Workshop on Mobile Computing Systems and Applications*, 2011, pp. 1–6, doi: 10.1145/2184489.2184491.

[9] M. Taufiq Nuruzzaman, C. Lee, M. F. A. bin Abdullah, and D. Choi, “Simple SMS spam filtering on independent mobile phone,” *Secur. Commun. Networks*, vol. 5, no. 10, pp. 1209–1220, Oct. 2012, doi: 10.1002/sec.577.

[10] A. K. Uysal, S. Gunal, S. Ergin, and E. S. Gunal, “The impact of feature extraction and selection on SMS spam filtering,” *Elektron. ir Elektrotechnika*, vol. 19, no. 5, pp. 67–72, 2013, doi: 10.5755/j01.eee.19.5.1829.

[11] I. Ahmed, D. Guan, and T. C. Chung, “SMS Classification Based on Naïve Bayes Classifier and Apriori Algorithm Frequent Itemset,” *Int. J. Mach. Learn. Comput.*, vol. 4, no. 2, pp. 183–187, Apr. 2014, doi: 10.7763/ijmlc.2014.v4.409.

[12] F. Akbari and H. Sajedi, “SMS spam detection using selected text features and Boosting Classifiers,” in *2015 7th Conference on Information and Knowledge Technology, IKT 2015*, 2015, doi: 10.1109/IKT.2015.7288782.

[13] X. Zhang, G. Xiong, Y. Hu, F. Zhu, X. Dong, and T. R. Nyberg, “A method of SMS spam filtering based on AdaBoost algorithm,” in *Proceedings of the World Congress on Intelligent Control and Automation (WCICA)*, 2016, vol. 2016-Septe, pp. 2328–2332, doi: 10.1109/WCICA.2016.7578522.

[14] N. Choudhary and A. K. Jain, “Towards filtering of SMS spam messages using

machine learning based technique," in *Communications in Computer and Information Science*, 2017, vol. 712, pp. 18–30, doi: 10.1007/978-981-10-5780-9_2.

[15] J. Ma, Y. Zhang, Z. Wang, and B. Chen, "A new fine-grain SMS corpus and its corresponding classifier using probabilistic topic model," *KSII Trans. Internet Inf. Syst.*, vol. 12, no. 2, pp. 604–625, Feb. 2018, doi: 10.3837/tiis.2018.02.004.

[16] M. Popovac, M. Karanovic, S. Sladojevic, M. Arsenovic, and A. Anderla, "Convolutional Neural Network Based SMS Spam Detection," in *2018 26th Telecommunications Forum, TELFOR 2018 - Proceedings*, 2018, doi: 10.1109/TELFOR.2018.8611916.

[17] N. N. Amir Sjarif, N. F. Mohd Azmi, S. Chuprat, H. M. Sarkan, Y. Yahya, and S. M. Sam, "SMS Spam Message Detection using Term Frequency-Inverse Document Frequency and Random Forest Algorithm," *Procedia Comput. Sci.*, vol. 161, pp. 509–515, 2019, doi: 10.1016/j.procs.2019.11.150.

[18] M. A. Mohammad and A. Norhayati, "A short message service for campus wide information delivery," in *4th National Conference on Telecommunication Technology, NCTT 2003 - Proceedings*, 2003, pp. 216–221, doi: 10.1109/NCTT.2003.1188339.

[19] K. K. Veena and V. M. Thakare, "Short Message Service using SMS Gateway," *Int. J. Comput. Sci. Eng.*, vol. 2, 2010.

[20] "Web Menu Interface | NowSMS." [Online]. Available: https://www.nowsms.com/doc/web-menu-interface. [Accessed: 25-Jan-2020].

[21] "SMS Denial-of-Service Attack - Schneier on Security." [Online]. Available: https://www.schneier.com/blog/archives/2005/10/sms_denialofser_1.html. [Accessed: 25-Jan-2020].

[22] "SMS phone crash exploit a risk for older Nokias • The Register." [Online]. Available: https://www.theregister.co.uk/2001/12/06/sms_phone_crash_exploit/. [Accessed: 25-Jan-2020].

[23] B. Reaves, N. Scaife, D. Tian, L. Blue, P. Traynor, and K. R. B. Butler, "Sending Out an SMS: Characterizing the Security of the SMS Ecosystem with Public Gateways," in *Proceedings - 2016 IEEE Symposium on Security and Privacy, SP 2016*, 2016, pp. 339–356, doi: 10.1109/SP.2016.28.

[24] L. Aouad, A. Mosquera, S. Grzonkowski, and D. Morss, "SMS Spam: A Holistic View," 2014, doi: 10.5220/0005023202210228.

[25] A. Lambert, "Analysis of Spam," *Library (Lond).*, no. September, 2003.

[26] C. Wang *et al.*, "A behavior-based SMS antispam system," *IBM J. Res. Dev.*, vol. 54, no. 6, Nov. 2010, doi: 10.1147/JRD.2010.2066050.

[27] T. Yamakami, "Impact from mobile SPAM mail on mobile Internet services," *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, vol. 2745, pp. 179–184, 2003, doi: 10.1007/3-540-37619-4_19.

[28] D. Goel and A. K. Jain, "Mobile phishing attacks and defence mechanisms: State of art and open research challenges," *Computers and Security*, vol. 73. Elsevier Ltd, pp. 519–544, 01-Mar-2018, doi: 10.1016/j.cose.2017.12.006.

[29] E. O. Yeboah-Boateng and P. M. Amanor, "Phishing , SMiShing & Vishing : An Assessment of Threats against Mobile Devices," *J. Emerg. Trends Comput. Inf. Sci.*, vol. 5, no. 4, pp. 297–307, 2014.

[30] I. Guyon and A. Elisseefl, “An introduction to feature extraction,” *Stud. Fuzziness Soft Comput.*, vol. 207, pp. 1–25, 2006, doi: 10.1007/978-3-540-35488-8_1.

[31] ETSI, “Technical realization of the Short Message Service – Point to Point,” *GSM 03.40*, 1992.

[32] S. Günal, S. Ergin, M. B. Gülmezoğlu, and Ö. N. Gerek, “On feature extraction for spam e-mail detection,” in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 2006, vol. 4105 LNCS, pp. 635–642, doi: 10.1007/11848035_84.

[33] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Inf. Process. Manag.*, vol. 24, no. 5, pp. 513–523, 1988, doi: 10.1016/0306-4573(88)90021-0.

[34] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. USA: Cambridge University Press, 2008.

[35] I. Iguyon and A. Elisseeff, “An introduction to variable and feature selection,” *Journal of Machine Learning Research*, vol. 3. pp. 1157–1182, Mar-2003, doi: 10.1162/153244303322753616.

[36] G. Chandrashekar and F. Sahin, “A survey on feature selection methods,” *Comput. Electr. Eng.*, vol. 40, no. 1, pp. 16–28, Jan. 2014, doi: 10.1016/j.compeleceng.2013.11.024.

[37] E. Alpaydin, *Introduction to Machine Learning*. The MIT Press, 2014.

[38] M. H. C. Law, M. A. T. Figueiredo, and A. K. Jain, “Simultaneous feature selection and clustering using mixture models,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 26, no. 9, pp. 1154–1166, Sep. 2004, doi: 10.1109/TPAMI.2004.71.

[39] H. Liu and H. Motoda, *Computational Methods of Feature Selection (Chapman & Hall/Crc Data Mining and Knowledge Discovery Series)*. Chapman & Hall/CRC, 2007.

[40] J. Hastie, Trevor, Tibshirani, Robert, Friedman, *The Elements of Statistical Learning The Elements of Statistical LearningData Mining, Inference, and Prediction, Second Edition*. 2009.

[41] B. Sui, “INFORMATION GAIN FEATURE SELECTION BASED ON FEATURE INTERACTIONS.” 2013.

[42] H. Liu and L. Yu, “Toward integrating feature selection algorithms for classification and clustering,” *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 4, pp. 491–502, Apr. 2005, doi: 10.1109/TKDE.2005.66.

[43] J. Tang, S. Alelyani, and H. Liu, “Feature selection for classification: A review,” *Data Classif. Algorithms Appl.*, pp. 37–64, 2014.

[44] M. Dash and H. Liu, “Feature selection for classification,” *Intell. Data Anal.*, vol. 1, no. 3, pp. 131–156, 1997, doi: 10.3233/IDA-1997-1302.

[45] R. Kohavi and G. H. John, “Wrappers for feature subset selection,” *Artif. Intell.*, vol. 97, no. 1–2, pp. 273–324, Dec. 1997, doi: 10.1016/s0004-3702(97)00043-x.

[46] M. A. Hall and L. A. Smith, “Feature Selection for Machine Learning: Comparing a Correlation-Based Filter Approach to the Wrapper,” in *Proceedings of the Twelfth International Florida Artificial Intelligence Research Society Conference*, 1999, pp. 235–239.

[47] I. Inza, P. Larrañaga, R. Blanco, and A. J. Cerrolaza, “Filter versus wrapper gene selection approaches in DNA microarray domains,” *Artif. Intell. Med.*, vol. 31, no. 2, pp. 91–103, Jun. 2004, doi: 10.1016/j.artmed.2004.01.007.

[48] V. Kumar and S. Minz, “Feature selection: A literature review,” *Smart Comput. Rev.*, vol. 4, pp. 211–229, 2014, doi: 10.1145/2740070.2626320.

[49] H. Liu and H. Motoda, *Feature Selection for Knowledge Discovery and Data Mining*. USA: Kluwer Academic Publishers, 1998.

[50] P. Pudil, J. Novovičová, and J. Kittler, “Floating search methods in feature selection,” *Pattern Recognit. Lett.*, vol. 15, no. 11, pp. 1119–1125, 1994, doi: 10.1016/0167-8655(94)90127-9.

[51] J. Reunanen, “Overfitting in Making Comparisons between Variable Selection Methods,” *J. Mach. Learn. Res.*, vol. 3, no. null, pp. 1371–1382, Mar. 2003.

[52] A. A. Yildirim, C. Özdoğan, and D. Watson, “Parallel data reduction techniques for big datasets,” in *Big Data Management, Technologies, and Applications*, IGI Global, 2013, pp. 72–93.

[53] S. Raschka, “Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning,” Nov. 2018.

[54] L. Breiman, “Bias, Variance , And Arcing Classifiers,” *Tech. Rep. 460, Stat. Dep. Univ. Calif.*, 2000.

[55] Z.-H. Zhou, *Ensemble Methods: Foundations and Algorithms*, 1st ed. Chapman & Hall/CRC, 2012.

[56] R. E. Schapire, “Boosting: Foundations and Algorithms,” *Kybernetes*, vol. 42, no. 1. pp. 164–166, 2013, doi: 10.1108/03684921311295547.

[57] Zulaikha Lateef, “A Beginners Guide To Boosting Machine Learning Algorithms | Edureka.” [Online]. Available: https://www.edureka.co/blog/boosting-machine-learning/. [Accessed: 25-Jan-2020].

[58] A. Natekin and A. Knoll, “Gradient boosting machines, a tutorial,” *Front. Neurorobot.*, vol. 7, no. DEC, 2013, doi: 10.3389/fnbot.2013.00021.

[59] A. Bissacco, M.-H. Yang, and S. Soatto, “Fast Human Pose Estimation using Appearance and Motion via MultiDimensional Boosting Regression,” 2007.

[60] R. A. Hutchinson, L.-P. Liu, and T. G. Dietterich, “Incorporating Boosted Regression Trees into Ecological Latent Variable Models,” in *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence*, 2011, pp. 1343–1348.

[61] “GitHub - dmlc/xgboost: Scalable, Portable and Distributed Gradient Boosting (GBDT, GBRT or GBM) Library, for Python, R, Java, Scala, C++ and more. Runs on single machine, Hadoop, Spark, Flink and DataFlow.” [Online]. Available: https://github.com/dmlc/xgboost. [Accessed: 25-Jan-2020].

[62] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, vol. 13-17-Augu, pp. 785–794, doi: 10.1145/2939672.2939785.

[63] “ML | XGBoost (eXtreme Gradient Boosting) - GeeksforGeeks.” [Online]. Available: https://www.geeksforgeeks.org/ml-xgboost-extreme-gradient-boosting/. [Accessed: 25-Jan-2020].

[64] “XGBoost Parameters — xgboost 1.0.0-SNAPSHOT documentation.” [Online]. Available: https://xgboost.readthedocs.io/en/latest/parameter.html. [Accessed: 25-Jan-2020].

[65] G. Ke *et al.*, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, 2017, vol. 2017-Decem, pp. 3147–3155.

[66] Pranjal Khandelwal, “Which algorithm takes the crown: Light GBM vs XGBOOST?,” 2017. [Online]. Available: https://www.analyticsvidhya.com/blog/2017/06/which-algorithm-takes-the-crown-light-gbm-vs-xgboost/. [Accessed: 25-Jan-2020].

[67] “Welcome to LightGBM’s documentation! — LightGBM 2.3.2 documentation.” [Online]. Available: https://lightgbm.readthedocs.io/en/latest/. [Accessed: 25-Jan-2020].

[68] J. Novakovic, A. Veljovi, S. Iiic, Z. Papic, and M. Tomovic, “Evaluation of Classification Models in Machine Learning,” *Theory Appl. Math. Comput. Sci.*, vol. 7, no. 1, pp. 39–46, 2017.

[69] R. Kohavi and F. Provost, “Glossary of Terms,” *Mach. Learn.*, vol. 30, no. 2/3, pp. 271–274, 1998, doi: 10.1023/A:1017181826899.

Published Papers

Diyari Jalal Mussa and Noor Ghazi M. Jameel, **"***Relevant SMS Spam Feature Selection Using Wrapper Approach and XGBoost Algorithm,***"** at Kurdistan Journal of Applied Research (KJAR), Vol. 4, Issue 2, November 2019. Available online on: DOI: 10.24017/science.2019.2.11.

خلاصة

في العصر الحديث اصبح الهاتف المحمول جزءً من الحياة اليومية للانسان, وذلك عن طريق توفير العديد من الخدمات المختلفة مما جعلها مرغوبة للمستخدمين. خدمة الرسائل القصيرة (SMS)هي احدى مميزات التواصل الاساسية في الهواتف المحمولة, الاستخدام الاخلاقي والصحيح لهذة الخدمة يوفر العديد من الفوائد. ولاكن الاستخدام الواسع لهذة الخدمة لفت انظار المهاجمين الالكترونيين لنوع من انواع الهجوم وهي ارسال الرسائل القصيرة الغير مرغوبة او المزعجة. والتي هي نوع من الرسائل الغير المرغوبة ترسل عن طريق خدمة الرسائل القصيرة, في معظم الاوقات تعتبر رسائل السبام مزعجة او حتى مضرة لانها تثير اهتمام المستخدم لفتح مواقع تسهل انتشار التطبيقات المضرة و سرقة المعلومات الشخصية. الهدف الرئيسي من هذا البحث هو تمييز الرسائل القصيرة الغير مرغوبة او المزعجة من الرسائل الاعتيادية عن طريق خوارزمية التعليم الآلي (Gradient Boosting Machine Learning) و خوارزمية (Wrapper). ولتمييز هذه الرسائل , عدد من خصائص هذه الرسائل تم اختيارها من بحوث سابقة مع اقتراح خصائص جديدة, بهدف اختيار افضل مجموعة من الصفات وايجاد انسب حل للمشكلة,لأن احد اهم طرق التعليم الآلي هي اختيار الخصائص ذات صلة. والتي تم تصنيفها في هذه الدراسه الى ثلاثة مجموعات 18 ثنائيات, 8 رقميات و المجموعة الثالثة مكونة من الجمع بين المجموعتين السابقتين. اختيرت هذه الخصائص على اساس هيكلية ومحتوى الرسالة المؤذية مثل الاعلانات, فوز بجائزة, احرف كبيرة .. الخ. ومن اجل اختيار افضل مجموعة من الصفات , تمت مقارنة نوعين من طرق انتقاء الخصائص (Wrapper) مدعمة بأنواع من(Boosting Algorithm) .طريقة اختيار التسلل العكسي (SBS) اعطت نتائج افضل من اختيار التسلل الأمامي (SFS), تمت المقارنة بين طرق Gardient boosting machine (GBM)، XGBoosting و LightGBM على اساس دقة النتائج و الوقت عن طريق (10-Fold Cross-Validation). بأستخدام خوارزمية LightGBM تم الحصول على افضل نتائج بقدر 99.139% ب 9 خصائص اختيرت من قبل (SBS) بالمقارنة مع GBM وXGBoost. النظام المقترح تم تجربته على مجموعة بيانات تحتوي على 747 رسالة مؤذية و 4827 رسالة اعتيادية.النتائج تشير الى ان كل من الخصائص المقترحة : رمز العملات,رقم الخدمات, كلمة مال هي خصائص ذات صلة لانها اعطت نتائج دقة تصل الى 99.121% عن طريق استخدام مصنف LightGBM و 99.103% عن طريق استخدام مصنف XGBoost.ولكن مع ذالك خاصية رمز العملات و رقم الخدمات تعتبر كخصائص ذات صلة عندما تعطي خوارزمية LightGBM افضل النتائج.

نظام الكشف عن الرسائل غير المرغوب فيها باستخدام خوارزميات تعزيز التدرج الهجين مع تقنية التجميع

رسالة

مقدمة الى مجلس كلية التقنية معلوماتية في جامعة السليمانية التقنية

كجزء من متطلبات نيل شهادة الماجستير في تكنولوجيا المعلومات

من قبل

دياري جلال موسى

بكالوريوس في الاحصاء و الكمبيوتر – جامعة سليمانية ٢٠١٥

اشراف

د. نور غازي محمد جميل

استاذ مساعد

شباط ٢٠٢٠ صَفَر ١٤٤١

پوختە

لەسەردەمی ئەمڕۆدا، مۆبایل بووەتە بەشێکی سەرەکی ژیانی ڕۆژانەی مرۆڤ لە ڕێگەی بەردەستخستنی چەندین خزمەتگوزاری جیاواز بۆ بەکارهێنەر ئەمەش وادەکات خواستی زیاتری لەسەربێت. خزمەتگوزاری کورتەنامە (SMS) یەکێکە لە سەرەکیترین ئەو تایبەتمەندیانەی خراوەتە ژێر دەستی بەکارهێنەری مۆبایل. باش بەکارهێنانی ئەم خزمەتگوزاریە چەندین سودی جۆراوجۆری هەیە. بەڵام، بەکارهێنانی بەرفراوانی ئەم خزمەتگوزاریە سەرنجی چەندین هێرشبەری ئەلکترۆنی ڕاکێشاوە بۆ ئەنجامدانی سپامی مۆبایل، کە جۆرێکە لە نامەی بێزارکەر و لە ڕێگەی کورتەنامەوە ئاراستە دەکرێت. زۆرجار کورتەنامەی سپام بە بێزارکەر یان تەنانەت بە زیانبەخشیش هەژماردەکرێت لەبەرئەوەی سەرنجی بەکارهێنەر ڕادەکێشێت بۆ کردنەوەی چەند ماڵپەڕێک کە ڕێگەخۆشدەکات بۆ بڵاوکردەنەوەی ئەپڵیکەیشنی زیانبەخش و دزینی زانیاری تایبەتی کەسێتی. مەبەستی سەرەکی ئەم توێژینەوەیە جیاکردنەوەی کورتەنامەی زیانبەخشە لە کورتەنامەی ئاسایی لە ڕێگەی سیستمی فێرکردنی ئەلگۆریزمی بەهێزکردنی لاری (Gradient Boosting Algorithms)لەگەڵ ڕێگەی هەڵبژاردنی تایبەتمەندی کۆکردنەوە (Wrapper). ژمارەیەک تایبەتمەندی بونیادی لە توێژینەوەکانی پێشوتر هەڵبژێردرا و کۆکرانەوە لەگەڵ پێشنیارکردنی چەند تایبەتمەندیەکی نوێ، بە مەبەستی دۆزینەوەی باشترین سێتی تایبەتمەندی بۆ دەستنیشان کردنی باشترین چارەسەر بۆ کێشەکە. چونکە یەکێک لە گرنگترین هۆکارەکان بۆ بەرەوپێشبردنی سیستەمی فێرکردن بریتیە لە تایبەتمەندیە پەیوەندی دارەکان. کە کردوومانن بە سێ گرووپەوە 18 باینەری و 8 ژمارەیی و گروپی سێیەم بریتیە لە تێکەڵەیەک لە هەردوو تایبەتمەندیەکە. هەروەها تایبەتمەندیە دەرهێنراوەکان لەسەر بنچینەی ناوەڕۆکی کورتە نامە بێزارکەرەکە دیاریدەکرێت وەک ریکلام، بردنەوەی خەڵات، پیتی گەورە و هتد. بۆ هەڵبژاردنی باشترین تایبەتمەندی دوو جۆر شێوازی هەڵبژاردەی (Wrapper) بەراورد دەکرێت و بە چەند جۆرێک ئەلگۆریزمی بەهێزکردن پاڵپشتی دەکرێت. بە گوێرەی تاقیکردنەوەکانی ئەنجامماندا هەڵبژاردەی زنجیرەی هەڵگەڕاوە (SBS) ئەنجامی ڕاستری هەیە وەک لە هەڵبژاردەی زنجیرەی بەرەوپێش (SBS)، هەرسێ ئەلگۆریسمی Gradient Boosting Machine (GBM)، XGBoost و LightGBM لە ڕێگەی بەکارهێنانی (10-Fold Cross-Validation) لە ڕووی ڕاستی ئەنجام و کاتەوە بەراورد دەکرێن. LightGBM باشترین ئەنجامی هەبوو 99.139٪ بەدەستهێنا بە 9 تایبەتمەندیەوە کەلەلایەن SBS وە بۆی هەڵبژێردرا بە بەراورد بە XGBoost و GBM.بۆچوونە پێشنیازکراوەکە لەسەر داتاسێتێک ئەنجامدراوە کە 747 کورتەنامەی بێزارکەر و 4827 کورتە نامەی ئاسایی تێدایە. ئەنجامەکە ئاشکرایدەکات کە هەریەکە لە تایبەتمەندیەکانی سیمبۆڵی پارە، ژمارەی خزمەتگوزاری و وشەی پارە تایبەتمەندی پەیوەندیدارن لەبەر ئەوەی ئەنجامی 99.121٪ بەدەستهێناوە لە ڕێگەی بەکارهێنانی پۆلێنی LightGBM و 99.103٪ بەدەست هێناوە لە ڕێگەی بەکارهێنانی پۆلێنی XGBoost. بەڵام لەگەڵ ئەوەشدا تەنیا تایبەتمەندی سیمبۆڵی پارە و ژمارەی خزمەتگوزاری بە تایبەتمەندیەکی پەیوەندی دارهەژماردەکرێت کاتێک LightGBM باشترین ئەنجام بەدەست دەهێنێت.

سیستەمی ئاشکراکردنی کورتەنامەی نەخوازراو لەڕێگەی بەکارهێنانی ئەلگۆریزمی بەهێزکردنی لاری دووڕەگ و تەکنیکی کۆکردنەوە

نامەیەکە
پێشکەش کراوە بە ئەنجوومەنی کۆلێجی تەکنیکی ئینفۆرماتیک لە زانکۆی پۆلیتەکنیکی سلێمانی وەک
بەشێک لە پێداویستیەکانی بەدەستهێنانی بروانامەی ماستەر لە
تەکنەلۆژیای زانیاری

لە لایەن

دیاری جلال موسی

بەکالۆریۆس لە ئاماروکۆمپیوتەر - زانکۆی سلێمانی ٢٠١٥

بەسەرپەرشتی

د. نور غازی محمد جمیل

پرۆفیسۆری یاریدەدەر

ڕەشەمێ ٢٧٢٠ شوبات ٢٠٢٠